Advanced Programming in the Linux Environment

Volume VI

IPC and Terminals

Marco Bruno

Volume VI — IPC and Terminals

This chapter covers the main IPC mechanisms available in Linux, including POSIX message queues, semaphores, shared memory, and sockets. It also explores UNIX domain sockets in detail, along with terminal and pseudoterminal interfaces, providing numerous examples to illustrate their use in real-world scenarios.

Table of contents

Volume VI – Interprocess Communication and Terminals

VI.1 Introduction

Inter-Process Communication (IPC) refers to mechanisms that allow processes to communicate and coordinate with each other. Communication includes sending and receiving data, sharing resources, and synchronizing execution between processes.
IPC is essential because processes in modern operating systems are typically isolated for safety and stability, and therefore need controlled methods to exchange information.

In Linux, IPC can be implemented using several mechanisms:

- *Pipes*
  Used for unidirectional (half-duplex) communication between related processes (e.g., parent and child).
  - *Unnamed pipes*: typically created with pipe
  - *Named pipes (FIFOs)*: allow communication between unrelated processes

- *Message Queues*
  Allow processes to exchange messages in a structured and asynchronous way. Messages are stored in a queue and can be retrieved in order or based on priority

- *Semaphores*
  Used for process synchronization rather than data exchange. They help control access to shared resources and prevent race conditions

- *Shared Memory*
  The fastest IPC mechanism, where multiple processes share a common *memory segment*. Processes can read and write directly to this memory with synchronization (e.g., using semaphores), that is required to avoid conflicts

- *Sockets*
  Provide communication between processes, either on the same machine or over a network. They are commonly used for client-server communication

We have already examined some IPC mechanisms. In this section, we will focus on named semaphores, message queues, and shared memory, followed by an introduction to sockets.

Message queues, semaphores, and shared memory are part of XSI IPC, a standardized set of IPC mechanisms originally introduced in System V UNIX[1].

1 See the `sysvipc(7)` manual page

VI.2 POSIX Message Queues

Message queues are an IPC mechanism that allow processes or threads to communicate by sending and receiving messages through a queue managed by the operating system. The first implementation of message queues came from System V, but it had several limitations. POSIX message queues were later introduced to provide a simpler API, support message priorities, and offer better integration with modern system features like asynchronous notification.

POSIX message queues are widely used in modern applications. They provide a file-like interface with operations such as `open`, `close`, and `unlink`. Furthermore, they support message priorities directly and integrate seamlessly with I/O multiplexing mechanisms like `select` and `poll` via file descriptors.

Message queues are created and opened using `mq_open`. This function returns a *message queue descriptor* (`mqd_t`), which is used in subsequent operations to refer to the opened queue.
Each message queue is identified by a name of the form `/somename`, which is a null-terminated string of up to `NAME_MAX` (typically 255) characters. The name must begin with a forward slash (`/`) and must not contain any additional slashes. Two processes can communicate using the same queue by passing the same name to `mq_open`.

Messages are sent and received using `mq_send` and `mq_receive`. When a process no longer needs a queue, it closes it using `mq_close`. The queue itself can be removed from the system using `mq_unlink`; when it is no longer required. Queue attributes can be retrieved and, in some cases, modified using `mq_getattr` and `mq_setattr`. Additionally, a process can request asynchronous notification of message arrival on an empty queue using `mq_notify`.

A *message queue descriptor* refers to an *open message queue description*. After a `fork`, the child process inherits copies of its parent's message queue descriptors, and both processes refer to the same underlying open queue. The associated flags (`mq_flags`) are shared between corresponding descriptors.

Each message has an associated *priority*, and messages are delivered to the receiving process in order of highest priority first. Message priorities range from 0 (lowest) to `sysconf(_SC_MQ_PRIO_MAX) - 1` (highest). This value is typically `32768`[2].

Message queues are created in a virtual filesystem that can be mounted using the commands:

```
[linux@ipc] sudo mkdir /dev/mqueue

[linux@ipc] sudo mount -t mqueue none /dev/mqueue
```

On the mount directory the *sticky bit* is automatically enabled. The message queues on the system can be viewed and manipulated using the commands usually used for files such as `ls` and `rm`. Each file consists in a single line containing information about the queue

```
[linux@ipc] cat /dev/mqueue/mymq
        QSIZE:129     NOTIFY:2    SIGNO:0    NOTIFY_PID:8260
```

2 POSIX only requires support for at least priorities in the range 0 to 31

The fields are:

`QSIZE`
: Number of bytes of data in all messages in the queue

`NOTIFY_PID`
: If this is nonzero, then the process with this PID has used mq_notify to register for asynchronous message notification, the remaining fields describe how notification occurs

`NOTIFY`
: Notification method:
0 → `SIGEV_SIGNAL`
1 → `SIGEV_NONE`
2 → `SIGEV_THREAD`

`SIGNO`
: Signal number to be used for `SIGEV_SIGNAL`

Message queues use cases

Message queues can be used in a wide variety of applications, particularly those requiring *asynchronous communication* and coordination between processes.

Print spooler system
In such a system, multiple applications (processes) may request printing services, while a dedicated printer daemon is responsible for managing the physical printing device.
Message queues facilitate communication between these components by organizing the system into producers and a consumer. The applications act as producers, generating print requests, while the printer daemon acts as the consumer, processing those requests.
The workflow proceeds as follows. Applications submit print jobs by sending messages to a message queue. Each message typically contains relevant information such as the file name, the number of copies to be printed, and possibly a priority value encoded in the message type (`mtype`). The printer daemon retrieves messages from the queue and processes them sequentially or according to their priority.
This approach decouples the sending and receiving processes. Applications do not need to wait for the printing operation to complete; instead, they continue execution immediately after submitting their request. The printer daemon handles all jobs asynchronously, improving overall system efficiency.

Task / Job Queues
In a task distribution systems a server can enqueue tasks, such as image processing requests, while multiple worker processes dequeue and execute them. This model resembles modern job processing systems and enables scalable workload distribution.

Logging Systems
In a logging architecture, multiple processes can send log messages to a message queue, while a dedicated logging process retrieves these messages and writes them to disk. This design prevents concurrent write conflicts and centralizes logging functionality.

Client–Server communication on a single machine
Message queues can also be used to implement client–server communication on the same machine. Clients send requests to a queue, and the server process reads and processes them, optionally sending responses back. This provides a lightweight alternative to sockets for local inter-process communication.

VI.2.1 Creating a message queue: `mq_open`, `mq_close`, `mq_unlink`

The library functions are implemented on top of underlying system calls as listed in the following table.[3]

Library interface	System call
`mq_close(3)`	`close(2)`
`mq_getattr(3)`	`mq_getsetattr(2)`
`mq_notify(3)`	`mq_notify(2)`
`mq_open(3)`	`mq_open(2)`
`mq_receive(3)`	`mq_timedreceive(2)`
`mq_send(3)`	`mq_timedsend(2)`
`mq_setattr(3)`	`mq_getsetattr(2)`
`mq_timedreceive(3)`	`mq_timedreceive(2)`
`mq_timedsend(3)`	`mq_timedsend(2)`
`mq_unlink(3)`	`mq_unlink(2)`

Tab 3. POSIX Message queues library functions

To create/open a message queue we can use the `mq_open`[4] function.

```
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

mqd_t mq_open(const char *name, int oflag);

mqd_t mq_open(const char *name, int oflag, mode_t mode, struct mq_attr
              *attr);

       Return: a message queue descriptor, on error (mqd_t) -1 and errno is set
```

The `mq_open` function creates a new message queue or opens an existing queue. The queue is identified by `name`.

The `oflag` argument specifies the flags that control the operation as in `open`, and exactly only one flag must be specified.
The `mode` argument, in the case of `O_CREAT`, specifies the permissions for the new queue.

The `attr` argument is a structure defined as follows

3 See the `mq_overview(7)` and `mqueue.h(7POSIX)` manual pages
4 See the `mq_open(3)` manual page

```
struct mq_attr {
   long mq_flags;      /* Flags (ignored for mq_open()) */
   long mq_maxmsg;     /* Max. # of messages on queue */
   long mq_msgsize;    /* Max. message size (bytes) */
   long mq_curmsgs;    /* # of messages currently in queue ignored for
                          mq_open() */
};
```

When calling `mq_open`, only `mq_maxmsg` and `mq_msgsize` are employed, the other fields are ignored. If `attr` is `NULL`, then the queue is created with implementation-defined default attributes.

A descriptor returned by `mq_open` has the *close-on-exec* flag automatically set.

To close a message queue we can use `mq_close`[5].

```
#include <mqueue.h>

int mq_close(mqd_t mqdes);

                              Returns: 0 on success, -1 on error and errno is set
```

The `mq_close` function closes the message queue descriptor `mqdes`. If the calling process has attached a notification request (`mq_notify`) to this message queue via `mqdes`, then this request is removed.

To remove a message queue we can use `mq_unlink`[6].

```
#include <mqueue.h>

int mq_unlink(const char *name);

                              Returns: 0 on success, -1 on error and errno is set
```

The `mq_unlink` function removes the specified message queue name. The message queue name is removed immediately. The queue itself is destroyed once any other processes that have the queue open close their descriptors referring to the queue.

5 See the `mq_close(3)` manual page
6 See the `mq_unlink(3)` manul page

VI.2.2 Sending and receiving messages: `mq_send` and `mq_receive`

To send a message to a message queue we can use the `mq_send`[7] function, defined as follows.

```
#include <mqueue.h>
#include <time.h>

int mq_send(mqd_t mqdes, const char msg_ptr[msg_len], size_t msg_len,
            unsigned int msg_prio);

int mq_timedsend(mqd_t mqdes, const char msg_ptr[msg_len], size_t
                 msg_len, unsigned int msg_prio, const struct timespec
                 *abs_timeout);

                              Return: on success 0, on error -1 and errno is set
```

The `mq_send` function inserts the message pointed to by `msg_ptr` into the message queue identified by the descriptor `mqdes`. The `msg_len` argument specifies the size of the message and must be less than or equal to the queue's `mq_msgsize` attribute. Messages of zero length are permitted.
The `msg_prio` argument specifies the message priority as a nonnegative integer. Messages are ordered in the queue by decreasing priority; messages with the same priority are delivered in FIFO order, with newer messages placed after older ones.
If the queue is full (i.e., the number of messages equals the `mq_maxmsg` limit), the default behavior of `mq_send` is to block until space becomes available or the call is interrupted by a signal. If the `O_NONBLOCK` flag is set for the queue, the call does not block and instead fails immediately with the error `EAGAIN`.
The `mq_timedsend` function behaves similarly to `mq_send`, but allows a timeout to be specified. If the queue is full and `O_NONBLOCK` is not set, the call blocks only until the absolute timeout specified by `abs_timeout` is reached.
If the timeout has already expired at the time of the call, `mq_timedsend` returns immediately without sending the message.

To receive a message from the message queue we can use the `mq_receive`[8] function, defined as follows

```
#include <mqueue.h>
#include <time.h>

ssize_t mq_receive(mqd_t mqdes, char msg_ptr[msg_len], size_t msg_len,
                   unsigned int *msg_prio);

ssize_t mq_timedreceive(mqd_t mqdes, char *restrict msg_ptr[msg_len],
                        size_t msg_len, unsigned int *restrict msg_prio,
                        const struct timespec *restrict abs_timeout);

     Return: the number of bytes in the received message, on error -1 and errno
```

7 See the `mq_send(3)` manual page
8 See the `mq_receive(3)` manual page

The mq_receive function retrieves and removes the oldest message with the highest priority from the message queue identified by the descriptor mqdes, and stores it in the buffer pointed to by msg_ptr. The msg_len argument specifies the size of this buffer and must be greater than or equal to the queue's mq_msgsize attribute.
If msg_prio is not NULL, it is used to return the priority of the received message.
If the queue is empty, the default behavior of mq_receive is to block until a message becomes available or the call is interrupted by a signal. If the O_NONBLOCK flag is set, the function does not block and instead fails immediately with the error EAGAIN.

The mq_timedreceive function behaves similarly to mq_receive, but allows a timeout to be specified. If the queue is empty and O_NONBLOCK is not set, the call blocks only until the absolute timeout specified by abs_timeout.
If no message is available and the timeout has already expired at the time of the call, mq_timedreceive returns immediately.

The mq_send.c and mq_recv.c programs uses a POSIX message queue. mq_recv.c waits for a message to be received, mq_send.c sends the message inserted by the user.

```c
#include <stdio.h>
#include <stdlib.h>
#include <mqueue.h>
#include <fcntl.h>
#include <sys/stat.h>

#define QUEUE_NAME "/myqueue"
#define MAX_SIZE 128

int main() {
    mqd_t mq;
    char buffer[MAX_SIZE];
    unsigned int priority;
    struct mq_attr attr;

    attr.mq_flags = 0;
    attr.mq_maxmsg = 10;
    attr.mq_msgsize = MAX_SIZE;
    attr.mq_curmsgs = 0;

    mq = mq_open(QUEUE_NAME, O_CREAT | O_RDONLY, 0644, &attr);
    if (mq == (mqd_t) -1) {
        perror("mq_open");
        exit(1);
    }
    if (mq_receive(mq, buffer, MAX_SIZE, &priority) == -1) {
        perror("mq_receive");
        exit(1);
    }

    printf("Received message: %s", buffer);
```

```
27    printf("Priority: %u\n", priority);
28    mq_close(mq);
29    mq_unlink(QUEUE_NAME);
30    return 0;
31 }
```

---

mq_recv.c Retrieves a message from the queue

At line 6, the name of the message queue is defined. On Linux systems, this queue will appear as an entry under `/dev/mqueue`.

At line 12, a `struct mq_attr` variable is declared to specify the attributes of the message queue.
In lines 13–16, the queue attributes are initialized:

- `mq_maxmsg` is set to 10, defining the maximum number of messages the queue can hold
- `mq_msgsize` is set to 128 bytes, defining the maximum size of each message
- `mq_flags` is set to 0, meaning blocking mode is used by default
- `mq_curmsgs` is initialized to 0 (this field is ignored when creating the queue)

The queue is then created (or opened) with the call

```
17    mq = mq_open(QUEUE_NAME, O_CREAT | O_RDONLY, 0644, &attr);
```

This call creates the queue `/myqueue` (visible as `/dev/mqueue/myqueue`) if it does not already exist, using the specified permissions and attributes. If the queue already exists, it is simply opened, and the `attr` structure is ignored.

Next, the program attempts to receive a message using

```
22    if (mq_receive(mq, buffer, MAX_SIZE, &priority) == -1) {
```

The arguments are

`mq`	the message queue descriptor
`buffer`	the buffer where the received message will be stored
`MAX_SIZE`	the size of the buffer (which must be at least the queue's `mq_msgsize`)
`priority`	a variable used to retrieve the message priority

Since the queue is opened in blocking mode and no `O_NONBLOCK` flag is specified, this call blocks until a message becomes available. After successfully receiving the message, the program prints its contents and priority.

At line 28, the queue descriptor is closed using `mq_close`
At line 29, the queue is removed from the system using `mq_unlink`. This deletes the queue name, and the queue itself is destroyed once no processes are using it.

```
1 #include <stdio.h>
2 #include <stdlib.h>
```

```c
3 #include <string.h>
4 #include <mqueue.h>
5 #include <fcntl.h>

6 #define QUEUE_NAME "/myqueue"
7 #define MAX_SIZE 128

8 int main() {
9     mqd_t mq;
10    struct mq_attr attr;

11    mq = mq_open(QUEUE_NAME, O_WRONLY, 0644, &attr);
12    if (mq == (mqd_t) -1) {
13          perror("mq_open");
14          exit(1);
15    }

16    char message[MAX_SIZE];
17    printf("Enter message: ");
18    fgets(message, MAX_SIZE, stdin);
19    if (mq_send(mq, message, strlen(message) + 1, 1) == -1) {
20          perror("mq_send");
21          exit(1);
22    }
23    printf("Message sent: %s\n", message);
24    mq_close(mq);
25    return 0;
26 }
```

---

mq_send.c Sends a message to the queue

At line 6, the name of the message queue is defined. This must match the name used by the receiver so that both processes operate on the same queue.

At line 10, a `struct mq_attr` variable is declared. However, in this program it is not actually used, since the queue is opened without the `O_CREAT` flag. Therefore, the `attr` argument passed to `mq_open` is ignored and could be replaced with `NULL`.

The queue is then opened using:

```c
11    mq = mq_open(QUEUE_NAME, O_WRONLY, 0644, &attr);
```

This call opens an existing message queue in write-only mode. It assumes that the queue has already been created (for example, by the receiver). If the queue does not exist, the call fails.

In lines 16–18, the program reads a message from standard input using `fgets` and stores it in the `message` buffer.

The message is then sent to the queue using:

```c
19    if (mq_send(mq, message, strlen(message) + 1, 1) == -1) {
```

The arguments are

- `mq` the message queue descriptor
- `message`: the buffer containing the message
- `strlen(message) + 1` the size of the message (including the null terminator)
- `1` the priority assigned to the message

The message is inserted into the queue and will be delivered according to its priority.
Finally, at line 24, the queue descriptor is closed using `mq_close`.

Let's execute the programs

Shell 1

```
[linux@ipc] ./mq_recv
```

Shell 2

```
[linux@ipc] ./mq_send
Enter message: hello receiver!
Message sent: hello receiver!
```

Shell 1

```
Received message: hello receiver!
Priority: 1
```

After the sender sends the message, the receiver reads it from the queue and prints its contents.

If we comment out the `mq_unlink` call, we can see the `myqueue` file under `/dev/mqueue`.

```
[linux@ipc] cat /dev/mqueue/myqueue
QSIZE:0         NOTIFY:0     SIGNO:0     NOTIFY_PID:0
```

In this example, the message queue is created by the receiver, while the sender simply uses the existing queue. This is a design choice made for illustrative purposes. In practice, the specific process responsible for creating the queue is less important than ensuring that all participating components can access and use it correctly.
For instance, the queue could instead be created by the sender, which might even send messages before the receiver starts executing. Alternatively, both the sender and the receiver could attempt to create the queue using the `O_CREAT` flag; since the queue is only created if it does not already exist, this approach improves robustness by removing dependencies on process startup order.
In this example, however, creating the queue in the receiver allows us to demonstrate that `mq_receive` blocks when no messages are available, waiting until a message is delivered.
The process that creates the queue defines its attributes, while other processes can retrieve these attributes as needed.

VI.2.3 Message queue attributes: `mq_getattr` and `mq_setattr`

The `mq_getattr` and `mq_setattr` functions[9] respectively retrieve and modify attributes of the message queue referred to by the message queue descriptor `mqdes`.

```
#include <mqueue.h>

int mq_getattr(mqd_t mqdes, struct mq_attr *attr);

int mq_setattr(mqd_t mqdes, const struct mq_attr *restrict newattr,
               struct mq_attr *restrict oldattr);

                                Return: on success 0, on error -1 and errno is set
```

The `mq_getattr` and `mq_setattr` functions, respectively retrieve and modify attributes of the message queue referred to by the message queue descriptor `mqdes`.

`mq_getattr` returns an `mq_attr` structure in the buffer pointed by `attr`. In the `mq_attr` structure, the `mq_flags` field contains flags associated with the open message queue description. This field is initialized when the queue is created by `mq_open`. The only flag this field can have is `O_NONBLOCK`.
Also `mq_maxmsg` and `mq_msgsize` are set when the message queue is created. `mq_maxmsg` is the upper limit on the number of messages that may be placed on the queue using `mq_send`. `mq_msgsize` is the upper limit on the size of messages that may be placed on the queue. Both of these fields must have a value greater than zero.
The `mq_curmsgs` field returns the number of messages currently held in the queue.

The `mq_setattr` function sets message queue attributes using information supplied in the `mq_attr` structure pointed to by `newattr`. The only attribute that can be modified is the setting of the `O_NONBLOCK` flag in `mq_flags`, the other fields in `newattr` are ignored. If the `oldattr` field is not `NULL`, then the buffer that it points to is used to return an `mq_attr` structure that contains the same information that is returned by `mq_getattr`.

The `mqgetattr.c` program is taken from the `mq_getattr(2)` manual page. It shows how to retrieve the attributes associated to a message queue created by passing the `attr` argument as `NULL`.

```
1 #include <err.h>
2 #include <fcntl.h>
3 #include <mqueue.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <unistd.h>

7 int main(int argc, char *argv[])
8 {
9     mqd_t mqd;
```

9 See the `mq_getattr(3)` manual page

```
10    struct mq_attr attr;

11    if (argc != 2) {
12          fprintf(stderr, "Usage: %s mq-name\n", argv[0]);
13          exit(EXIT_FAILURE);
14    }

15    mqd = mq_open(argv[1], O_CREAT | O_EXCL, 0600, NULL);
16    if (mqd == (mqd_t) -1)
17          err(EXIT_FAILURE, "mq_open");

18    if (mq_getattr(mqd, &attr) == -1)
19          err(EXIT_FAILURE, "mq_getattr");

20    printf("Maximum # of messages on queue: %ld\n", attr.mq_maxmsg);
21    printf("Maximum message size: %ld\n", attr.mq_msgsize);
22    if (mq_unlink(argv[1]) == -1)
23          err(EXIT_FAILURE, "mq_unlink");
24    exit(EXIT_SUCCESS);
25 }
```

---

mqgetattr.c

The program creates a message queue with the name specified on the command line. When `NULL` is passed as the `attr` argument, the queue is initialized with default attributes. These attributes are subsequently retrieved and printed by the program.

The call

```
18    if (mq_getattr(mqd, &attr) == -1)
```

retrieves the attributes associated with the queue `mqd`.
Before removing the queue, the program prints the values of the `mq_maxmsg` and `mq_msgsize` attributes.

Let's execute

```
[linux@ipc] ./mqgetattr /mynewq
Maximum # of messages on queue:   10
Maximum message size:             8192
```

We can check these two values in the `/proc/sys/fs/mqueue/msg_default` and `/proc/sys/fs/mqueue/msgsize_max` files.

Both `mq_getattr` and `mq_setattr` are implemented on the top of the `mq_getsetattr` system call. For further information about this system call the reader is encouraged to read the `mq_getsetattr(2)` manual page.

As previously noted, message priority determines the order in which messages are retrieved from the queue. Messages with higher priority values are delivered before those with lower priorities. POSIX message queues operate as priority-based queues rather than simple FIFO queues.

The `mqsndmsg.c` and `mqrcvmsg.c` programs allows the user to send messages with different priorities. `mqsndmsg.c` takes from the command-line the name of the queue, the message and the priority.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <mqueue.h>
5 #include <fcntl.h>
6 #include <err.h>

7 #define MAX_SIZE 128

8 int main(int argc, char *argv[]) {
9     mqd_t mq;
10    struct mq_attr attr;
11    char *message;

12    if (argc != 4) {
13          fprintf(stderr, "Usage: %s <mq-name> <msg> <pri> \n",
                    argv[0]);
14          exit(EXIT_FAILURE);
15    }

16    attr.mq_flags = 0;
17    attr.mq_maxmsg = 10;
18    attr.mq_msgsize = MAX_SIZE;
19    attr.mq_curmsgs = 0;

20    message = argv[2];
21    printf("%s %d", message, strlen(message));

22    mq = mq_open(argv[1], O_CREAT | O_WRONLY, 0644, NULL);
23    if (mq == (mqd_t) -1) {
24          perror("mq_open");
25          exit(1);
26    }
27    if (mq_send(mq,message,strlen(message) + 1, atoi(argv[3])) == -1) {
28          perror("mq_send");
29          exit(1);
30    }
31    mq_close(mq);
32    return 0;
33 }
```

mqsndmsg.c Sender

After setting the attributes, the program creates/opens the queue, sends the message and closes the queue.

```c
#include <stdio.h>
#include <stdlib.h>
#include <mqueue.h>
#include <fcntl.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    mqd_t mq;
    void *buffer;
    unsigned int priority;
    struct mq_attr attr;

    if (argc != 2) {
         fprintf(stderr, "\nUsage: %s <mq-name>", argv[0]);
         exit(EXIT_FAILURE);
    }

    mq = mq_open(argv[1], O_RDONLY, 0644, &attr);
    if (mq == (mqd_t) -1) {
         perror("mq_open");
         exit(1);
    }

    if (mq_getattr(mq, &attr) == -1) {
         perror("mq_getattr");
         exit(EXIT_FAILURE);
    }

    buffer = malloc(attr.mq_msgsize);
    if (buffer == NULL) {
         perror("malloc");
         exit(EXIT_FAILURE);
    }
    if (mq_receive(mq, buffer, attr.mq_msgsize, &priority) == -1) {
         perror("mq_receive");
         exit(1);
    }

    printf("Received message: %s ", buffer);
    printf("Priority: %u\n", priority);
    mq_close(mq);
    free(buffer);
    return 0;
}
```

mqrcvmsg.c Receiver

The receiver opens the queue at line 15, and retrieves the attributes at line 20 using `mq_getattr`.
Next, at line 24, it allocates memory for the buffer using the `mq_msgsize` field of the `attr` structure.
At line 29, it reads the message from the queue using `mq_receive`.
In lines 35-36, the program closes the queue and frees the memory.

Let’s execute.

```
[linux@ipc] ./mqsndmsg /newqu "msg-a" 5
msg-a 5

[linux@ipc] ./mqsndmsg /newqu  "msg-b" 1
msg-b 5

[linux@ipc] ./mqsndmsg /newqu "msg-c" 10
msg-c 5

[linux@ipc] ./mqsndmsg /newqu "msg-d" 3
msg-d 5

[linux@ipc]  ./mqrcvmsg /newqu
Received message: msg-c Priority: 10

[linux@ipc]  ./mqrcvmsg /newqu
Received message: msg-a Priority: 5

[linux@ipc]  ./mqrcvmsg /newqu
Received message: msg-d Priority: 3

[linux@ipc]  ./mqrcvmsg /newqu
Received message: msg-b Priority: 1
```

Messages are read from the queue based on their priority.

VI.2.4 Message queue notification: `mq_notify`

A process can register to receive asynchronous notifications when a message arrives in the message queue. The `mq_notify`[10] function alllows the calling process to register or unregister for delivery of an asynchronous notification.

```
#include <mqueue.h>
#include <signal.h>

int mq_notify(mqd_t mqdes, const struct sigevent *sevp);

                                    Returns: on success 0, on error -1 and errno
```

`mqdes` is the message queue descriptor.
The `sevp` argument is a pointer to a `sigevent` structure. If this parameter is a non-null pointer, `mq_notify` registers the process to receive notifications. The `sigev_notify` of `sevp` specifies how notification is performed. It can have one of the following value:

`SIGEV_NONE`
: A "null" notification: the calling process is registered as the target for notification, but when a message arrives, no notification is sent

`SIGEV_SIGNAL`
: Notify the process by sending the signal specified in `sigev_signo`. The `si_code` field of the `siginfo_t` structure will be set to `SI_MESGQ`. In addition, `si_pid` will be set to the PID the process that sent the message, and `si_uid` will be set to the real user ID of the sending process

`SIGEV_THREAD`
: Upon message delivery, invoke `sigev_notify_function` as if it were the start function of a new thread

Only one process can be registered to receive notification from a message queue.
If `sevp` is `NULL` and the calling process is currently registered to receive notifications for the message queue, the registration is removed. This allows another process to register for message notifications on the same queue.
A notification is generated only when a new message arrives in an empty queue. If the queue is not empty at the time `mq_notify` is called, a notification will be triggered only after the queue has been emptied and a new message arrives.

If another process or thread is already blocked in `mq_receive` waiting for a message from an empty queue, the notification mechanism is bypassed. In this case, the message is delivered directly to the waiting process or thread, and the notification registration remains active.

Notifications are delivered only once. After a notification is triggered, the registration is automatically removed, allowing another process to register. If the same process wishes to receive further notifications, it must call `mq_notify` again. This re-registration should typically occur before all pending messages are consumed. Using nonblocking mode can help empty the queue without blocking when no messages remain.

---

10 See the `mq_notify(3)` manual page

VI.2.5 Notification via thread

The `mqnotify.c` program is taken from the `mq_notify(3)` manual page, it registers a notification request for the message queue specified as a command-line argument. Notification is implemented by creating a thread, which executes a handler function that reads one message from the queue and then terminates the process.

```
1 #include <err.h>
2 #include <mqueue.h>
3 #include <pthread.h>
4 #include <signal.h>
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <unistd.h>

8 static void tfunc(union sigval sv)
9 {
10    struct mq_attr attr;
11    ssize_t nr;
12    void *buf;
13    mqd_t mqdes = *((mqd_t *) sv.sival_ptr);

14    if (mq_getattr(mqdes, &attr) == -1)
15         err(EXIT_FAILURE, "mq_getattr");
16    buf = malloc(attr.mq_msgsize);
17    if (buf == NULL)
18         err(EXIT_FAILURE, "malloc");

19    nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
20    if (nr == -1)
21         err(EXIT_FAILURE, "mq_receive");

22    printf("Read %zd bytes from MQ\n", nr);
23    free(buf);
24    exit(EXIT_SUCCESS);
25 }

26 int main(int argc, char *argv[])
27 {
28    mqd_t mqdes;
29    struct sigevent sev;

30    if (argc != 2) {
31         fprintf(stderr, "Usage: %s <mq-name>\n", argv[0]);
32         exit(EXIT_FAILURE);
33    }

34    mqdes = mq_open(argv[1], O_RDONLY);
35    if (mqdes == (mqd_t) -1)
```

```
36          err(EXIT_FAILURE, "mq_open");
37      sev.sigev_notify = SIGEV_THREAD;
38      sev.sigev_notify_function = tfunc;
39      sev.sigev_notify_attributes = NULL;
40      sev.sigev_value.sival_ptr = &mqdes;
41      if (mq_notify(mqdes, &sev) == -1)
42          err(EXIT_FAILURE, "mq_notify");

43      pause();
44  }
```

---

mqnotify.c Notification

At line 29, a `struct sigevent` variable is declared. This structure is used to specify how the process will be notified when a message arrives in the queue.
Next, the program opens the message queue using:

```
34      mqdes = mq_open(argv[1], O_RDONLY);
```

This call opens an existing message queue in read-only mode. It assumes that the queue has already been created; otherwise, the call fails.

In lines 37–40, the `sigevent` structure is initialized:

- `sigev_notify` is set to `SIGEV_THREAD`, indicating that notification will be delivered by creating a new thread
- `sigev_notify_function` is set to `tfunc`, which will be executed when a message arrives
- `sigev_notify_attributes` is set to `NULL`, meaning default thread attributes are used
- `sigev_value.sival_ptr` is set to the address of `mqdes`, allowing the thread function to access the queue descriptor

The program then registers for notification using:

```
41      if (mq_notify(mqdes, &sev) == -1)
```

This call associates the notification request with the queue. When a message arrives on an empty queue, the kernel will create a new thread and invoke `tfunc`.
The call to `pause` at line 43 suspends the main thread indefinitely, waiting for a signal. In practice, the process will terminate when the notification thread calls `exit`.

The function `tfunc` is executed in a new thread when a notification is triggered.
At line 13, the queue descriptor is retrieved from the `sigevent` structure:

```
13      mqd_t mqdes = *((mqd_t *) sv.sival_ptr);
```

The `sival_ptr` field is cast to a pointer to `mqd_t`, and its value is dereferenced to obtain the queue descriptor.
Next, the function retrieves the queue attributes:

```
14      if (mq_getattr(mqdes, &attr) == -1)
```

This is necessary to determine the maximum message size (`mq_msgsize`), which is used to allocate a suitable buffer.
At line 16, memory is allocated dynamically with `malloc`

```
16    buf = malloc(attr.mq_msgsize);
```

This ensures the buffer is large enough to hold any message from the queue.
The function then calls:

```
19    nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
```

This retrieves a message from the queue. In this context, the call should not block because the notification is only triggered when a message is available.

At line 22, the number of bytes read is printed.
At line 23, the allocated buffer is freed.
At line 24, `exit(EXIT_SUCCESS)` is called to terminate the entire process including the main thread.

To send a message to this program, we can use the `mq_send.c` program. We add the `O_CREAT` flag to the `mq_open` call of `mq_send.c` as follows

```
11    mq = mq_open(QUEUE_NAME, O_CREAT | O_WRONLY, 0644, &attr);
```

Let's execute

Shell 1

```
[linux@ipc] ./mq_send /myqueue
Enter message:
```

Shell 2

```
[linux@ipc] ./mqnotify /myqueue
```

Shell 1

```
Enter message: hello notification program!!
```

Shell 2

```
Read 30 bytes from MQ
```

The execution shows that when a message is sent to an empty queue, the registered notification is triggered, causing the thread function to execute and read the message.

The program demonstrates *asynchronous notification* using `mq_notify` with `SIGEV_THREAD`, where message arrival triggers the execution of a handler function in a new thread. Notification occurs only when a message arrives in an empty queue, ensuring that the handler is invoked exactly once per registration.

VI.2.6 Notification via signal

The `mqsig.c` program demonstrates *asynchronous notification* using `mq_notify` with `SIGEV_SIGNAL`, where message arrival triggers the delivery of a signal. The *signal handler* is responsible for reading the message from the queue and terminating the process. As with all `mq_notify` usage, the notification is delivered only once and only when a message arrives in an empty queue.

```c
#include <stdio.h>
#include <stdlib.h>
#include <mqueue.h>
#include <signal.h>
#include <fcntl.h>
#include <unistd.h>

#define QUEUE_NAME "/myqueue"
mqd_t mqdes;

void handler(int sig) {
    struct mq_attr attr;
    char *buf;
    ssize_t nr;

    printf("Signal received: message available\n");
    if (mq_getattr(mqdes, &attr) == -1) {
        perror("mq_getattr");
        exit(1);
    }

    buf = malloc(attr.mq_msgsize);
    if (buf == NULL) {
        perror("malloc");
        exit(1);
    }

    nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
    if (nr == -1) {
        perror("mq_receive");
        exit(1);
    }

    printf("Received message (%zd bytes): %s\n", nr, buf);
    free(buf);
    exit(0);
}

int main() {
    struct sigevent sev;
    struct sigaction sa;
```

```
35    mqdes = mq_open(QUEUE_NAME, O_RDONLY);
36    if (mqdes == (mqd_t)-1) {
37          perror("mq_open");
38          exit(1);
39    }

40    sa.sa_handler = handler;
41    sigemptyset(&sa.sa_mask);
42    sa.sa_flags = 0;
43    if (sigaction(SIGUSR1, &sa, NULL) == -1) {
44          perror("sigaction");
45          exit(1);
46    }

47    sev.sigev_notify = SIGEV_SIGNAL;
48    sev.sigev_signo = SIGUSR1;
49    if (mq_notify(mqdes, &sev) == -1) {
50          perror("mq_notify");
51          exit(1);
52    }
53    printf("Waiting for message...\n");
54    pause();
55    return 0;
56 }
```

---

mqsig.c Notification via signal

The function `handler` is a signal handler that is invoked when a notification signal is delivered.

At line 14, the handler retrieves the attributes of the message queue using `mq_getattr`. This is necessary to determine the maximum message size.

At line 18, memory is dynamically allocated for the message buffer using `malloc`, with a size equal to `mq_msgsize`.

The message is then read from the queue using

```
23    nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
```

This call retrieves one message from the queue. In this context, it should not block, since the signal is delivered only when a message is available.

At lines 29–30, the allocated memory is freed, and the process is terminated using `exit`. As a result, the program handles a single message and then exits.

In `main`, at lines 33–34, two structures are declared:

- `struct sigevent sev`, used to specify the notification mechanism
- `struct sigaction sa`, used to define the signal handler

At line 35, the program opens an existing message queue using

```
35    mqdes = mq_open(QUEUE_NAME, O_RDONLY);
```

This assumes that the queue has already been created by another process.

At lines 40–43, the signal handler is installed using `sigaction`. The handler function `handler` is associated with the signal `SIGUSR1`.

At lines 47–48, the notification mechanism is configured:

- `sigev_notify` is set to `SIGEV_SIGNAL`, meaning that a signal will be sent when a message arrives
- `sigev_signo` is set to `SIGUSR1`, the signal that will trigger the handler

The program then registers for notification using

```
49    if (mq_notify(mqdes, &sev) == -1) {
```

This call requests that the process be notified when a message arrives on an empty queue.

Finally, at line 54, the call to `pause` suspends the process until a signal is received. When a message arrives, the signal is delivered, the handler is executed, and the process terminates.

This program is intended for demonstration purposes. It is important to note that functions such as `printf`, `malloc`, and `mq_receive` are not guaranteed to be async-signal-safe, and therefore should not be used inside a signal handler in production code.

To send a message we can use the `mq_send.c` program.

Let's execute

Shell 1

```
[linux@ipc] ./mq_send
Enter message:
```

Shell 2

```
[linux@ipc] ./mqsig
Waiting for message…
```

Shell 1

```
Enter message: Hello notification by signal
Message sent: Hello notification via signal
```

Shell 2

```
Waiting for message…
Signal received: message available
Received message (31 bytes): Hello notification via signal
```

The execution demonstrates that, since `sev.sigev_notify` is set to `SIGEV_SIGNAL`, when the messge is received, `SIGUSR1` is delivered and the handler invokes `mq_receive`.

A safer alternative is to use `sigwait` which allows signals to be handled synchronously and avoids the limitations of signal handlers. The `mqsigwait.c` program demonstrates a safe pattern for *asynchronous notification* using `mq_notify` with `SIGEV_SIGNAL` combined with `sigwait`. By blocking the signal and handling it synchronously, the program avoids the limitations of traditional signal handlers and can safely use *standard library functions* such as `malloc` and `printf`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <mqueue.h>
#include <signal.h>
#include <fcntl.h>
#include <unistd.h>

#define QUEUE_NAME "/myqueue"

int main() {
    mqd_t mqdes;
    struct sigevent sev;
    struct mq_attr attr;
    sigset_t mask;
    int sig;

    sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1);
    if (sigprocmask(SIG_BLOCK, &mask, NULL) == -1) {
        perror("sigprocmask");
        exit(1);
    }

    mqdes = mq_open(QUEUE_NAME, O_RDONLY);
    if (mqdes == (mqd_t)-1) {
        perror("mq_open");
        exit(1);
    }

    sev.sigev_notify = SIGEV_SIGNAL;
    sev.sigev_signo = SIGUSR1;
    if (mq_notify(mqdes, &sev) == -1) {
        perror("mq_notify");
        exit(1);
    }

    printf("Waiting for message...\n");
    if (sigwait(&mask, &sig) != 0) {
        perror("sigwait");
```

```
34          exit(1);
35      }

36      printf("Signal received: message available\n");
37      if (mq_getattr(mqdes, &attr) == -1) {
38          perror("mq_getattr");
39          exit(1);
40      }

41      char *buf = malloc(attr.mq_msgsize);
42      if (buf == NULL) {
43          perror("malloc");
44          exit(1);
45      }

46      ssize_t nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
47      if (nr == -1) {
48          perror("mq_receive");
49          exit(1);
50      }

51      printf("Received message (%zd bytes): %s\n", nr, buf);
52      free(buf);
53      mq_close(mqdes);
54      return 0;
55  }
```

---

mqsigwait.c Safer notification via signal

At line 14, the program initializes an empty signal set using `sigemptyset`.
At line 15, the signal `SIGUSR1` is added to this set using `sigaddset`.

In lines 16–19, the program blocks `SIGUSR1` using `sigprocmask`. This is essential because it ensures that the signal will not be delivered asynchronously, but instead can be handled synchronously via `sigwait`.

At line 20, the program opens an existing message queue

```
20      mqdes = mq_open(QUEUE_NAME, O_RDONLY);
```

This assumes that the queue has already been created by another process.

In lines 25–26, the `sigevent` structure is configured by setting `sigev_notify` to `SIGEV_SIGNAL`, and `sigev_signo` to `SIGUSR1`.

The program then registers for notification using

```
27      if (mq_notify(mqdes, &sev) == -1) {
```

At line 31, the program prints a message indicating that it is waiting for input.

At line 32, the program calls

```
32    if (sigwait(&mask, &sig) != 0) {
```

This call suspends execution until one of the signals in `mask`, in this case `SIGUSR1`, is delivered. Since the signal is blocked, it is not handled asynchronously but instead causes `sigwait` to return. Once `sigwait` returns, the program continues execution in a safe context.

At line 37, the program retrieves the queue attributes using `mq_getattr`. This is necessary to determine the maximum message size.
At line 41, memory is dynamically allocated for the message buffer using `malloc`, with a size equal to `mq_msgsize`.
At line 46, the program reads a message from the queue using

```
46    ssize_t nr = mq_receive(mqdes, buf, attr.mq_msgsize, NULL);
```

This call retrieves one message from the queue. Since the signal was triggered by message arrival, the call is expected to succeed without blocking.

Finally at line 52, the allocated memory is freed and at line 53, the queue descriptor is closed using `mq_close`.

The combination of `SIGEV_SIGNAL` and `sigwait` is the recommended safe pattern for signal-based notification. The signal must be blocked before calling `mq_notify`, otherwise it may be delivered before `sigwait` is ready to receive it.

Let' execute

Shell 1

```
[linux@ipc] ./mq_send
Enter message:
```

Shell 2

```
[linux@ipc] ./mqsigwait
Waiting for message…
```

Shell 1

```
Enter message: hello from sender
Message sent: hello from sender
```

Shell 2

```
Waiting for message…
Signal received: message available
Received message (19 bytes): hello from sender
```

When the message is received, the signal is generated, and this causes `sigwait` to return. The program then retrieves the message from the queue and prints its contents.

VI.3 POSIX Named Semaphores

A *named semaphore* is identified by a unique string name, typically beginning with a forward slash (e.g., `/example`). This naming allows multiple, unrelated processes to access and operate on the same semaphore by referring to it with the same identifier.
Named semaphores are created or opened using the `sem_open` function. Once opened, they can be manipulated using `sem_post` and `sem_wait`. When a process no longer needs the semaphore, it should call `sem_close` to release its reference. After all processes have finished using the semaphore, it can be removed from the system using `sem_unlink`.
Named semaphores are implemented using a *virtual filesystem*, which is typically mounted at `/dev/shm`. Internally, each named semaphore is stored as a file in this location. Although applications refer to a semaphore using a name like `/somename`, the system actually represents it as a file named `sem.somename` inside `/dev/shm`.
Because of this naming scheme, the maximum length of a semaphore name is slightly reduced. Specifically, it must be `NAME_MAX − 4` characters, since the prefix `sem.` takes up four characters in the underlying filename.

Named semaphores have *kernel persistence*: if not removed by `sem_unlink`, a semaphore will exist until the system is shut down.

Common patterns with semaphores are as follows

1. Resource control (*mutual exclusion*)

- Semaphore initialized to 1
- Acts like a mutex
- Ensures only one execution flow enters the critical section

2. Signaling (*event notification*)

- Semaphore initialized to 0
- One process signals, another waits
- Process A finished → Process B continues

3. Counting / *resource limiting*

- Semaphore initialized to N
- Limits number of concurrent accesses
- e.g., max 5 processes using a resource

VI.3.1 Creating and deleting semaphores: `sem_open`, `sem_close`, `sem_unlink`

To create/open a semaphore we can use the `sem_open`[11] function, which is defined as follows

```
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>

sem_t *sem_open(const char *name, int oflag, … /* mode_t mode, unsigned
                int value */ );

           Returns: address of the new semaphore, on error SEM_FAILED and errno
```

The `sem_open` function is used to either create a new POSIX semaphore or open an existing one, identified by a given name.
The `oflag` argument controls how the function behaves:

- If `O_CREAT` is set, the semaphore is created if it does not already exist
- The new semaphore's owner (user ID) and group (group ID) are set to those of the calling process
- If both `O_CREAT` and `O_EXCL` are specified, the call fails if a semaphore with the same name already exists

When `O_CREAT` is used, two additional arguments are required:

- `mode` specifies the semaphore's permissions
- `value` sets the semaphore's initial value

If `O_CREAT` is specified but the semaphore already exists, `mode` and `value` are ignored.

To close a semaphore the `sem_close`[12] function is used.

```
#include <semaphore.h>

int sem_close(sem_t *sem);

                              Returns: on success 0, on error -1 and errno is set
```

The `sem_close` function closes the named semaphore referred to by `sem`, allowing any resources that the system has allocated to the calling process for this semaphore to be freed.

When a process exits, the system automatically closes all named semaphores it had opened. Likewise, if the process calls `execve` to replace its program image, any open named semaphores are closed as part of that transition.

To remove a named semaphore we use the `sem_unlink`[13] function.

---

11 See the `sem_open(3)` manual page
12 See the `sem_close(3)` manual page
13 See the `sem_unlink(3)` manual page

```c
#include <semaphore.h>

int sem_unlink(const char *name);

                                Returns: on success 0, on error -1 and errno is set
```

The `sem_unlink` function removes the named semaphore referred to by `name`. The semaphore name is removed immediately and is destroyed once all other processes that have the semaphore open close it.

We have already seen the `sem_post` and `sem_wait` function in the section dedicated to *unnamed semaphores* in the previous chapter.

The programs `semprod.c` and `semcons.c` form a *producer–consumer* system in which the consumer blocks until the producer signals via the semaphore.

```c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <semaphore.h>
5 #include <unistd.h>

6 int main() {
7     const char *name = "/my_semaphore";
8     sem_t *sem = sem_open(name, O_CREAT, 0644, 0);
9     if (sem == SEM_FAILED) {
10          perror("sem_open");
11          exit(EXIT_FAILURE);
12    }
13    printf("Producer: doing some work...\n");
14    sleep(3);
15    printf("Producer: signaling semaphore\n");
16    sem_post(sem);
17    sem_close(sem);
18    return 0;
19 }
```

---

semprod.c

At line 8, the program opens a named semaphore, creating it only if it does not already exist. If the semaphore is created, its initial value is set to 0. If it already exists, the 0644 permissions and initial value 0 are ignored, and the existing semaphore is simply opened.

```c
8     sem_t *sem = sem_open(name, O_CREAT, 0644, 0);
```

The program simulates some work using `sleep`.
Then at line 16

```
16    sem_post(sem);
```

the producer increments the semaphore's value. If another process (e.g., a consumer) is blocked in `sem_wait`, it will be unblocked at this point. Otherwise, the semaphore's value is simply increased.

Finally, the program closes its reference to the semaphore

```
17    sem_close(sem);
```

This does not remove the semaphore from the system. The semaphore will continue to exist until it is explicitly removed using `sem_unlink`.

The `semcons.c` program acts as consumer, it waits for a signal before proceeding.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <semaphore.h>

int main() {
    const char *name = "/my_semaphore";
    sem_t *sem = sem_open(name, O_CREAT, 0644, 0);
    if (sem == SEM_FAILED) {
          perror("sem_open");
          exit(EXIT_FAILURE);
    }
    printf("Consumer: waiting for signal...\n");
    sem_wait(sem);
    printf("Consumer: received signal!\n");
    sem_close(sem);
    sem_unlink(name);
    return 0;
}
```

---

semcons.c

The program opens a named semaphore, creating it if it does not already exist. If created, it is initialized with a value of 0. If it already exists, the existing semaphore is opened and the 0644 permissions and initial value are ignored.

```
7     sem_t *sem = sem_open(name, O_CREAT, 0644, 0);
```

The program waits (blocks) on the semaphore

```
13    sem_wait(sem);
```

Since the initial value is 0, the call will block until another process (the producer) calls `sem_post` and increments the semaphore. Once the semaphore's value becomes greater than 0, `sem_wait` decrements it and continues execution.

After the signal is received, the program closes its reference to the semaphore

```
15    sem_close(sem);
```

This does not destroy the semaphore itself, it only releases the process's handle to it.
The program removes the semaphore name from the system

```
16    sem_unlink(name);
```

After this call, no new processes can open the semaphore using that name. After `sem_unlink`, the semaphore continues to exist in system memory until all processes referencing it have called `sem_close`.
Let's execute

Shell 1

```
[linux@ipc] ./semcons
Consumer: waiting for signal…
```

Shell 2

```
[linux@ipc] ./semprod
Producer: doing some work...
Producer: signaling semaphore
```

Shell 1

```
Consumer: waiting for signal…
Consumer: received signal!
```

When the producer signals the semaphore, the `sem_wait` call in the consumer returns.

VI.3.2 Process-local mutual exclusion

The `semutex.c` program is a modified version of the one in the APUE[14]. The program implements a simple locking mechanism based on POSIX named semaphores. It defines a custom `slock` abstraction that encapsulates a semaphore and provides functions to allocate, acquire, release, and free the lock.
The semaphore is created with a unique name and immediately unlinked, making it inaccessible by name while still usable by the process. This technique allows the semaphore to behave like a private, unnamed synchronization primitive managed by the kernel, similar to how mutexes operate. The resulting lock can be used to protect a critical section, ensuring mutual exclusion by allowing only one execution flow at a time to enter the protected region.

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <errno.h>
5 #include <semaphore.h>
6 #include <fcntl.h>
  #include <limits.h>

7 struct slock {
8     sem_t *semp;
9     char name[NAME_MAX];
10 };

11 struct slock *s_alloc()
12 {
13    struct slock *sp;
14    static int cnt;

15    if ((sp = malloc(sizeof(struct slock))) == NULL)
16          return(NULL);

17    do {
18          snprintf(sp->name, sizeof(sp->name), "/%ld.%d",
                    (long)getpid(),cnt++);
19          sp->semp = sem_open(sp->name, O_CREAT|O_EXCL, S_IRWXU, 1);
20    } while ((sp->semp == SEM_FAILED) && (errno == EEXIST));

21    if (sp->semp == SEM_FAILED) {
22          free(sp);
23          return(NULL);
24    }
25    sem_unlink(sp->name);
26    return(sp);
27 }

28 void s_free(struct slock *sp)
```

14 Section 15.10, fig. 15.35, pag. 583

```
29 {
30    sem_close(sp->semp);
31    free(sp);
32 }

33 int s_lock(struct slock *sp)
34 {
35    return(sem_wait(sp->semp));
36 }
37 int s_trylock(struct slock *sp)
38 {
39    return(sem_trywait(sp->semp));
40 }

41 int s_unlock(struct slock *sp)
42 {
43    return(sem_post(sp->semp));
44 }

45 int main() {
46    struct slock *lock = s_alloc();
47    if (lock == NULL) {
48          perror("s_alloc");
49          exit(EXIT_FAILURE);
50    }

51    printf("Trying to acquire lock...\n");
52    if (s_lock(lock) == -1) {
53          perror("s_lock");
54          s_free(lock);
55          exit(EXIT_FAILURE);
56    }

57    printf("Lock acquired. Doing work...\n");
58    sleep(3);
59    printf("Releasing lock...\n");

60    if (s_unlock(lock) == -1) {
61          perror("s_unlock");
62    }
63    s_free(lock);
64    return 0;
65 }
```

---

semutex.c Semaphore-based mutual exclusion primitive

At lines 7–10, a structure `slock` is defined, containing a pointer to a POSIX semaphore (`sem_t *semp`) and a character array used to store the semaphore's name.

The function `s_alloc` allocates memory for a `slock` structure using `malloc`.

```
13    struct slock *sp;
14    static int cnt;
```

`cnt` is a static variable used to help generate unique semaphore names.

Inside the `do-while` loop at line 17 a unique semaphore name is generated using the process ID and a counter via `snprintf`
The semaphore is then created using

```
19          sp->semp = sem_open(sp->name, O_CREAT|O_EXCL, S_IRWXU, 1);
```

The `O_EXCL` flag ensures that the call fails if a semaphore with the same name already exists, helping guarantee uniqueness. If the semaphore already exists (`errno == EEXIST`), the loop retries with a new name.
If `sem_open` fails for any other reason, the allocated memory is freed and `NULL` is returned.

At line 25, the semaphore's name is removed from the system immediately after creation.

```
25    sem_unlink(sp->name);
```

However, the semaphore itself continues to exist in memory as long as it is referenced by at least one process. This creates a private semaphore that behaves like an unnamed semaphore but uses the named semaphore API.

The functions:

`s_free` → closes the semaphore and frees the allocated memory
`s_lock` → locks the semaphore using `sem_wait`
`s_trylock` → attempts a non-blocking lock using `sem_trywait`
`s_unlock` → unlocks the semaphore using `sem_post`

In `main` a new `slock` is allocated via `s_alloc` and at line 52, the program acquires the lock by calling `s_lock` which calls `sem_wait` and may block until the semaphore is available.
At line 58, the program simulates work inside the critical section using `sleep`.
At line 60, the lock is released by calling `s_unlock` which calls `sem_post`.
Finally, the semaphore is closed and the allocated memory is freed via `s_free`.

Let's execute the program

```
[linux@ipc] ./semutex
Trying to acquire lock...
Lock acquired. Doing work...
Releasing lock...
```

From the execution, we can see that the semaphore-based lock works as expected: the process acquires the lock, performs its work inside the critical section, and then releases the lock.

This technique enforces mutual exclusion, ensuring that only one thread or process can access the critical section at any given time.
The program, although based on a named semaphore, can be used for synchronization among multiple threads within a single process.

Since threads share the same address space, they can access the same semaphore, allowing the program to enforce mutual exclusion in a manner similar to a mutex.

The `semutex_thread.c` program is a modified version of `semutex.c` that uses threads instead of a single execution flow. It demonstrates a *multi-threaded mutual exclusion* pattern implemented using a POSIX semaphore.
The `main` function is adapted to create multiple threads, and a `thread_func` routine is introduced to define the work performed by each thread. The remaining code, including the semaphore-based lock abstraction, remains unchanged.

```c
static void *thread_func(void *arg) {
      struct slock *lock = (struct slock *)arg;
      printf("Thread %lu: Trying to acquire lock...\n", pthread_self());

      if (s_lock(lock) == -1) {
            perror("s_lock");
            pthread_exit(NULL);
      }

      printf("Thread %lu: Lock acquired. Working..\n", pthread_self());
      sleep(2);
      printf("Thread %lu: Releasing lock...\n", pthread_self());

      if (s_unlock(lock) == -1)
            perror("s_unlock");
      pthread_exit(NULL);
}

int main() {
      struct slock *lock = s_alloc();
      if (lock == NULL) {
            perror("s_alloc");
            exit(EXIT_FAILURE);
      }

      pthread_t t1;
      pthread_t t2;
      pthread_create(&t1, NULL, &thread_func, lock);
      pthread_create(&t1, NULL, &thread_func, lock);
      pthread_join(t1, NULL);
      pthread_join(t2, NULL);

      s_free(lock);
      return 0;
}
```

---

semutex_thread.c

The `thread_func` function retrieves the `slock` structure passed as an argument. It then acquires the lock on the semaphore and simulates work in the critical section using `sleep`. Once the work is completed, it releases the lock by posting to the semaphore and terminates by calling `pthread_exit`.
The `main` function creates two threads and executes them concurrently.

Let's execute

```
[linux@ipc] ./semutex_thread
Thread 139968944023232: Trying to acquire lock...
Thread 139968944023232: Lock acquired. Working...
Thread 139968935630528: Trying to acquire lock...
Thread 139968944023232: Releasing lock...
Thread 139968935630528: Lock acquired. Working...
Thread 139968935630528: Releasing lock...
```

As we can see from the outptut, the threads are synchronized, only one thread at a time can access the critical section. The second thread acquires the lock only after thread 139968944023232 releases it, demonstrating proper mutual exclusion.

Thread synchronization is usually performed using mutexes. This pattern effectively replicates mutex-like behavior using a semaphore.

This pattern can be extended to enable synchronization among multiple processes, as demonstrated in the following section.

VI.3.3 Interprocess mutual exclusion

Named semaphores are often used in conjunction with *shared memory* to synchronize access to *shared data structures*. In this common pattern, shared memory provides a region where data can be exchanged efficiently between processes, while semaphores ensure that such access occurs in a controlled and mutually exclusive manner. However, named semaphores are not limited to this use case. They also serve as general-purpose *interprocess synchronization primitives* and can be employed independently of shared memory for tasks such as signaling, coordination, and controlling access to limited resources.

The previous example represents a *process-local* synchronization mechanism, whereas the following one extends this approach to support synchronization across multiple processes. In this case, a named semaphore is used as a shared synchronization primitive, enabling *interprocess mutual exclusion*.

The `semutex_multiprocess.c` program demonstrates how multiple independent processes can coordinate access to a *critical section* by using a common named semaphore. Each process attempts to acquire the semaphore before entering the critical section and releases it upon completion. As a result, at most one process at a time is allowed to execute the protected code, ensuring correct and consistent behavior across all participating processes.

A global constant is declared to store the name of the semaphore

```
#define SEM_NAME "/my_shared_lock"
```

The `s_alloc` function is changed as follows

```
struct slock *s_alloc()
{
      struct slock *sp;
      if ((sp = malloc(sizeof(struct slock))) == NULL)
            return NULL;

      snprintf(sp->name, sizeof(sp->name), "%s", SEM_NAME);
      sp->semp = sem_open(sp->name, O_CREAT, 0644, 1);
      if (sp->semp == SEM_FAILED) {
            free(sp);
            return NULL;
      }
      return sp;
}
```

The semaphore is created using a fixed name instead of a unique one, enabling multiple processes to share and synchronize on the same semaphore. Additionally, it is not unlinked immediately, ensuring that other processes can open and use it.

A `s_destroy` function is introduced to remove the semaphore from the system.

```
void s_destroy()
{
      if (sem_unlink(SEM_NAME) == -1)
            perror("sem_unlink");
}
```

The `main` is as follows

```
int main()
{
      struct slock *lock = s_alloc();
      if (lock == NULL) {
            perror("s_alloc");
            exit(EXIT_FAILURE);
      }

      printf("PID %d: Trying to acquire lock...\n", getpid());
      if (s_lock(lock) == -1) {
            perror("s_lock");
            s_free(lock);
            exit(EXIT_FAILURE);
      }

      printf("PID %d: Lock acquired. Entering critical section...\n",
             getpid());
      sleep(3);
      printf("PID %d: Leaving critical section\n", getpid());

      if (s_unlock(lock) == -1)
            perror("s_unlock");

      s_free(lock);
      return 0;
}
```

semutex_multiprocess.c Interprocess mutual exclusion

The `main` function acquires the lock, executes a simulated critical section using `sleep`, and then releases the lock before freeing the allocated memory.

The behavior of the program can be observed by executing multiple instances concurrently, allowing the synchronization mechanism to be clearly demonstrated.

Let’s execute

```
[linux@ipc] ./semutex_multiprocess & ; ./semutex_multiprocess & ; ./semutex_multiprocess &
[1] 1121927
[2] 1121928
[3] 1121929
PID 1121927: Trying to acquire lock...
PID 1121927: Lock acquired. Entering critical section...

PID 1121928: Trying to acquire lock...
PID 1121929: Trying to acquire lock...
[linux@ipc]  PID 1121927: Leaving critical section
PID 1121928: Lock acquired. Entering critical section...

[1]    done       ./semutex_multiprocess
[linux@ipc]  PID 1121928: Leaving critical section
PID 1121929: Lock acquired. Entering critical section...

[2]  - done       ./semutex_multiprocess
[linux@ipc]  PID 1121929: Leaving critical section

[3]  + done       ./semutex_multiprocess
```

As we can see from the output, the three processes are started concurrently in the background and each attempts to acquire the lock almost immediately.
The first process (PID 1121927) successfully acquires the semaphore and enters the critical section, while the other two processes (PIDs 1121928 and 1121929) attempt to acquire the lock but are blocked in the `sem_wait` call.
Only after the first process releases the semaphore by calling `sem_post` does one of the waiting processes (PID 1121928) acquire the lock and proceed into the critical section. The same behavior is then observed for the third process (PID 1121929), which enters the critical section only after the second process has completed its execution and released the semaphore.

This sequential access clearly shows that, although the processes execute concurrently, the semaphore enforces strict *mutual exclusion*: at any given time, only one process is allowed to execute the critical section, while all others are suspended until the resource becomes available. This guarantees correct synchronization and prevents race conditions on shared resources.

In this example, the semaphore is employed for *resource control*, enforcing *mutual exclusion* among processes. As we have seen, *named semaphores* are not limited to this use; they are also widely used for *signaling*, for instance to notify that one process has completed its execution so that another may proceed.

VI.3.4 Two-way synchronization pattern

The following two programs implement a *two-way handshake* using two named semaphores. This example illustrates a *bidirectional synchronization* pattern in which semaphores are used to coordinate a request–response interaction between two processes. By employing separate semaphores for each direction of communication, the processes enforce a strict execution order, ensuring that each step occurs only after the corresponding signal is received, without resorting to busy waiting.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <semaphore.h>
5 #include <unistd.h>

6 #define REQ_SEM "/req_sem"
7 #define RES_SEM "/res_sem"

8 int main() {
9     sem_t *req = sem_open(REQ_SEM, O_CREAT, 0644, 0);
10    sem_t *res = sem_open(RES_SEM, O_CREAT, 0644, 0);

11    if (req == SEM_FAILED || res == SEM_FAILED) {
12          perror("sem_open");
13          exit(EXIT_FAILURE);
14    }

15    printf("Client: sending request...\n");
16    sem_post(req);
17    printf("Client: waiting for response...\n");
18    sem_wait(res);
19    printf("Client: received response!\n");

20    sem_close(req);
21    sem_close(res);
22    return 0;
23 }
```

---

sem_client.c Client program

The `main` function creates (or opens) two semaphores: `req_sem` and `res_sem`.
At line 16, it sends a request by calling `sem_post` on `req_sem`, and at line 18 it waits for the response by calling `sem_wait` on `res_sem`.
Finally, at lines 20–21, the program closes both semaphores using `sem_close`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
```

```c
#include <semaphore.h>
#include <unistd.h>

#define REQ_SEM "/req_sem"
#define RES_SEM "/res_sem"

int main() {
    sem_t *req = sem_open(REQ_SEM, O_CREAT, 0644, 0);
    sem_t *res = sem_open(RES_SEM, O_CREAT, 0644, 0);

    if (req == SEM_FAILED || res == SEM_FAILED) {
        perror("sem_open");
        exit(EXIT_FAILURE);
    }
    printf("Server: waiting for request...\n");
    sem_wait(req);
    printf("Server: processing request...\n");
    sleep(2);
    printf("Server: sending response...\n");
    sem_post(res);

    sem_close(req);
    sem_close(res);
    sem_unlink(REQ_SEM);
    sem_unlink(RES_SEM);
    return 0;
}
```

---

sem_server.c Server program

The `main` function opens (or creates) the two semaphores. It then waits for a request from the client by calling `sem_wait` on `req_sem` and simulates processing using `sleep`. After completing the work, it sends the response by calling `sem_post` on `res_sem`.
Finally, it closes both semaphores using `sem_close`, and performs cleanup by removing them from the system with `sem_unlink`.

Let's execute

Shell 1

```
[linux@ipc]  ./sem_server
Server: waiting for request...
Server: processing request...
Server: sending response...
```

Shell 2

```
[linux@ipc]  ./sem_client
Client: sending request...
Client: waiting for response...
Client: received response!
```

From the execution, we can observe the correct coordination between the two processes. The client sends a request and blocks while waiting for a response, whereas the server waits for the request, processes it, and then signals completion.
This interaction demonstrates a two-way synchronization pattern in which semaphores enforce a strict ordering of operations between the processes.

Without this technique, the synchronization would have to be implemented using a busy-waiting loop, resulting in inefficient use of CPU resources.

For example:

```
volatile int ready = 0;

printf("Server: waiting for request...\n");
while (ready != 1) {
      /* spin (wastes CPU) */
}
//... process request...
```

A busy-waiting loop continuously polls a shared variable, leading to inefficient CPU utilization. Semaphores avoid this issue by suspending execution until the required condition is met.

VI.4 POSIX Shared Memory

Two processes can share a specific region of memory. Since data does not need to be copied between processes, shared memory represents the fastest form of IPC. Access to the shared region must be synchronized. To achieve proper synchronization, semaphores, record locking, and mutexes can be used.

POSIX shared memory objects have kernel persistence; this means a shared memory object will exist until the system is shut down, or until all processes have unmapped the object and it has been deleted. A shared memory object is created in a tmpfs virtual filesystem, usually mounted at `/dev/shm`.

IV.4.1 Overview

A shared memory object is created using the `shm_open` function and is deleted with `shm_unlink`.
The following functions can be used to operate on a shared memory object:

`ftruncate`
: Set the size of the shared memory object. A newly created shared memory object has a length of zero

`mmap`
: Map the shared memory object into the virtual address space of the calling process

`munmap`
: Unmap the shared memory object from the virtual address space of the calling process

`close`
: Close the file descriptor allocated by `shm_open`

`fstat`
: Obtain a stat structure that describes the shared memory object

`fchown`
: To change the ownership of a shared memory object

`fchmod`
: To change the permissions of a shared memory object

The shm_open and shm_unlink functions[15] are defined as follows

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>

int shm_open(const char *name, int oflag, mode_t mode);

int shm_unlink(const char *name);

      Return: on success a file descriptor, on success 0, on error -1 and errno
```

The shm_open function creates and opens a new POSIX shared memory object, or opens an existing one. A POSIX shared memory object is, in effect, a handle that can be used by unrelated processes to mmap the same region of shared memory.

The shm_unlink function performs the opposite operation, removing an object previously created by shm_open.
The operation of shm_open is analogous to that of open. The name parameter specifies the shared memory object to be created or opened. For portable use, a shared memory object should be identified by a name of the form /somename; that is, a null-terminated string of up to NAME_MAX characters, consisting of an initial slash followed by one or more characters, none of which are slashes.

The oflag argument is a bit mask created by OR-ing together exactly one of O_RDONLY or O_RDWR, along with any of the other flags listed below:

O_RDONLY
: Open the object for read access. A shared memory object opened in this way can be mmap-ed only for read (PROT_READ) access

O_RDWR
: Open the object for read-write access

O_CREAT
: Create the shared memory object if it does not exist. The user and group ownership of the object are taken from the corresponding effective IDs of the calling process, and the object's permission bits are set according to the low-order 9 bits of mode, except that those bits set in the process file mode creation mask are cleared for the new object.
A new shared memory object initially has zero length, the size of the object can be set using ftruncate. The newly allocated bytes of a shared memory object are automatically initialized to 0

O_EXCL
: If O_CREAT was also specified, and a shared memory object with the given name already exists, return an error. The check for the existence of the object, and its creation if it does not exist, are performed atomically

15 See the shm_open(3) manual page

`O_TRUNC`
: If the shared memory object already exists, truncate it to zero bytes

The new file descriptor returned upon a successful call to `shm_open` has the `FD_CLOEXEC` flag set. After a subsequent call to `mmap` that maps the object into the memory, the file descriptor can be closed.

`shm_unlink` behaves similarly to `unlink`: it removes a shared memory object name, and, once all processes have unmapped the object, deallocates and destroys the contents of the associated memory region.

The following two programs `shmcount.c` and `shmcount_send.c` use a shared memory object to share a counter. `shmcount.c` initializes a counter to 0 and keep the shared object alive for 30 seconds, `shmcount_send.c` increments the counter. Before exiting, `shmcount.c` prints the actual value of the counter on `stdout`.

```c
#include <stdio.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

struct shm_counter {
    int counter;
};

int main(void)
{
    int fd = shm_open("/shm_counter", O_CREAT | O_RDWR, 0600);
    ftruncate(fd, sizeof(struct shm_counter));
    struct shm_counter *shmp = mmap(NULL, sizeof(*shmp), PROT_READ |
                                    PROT_WRITE, MAP_SHARED, fd, 0);
    shmp->counter = 0;
    printf("Counter initialized to %d\n", shmp->counter);
    sleep(30);
    printf("Counter before exiting: %d\n", shmp->counter);
    shm_unlink("/shm_counter");
    return 0;
}
```

shmcount.c Creates and share a counter

```c
#include <stdio.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
```

```
6 struct shm_counter {
7     int counter;
8 };

9 int main(void)
10 {
11     int fd = shm_open("/shm_counter", O_RDWR, 0);
12     struct shm_counter *shmp = mmap(NULL, sizeof(*shmp), PROT_READ |
                                       PROT_WRITE, MAP_SHARED, fd, 0);

13     for (int i = 0; i < 10; i++) {
14         shmp->counter++;
15         printf("Counter = %d\n", shmp->counter);
16         sleep(1);
17     }
18     return 0;
19 }
```

---

shmcount_send.c Increments the shared counter

In the `shmcount.c` code we declare a structure `shm_counter` to keep the value

we create the shared object via `shm_open` using the name `/shm_counter`

```
11     int fd = shm_open("/shm_counter", O_CREAT | O_RDWR, 0600);
```

and truncate the size to that one of the structure

```
12     ftruncate(fd, sizeof(struct shm_counter));
```

then we create the *shared mapping* via `mmap` passing the descriptor returned by `shm_open`

```
13     struct shm_counter *shmp = mmap(NULL, sizeof(*shmp), PROT_READ |
                                       PROT_WRITE, MAP_SHARED, fd, 0);
```

we initialize the counter to 0

```
14     shmp->counter = 0;
```

in the lines 15-18 we print the initial value, wait for 30 seconds, print the final value and remove the object name

In the `shmcount_send.c` program we declare the same structure to store the counter value and in the `main` we open the shared object using the `/shm_counter` name

```
11     int fd = shm_open("/shm_counter", O_RDWR, 0);
```

We create the mapping via `mmap` passing the descriptor returned by `shm_open`

```
12    struct shm_counter *shmp = mmap(NULL, sizeof(*shmp), PROT_READ |
                                      PROT_WRITE, MAP_SHARED, fd, 0);
```

The for `loop` increments the counter up to 10.

Let's execute

Shell 1

```
[linux@ipc] ./shmcount
Counter initialized to 0
```

Shell 2

```
[linux@ipc] ./shmcount_send
Counter = 1
Counter = 2
Counter = 3
Counter = 4
Counter = 5
Counter = 6
Counter = 7
Counter = 8
Counter = 9
Counter = 10
[linux@ipc]
```

Shell 1

```
Counter initialized to 0
Counter before exiting: 10
```

The two programs map the same *memory region* via a *shared memory object*, therefore changes made by one process are immediately visible to the other. The shared memory object is used to share the counter between the two independent processes.

In this example, synchronization is intentionally omitted for simplicity. In real programs, however, access to *shared memory* must be synchronized to coordinate reads and writes. If multiple processes modify the shared counter concurrently, *data races* may occur.

As we have seen the usual pattern is:

- Create the shared memory object: `shm_open`
- Set the size: `ftruncate`
- Map the shared memory object: `mmap`
- Operate on the shared memory object
- Delete the shared memory object: `shm_unlink`

A more realistic example is using shared memory objects with a synchronization mechanism, e.g., POSIX semaphores.

The `pshm_ucase_bounce.c` and `pshm_ucase_send.c`[16] use a shared memory objects and two unnamed semaphores to exchange data. The *bounce* program converts to uppercase a string that is placed into the shared memeory object by the *send* program. Once the string is converted, the *bounce* program notifies the *send* that prints the new contents on `stdout`.

The `pshm_ucase.h` header file defines the `shmbuf` structure.

```
#ifndef PSHM_UCASE_H
#define PSHM_UCASE_H

#include <err.h>
#include <semaphore.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

#define BUF_SIZE 1024     /* Maximum size for exchanged string */

struct shmbuf {
      sem_t sem1;           /* POSIX unnamed semaphore */
      sem_t sem2;           /* POSIX unnamed semaphore */
      size_t cnt;           /* Number of bytes used in 'buf' */
      char buf[BUF_SIZE];   /* Data being transferred */
};

#endif
```

---

pshm_ucase.h Definitions

The structure contains two semaphores, the buffer and the size.

```
1 #include <ctype.h>
2 #include <fcntl.h>
3 #include <sys/mman.h>
4 #include <unistd.h>
5 #include "pshm_ucase.h"

6 int main(int argc, char *argv[])
7 {
8     int fd;
9     char *shmpath;
10    struct shmbuf *shmp;

11    if (argc != 2) {
12          fprintf(stderr, "Usage: %s /shm-path\n", argv[0]);
13          exit(EXIT_FAILURE);
14    }
```

16 The programs are taken from the `shm_open(3)` manual page

```
15    shmpath = argv[1];
16    fd = shm_open(shmpath, O_CREAT | O_EXCL | O_RDWR, 0600);
17    if (fd == -1)
18          err(EXIT_FAILURE, "shm_open");

19    if (ftruncate(fd, sizeof(struct shmbuf)) == -1)
20          err(EXIT_FAILURE, "ftruncate");

21    shmp = mmap(NULL, sizeof(*shmp), PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
22    if (shmp == MAP_FAILED)
23          err(EXIT_FAILURE, "mmap");

24    if (sem_init(&shmp->sem1, 1, 0) == -1)
25          err(EXIT_FAILURE, "sem_init-sem1");
26    if (sem_init(&shmp->sem2, 1, 0) == -1)
27          err(EXIT_FAILURE, "sem_init-sem2");

28    if (sem_wait(&shmp->sem1) == -1)
29          err(EXIT_FAILURE, "sem_wait");

30    for (size_t j = 0; j < shmp->cnt; j++)
31          shmp->buf[j] = toupper((unsigned char) shmp->buf[j]);

32    if (sem_post(&shmp->sem2) == -1)
33          err(EXIT_FAILURE, "sem_post");

34    shm_unlink(shmpath);
35    exit(EXIT_SUCCESS);
36 }
```

---

pshm_ucase_bounce.c

At lines 9–10, the program declares a file descriptor (`fd`), a pointer to the shared memory object name (`shmpath`), and a pointer to a `shmbuf` structure that will represent the shared memory region.

At line 15, it retrieves the shared memory path supplied on the command line.

At line 16, it creates and opens the shared memory object using `shm_open`(). The flags `O_CREAT | O_EXCL | O_RDWR` ensure that a new object is created (and the call fails if it already exists) with read/write permissions.
At line 19, it truncates the shared memory object to the size of struct `shmbuf` using `ftruncate`.

At line 21, it maps the shared memory object into the process's address space using `mmap`, with read and write permissions and the `MAP_SHARED` flag so that changes are visible to other processes.

Lines 24–27 initialize two semaphores (`sem1` and `sem2`) stored inside the shared memory. The second argument (`1`) indicates that the semaphores are shared between processes, and both are initialized to `0`, meaning they start in the locked state.

At line 28, the program waits on `sem1` using `sem_wait`, blocking until another process (the sender) signals that data is ready.

The loop at line 30 processes the data in the shared memory by converting each character in the buffer to uppercase.

At line 32, the program signals completion by posting `sem2` with `sem_post`, notifying the sender process that the modified data is ready.

Finally, at line 34, it unlinks the shared memory object using `shm_unlink`. Even if another process is still using the object, it will only be removed after all processes have unmapped and closed it.

```
1 #include <fcntl.h>
2 #include <stddef.h>
3 #include <string.h>
4 #include <sys/mman.h>
5 #include <unistd.h>
6 #include "pshm_ucase.h"

7 int main(int argc, char *argv[])
8 {
9     int fd;
10    char *shmpath, *string;
11    size_t len;
12    struct shmbuf *shmp;

13    if (argc != 3) {
14          fprintf(stderr, "Usage: %s /shm-path string\n", argv[0]);
15          exit(EXIT_FAILURE);
16    }

17    shmpath = argv[1];
18    string = argv[2];
19    len = strlen(string);
20    if (len > BUF_SIZE) {
21          fprintf(stderr, "String is too long\n");
22          exit(EXIT_FAILURE);
23    }

24    fd = shm_open(shmpath, O_RDWR, 0);
25    if (fd == -1)
26          err(EXIT_FAILURE, "shm_open");

27    shmp = mmap(NULL, sizeof(*shmp), PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
```

```
28    if (shmp == MAP_FAILED)
29          err(EXIT_FAILURE, "mmap");

30    shmp->cnt = len;
31    memcpy(&shmp->buf, string, len);

32    if (sem_post(&shmp->sem1) == -1)
33          err(EXIT_FAILURE, "sem_post");

34    if (sem_wait(&shmp->sem2) == -1)
35          err(EXIT_FAILURE, "sem_wait");

36    if (write(STDOUT_FILENO, &shmp->buf, len) == -1)
37          err(EXIT_FAILURE, "write");
38    if (write(STDOUT_FILENO, "\n", 1) == -1)
39          err(EXIT_FAILURE, "write");
40    exit(EXIT_SUCCESS);
41 }
```

---

pshm_ucase_send.c

At lines 17–18, the program retrieves the command-line arguments: the shared memory path and the input string.

At lines 19–23, it computes the length of the string and checks that it does not exceed `BUF_SIZE`, ensuring it fits within the shared memory buffer.

Next, at line 24, it opens the existing shared memory object using `shm_open` with read/write access. Unlike the *bounce* program, it does not create the object, but expects it to already exist.

At line 27, it maps the shared memory object into the process's address space using `mmap`, with read and write permissions and the `MAP_SHARED` flag.

At lines 30–31, it stores the length of the string in `shmp->cnt` and copies the string into the shared buffer using `memcpy`.

At line 32, it signals the *bounce* process by posting sem1 with `sem_post`, indicating that the data is ready to be processed.

At line 34, it waits on `sem2` using `sem_wait`, blocking until the *bounce* process finishes processing the data.

At lines 36–39, it writes the modified (uppercase) data from shared memory to standard output.

Let's execute the two programs

Shell 1

```
[linux@ipc] ./pshm_ucase_bounce /myshm
```

Shell 2

```
[linux@ipc] ./pshm_ucase_send /myshm hello
HELLO
```

Shell 1

```
[linux@ipc] ./pshm_ucase_bounce /myshm

[linux@ipc]
```

As we can see from the output, the two processes successfully communicate through shared memory. The `pshm_ucase_send` program writes the string "`hello`" into the shared memory and signals the `pshm_ucase_bounce` process.
The `pshm_ucase_bounce` program then processes the data by converting it to uppercase and signals back when it is done. The `pshm_ucase_send` program resumes execution, reads the modified data, and prints "`HELLO`" to standard output.
Finally, the `pshm_ucase_bounce` program terminates after unlinking the shared memory object.

VI.4.2 Producer-consumer pattern

The two programs `pc_producer.c` and `pc_consumer.c` compose a classic *producer–consumer system*, where the producer inserts data into a shared circular buffer and the consumer removes it. Synchronization is achieved using *semaphores* (`empty`, `full`) and a *mutex* to avoid race conditions and ensure correct access to the shared memory.

The header file contains defines the `shmseg` structure.

```c
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>

#define BUF_SIZE 10

struct shmseg {
      int buffer[BUF_SIZE];
      int in;
      int out;
      sem_t empty;     // free slots
      sem_t full;      // filled slots
      sem_t mutex;     // mutual exclusion
};
```

---

pc_shm.h

```c
#include "pc_shm.h"

int main() {
    int fd = shm_open("/pcshm", O_CREAT | O_RDWR, 0600);
    ftruncate(fd, sizeof(struct shmseg));
    struct shmseg *shm = mmap(NULL, sizeof(*shm), PROT_READ |
                              PROT_WRITE, MAP_SHARED, fd, 0);

    shm->in = 0;
    shm->out = 0;

    sem_init(&shm->empty, 1, BUF_SIZE);
    sem_init(&shm->full, 1, 0);
    sem_init(&shm->mutex, 1, 1);
    for (int i = 1; i <= 20; i++) {
         sem_wait(&shm->empty); // wait for space
         sem_wait(&shm->mutex); // enter critical section

         shm->buffer[shm->in] = i;
```

```
15          printf("Produced: %d\n", i);
16          shm->in = (shm->in + 1) % BUF_SIZE;

17          sem_post(&shm->mutex); // leave critical section
18          sem_post(&shm->full); // signal item available
19          sleep(1);
20      }
21      return 0;
22  }
```

---

pc_producer.c

The program creates a shared memory object using `shm_open` and then sets its size to that of struct `shmseg` using `ftruncate`.

Next, it maps the shared memory object into the process's address space using `mmap`, allowing it to access and modify the shared data.

At lines 6–7, the two indices `in` and `out` are initialized to 0. These indices represent positions in the circular buffer: in is used by the producer to insert data, while out will be used by the consumer to remove data.

At lines 8–10, the three semaphores are initialized:

- `empty` is initialized to `BUF_SIZE`, representing the number of available slots in the buffer
- `full` is initialized to 0, since the buffer is initially empty
- `mutex` is initialized to 1, ensuring mutual exclusion when accessing the shared buffer

.

The `for` loop performs 20 iterations, producing 20 items. At each iteration, the producer first waits on empty to ensure that there is space available in the buffer, and then waits on `mutex` to enter the critical section.

At line 14, the producer writes the data into the buffer at position in, and at line 16 updates the index using a circular increment.

At line 17, the producer leaves the critical section by posting `mutex`, and then posts the `full` semaphore to notify the consumer that a new item is available in the buffer.

Finally, the `sleep` call slows down production to make the interaction with the consumer easier to observe.

```
1  #include "pc_shm.h"

2  int main() {
3      int fd = shm_open("/pcshm", O_RDWR, 0);

4      struct shmseg *shm = mmap(NULL, sizeof(*shm), PROT_READ |
                                 PROT_WRITE, MAP_SHARED, fd, 0);
```

```
5     for (int i = 1; i <= 20; i++) {
6           sem_wait(&shm→full);
7           sem_wait(&shm→mutex);

8           int item = shm->buffer[shm->out];
9           printf("Consumed: %d\n", item);
10          shm->out = (shm->out + 1) % BUF_SIZE;
11          sem_wait(&shm→mutex);
12          sem_post(&shm->empty);
13          sleep(2);
14    }
15    shm_unlink("/pcshm");
16    return 0;
17 }
```

---

pc_consumer.c

The program opens the existing shared memory object using `shm_open` with read/write access, and maps it into the process's address space using `mmap`, allowing access to the shared buffer and synchronization primitives.

The `for` loop performs 20 iterations, consuming 20 items produced by the *producer*. At each iteration, the *consumer* first waits on the `full` semaphore using `sem_wait`, ensuring that at least one item is available in the buffer.

It then waits on the `mutex` semaphore to enter the critical section, guaranteeing exclusive access to the shared buffer and indices.

Inside the critical section, the *consumer* reads an item from the buffer at position `out`, prints it, and updates the `out` index using a circular increment.

After consuming the item, the *consumer* leaves the critical section by posting the `mutex` semaphore, and then signals that a buffer slot has become free by posting the empty semaphore.
The order of operations, `wait(full) → wait(mutex)`, is important to avoid *deadlocks* and ensure correctness.

The `sleep(2)` call slows down consumption, making the interaction with the *producer* easier to observe.

Finally, the program calls `shm_unlink` to remove the shared memory object. As with the *producer*, the object will only be deleted after all processes have unmapped and closed it.

We can execute the two programs simultaneously.

Shell 1

```
[linux@ipc] ./pc_producer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
```

```
Produced: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Produced: 11
Produced: 12
Produced: 13
...output...
```

Shell 2

```
[linux@ipc] ./pc_consumer
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
...output...
```

As we can see from the output, access to the shared data is properly synchronized between the *producer* and the *consumer.*

The *producer* generates items and inserts them into the shared buffer, while the *consumer* removes and processes them. The use of semaphores ensures that the *producer* does not write when the buffer is full, and the consumer does not read when the buffer is empty.
Moreover, mutual exclusion is guaranteed by the *mutex semaphore*, preventing concurrent access to the shared buffer and avoiding *race conditions*.
The interleaving of *produced* and *consumed* outputs demonstrates that the two processes are correctly coordinated and operate concurrently without interfering with each other.

VI.4.3 Client-server pattern

Another pattern that uses shared memory is the *client–server pattern*, where clients write requests into shared memory and a server process reads, processes them, and writes back responses, with synchronization ensured by semaphores.
In contrast to the *producer–consumer pattern* that is used for continuous data exchange through a shared buffer, the *client–server pattern* is based on request–response interactions where a client asks for a service and the server provides a result.

The following two programs `cs_server.c` and `cs_client.c` compose a client-server mechanism.

The `cs_shm.h` defines the `shmseg` structure.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <semaphore.h>

struct shmseg {
      int request;
      int response;
      sem_t req_sem;     // client → server
      sem_t res_sem;     // server → client
};
```

---

cs_shm.c

The `cs_server.c` program operates in a way similar to the *producer* in the previous program, since it waits for a signal and then writes data into shared memory. However, instead of continuously producing data, it runs in a loop that processes client requests: it waits for a request, performs the required computation, and writes the corresponding response back into shared memory.

```
1 #include "cs_shm.h"

2 int main() {
3     int fd = shm_open("/csshm", O_CREAT | O_RDWR, 0600);
4     ftruncate(fd, sizeof(struct shmseg));

5     struct shmseg *shm = mmap(NULL, sizeof(*shm), PROT_READ |
                                PROT_WRITE, MAP_SHARED, fd, 0);

6     sem_init(&shm->req_sem, 1, 0);
7     sem_init(&shm->res_sem, 1, 0);

8     while (1) {
```

```
9           sem_wait(&shm->req_sem);   // wait for request
10          int x = shm->request;
11          printf("Server received: %d\n", x);
12          shm->response = x * x;     // process request
13          sem_post(&shm->res_sem);   // signal response ready
14      }
15      shm_unlink("/csshm");
16      return 0;
17  }
```

---

cs_server.c Server

After initializing the `req_sem` and `res_sem` semaphores, the program enters an infinite loop to continuously serve client requests.

Inside the loop, at line 9, the server waits for a request by calling `sem_wait` on `req_sem`, blocking until a client signals that a request is available.

At line 10, it reads the request value from shared memory, and at line 12 processes the request (in this case, computing the square of the received number) and stores the result in the response field.

At line 13, the server signals that the response is ready by posting the `res_sem` semaphore, allowing the client to proceed and read the result.

Finally, although `shm_unlink` is present at line 15, it is never reached because the server runs in an infinite loop. In practice, cleanup would require an explicit termination mechanism.

The `cs_client.c` program writes a request into shared memory, signals the server, waits for the response, and then reads the processed result once the server signals completion.

```
1  #include "cs_shm.h"

2  int main() {
3      int fd = shm_open("/csshm", O_RDWR, 0);
4      struct shmseg *shm = mmap(NULL, sizeof(*shm), PROT_READ |
                                 PROT_WRITE, MAP_SHARED, fd, 0);

5      for (int i = 1; i <= 5; i++) {
6          shm->request = i;
7          printf("Client sent: %d\n", i);
8          sem_post(&shm->req_sem);   // send request
9          sem_wait(&shm->res_sem);   // wait for response
10         printf("Client received: %d\n", shm->response);
11     }
12     return 0;
13 }
```

---

cs_client.c Client

After opening the existing shared memory object using `shm_open`, the program maps it into the process's address space using `mmap`, allowing access to the shared request and response fields as well as the synchronization semaphores.

The program then enters a loop where it sends requests to the server. At each iteration, it writes a value into the request field of the shared memory.

Next, it signals the server that a request is available by posting the `req_sem` semaphore using `sem_post`.

It then waits for the server to process the request by calling `sem_wait` on the `res_sem` semaphore, blocking until the response is ready.

Once unblocked, the client reads the result from the response field and prints it.

Let's execute

Shell 1

```
[linux@ipc] ./cs_server
```

The server starts and waits for incoming requests from the client.

Shell 2

```
[linux@ipc] ./cs_client
Client sent: 1
Client received: 1
Client sent: 2
Client received: 4
Client sent: 3
Client received: 9
Client sent: 4
Client received: 16
Client sent: 5
Client received: 25
```

The client sends 5 requests to the server. For each request, it writes a value into shared memory, signals the server, waits for the response, and then prints the result returned by the server.

Shell 1

```
[linux@ipc] ./cs_server
Server received: 1
Server received: 2
Server received: 3
Server received: 4
Server received: 5
```

The client sends 5 requests to the server. For each request, it writes a value into shared memory, signals the server, waits for the response, and then prints the result returned by the server.

From the execution, we can see that the *client* and *server* are correctly synchronized using semaphores. The *client* sends requests and waits for responses, while the *server* processes each request and returns the result. The coordination between `req_sem` and `res_sem` ensures that data is exchanged safely and in the correct order.
When the *client* terminates, the *server* continues its execution (we don’t remove the shared memory object), remaining blocked while waiting for new requests. If the *client* is executed again, it can reconnect to the same *shared memory object*, and the *server* will process the new requests as they arrive.

VI.5 Sockets

The socket-based IPC interface allows processes to communicate with each other regardless of whether they are running on the same machine or across different machines in a network. It supports both local (intra-machine) and network communication.

The POSIX socket API is derived from the 4.4BSD interface, which itself evolved from the original 4.2BSD implementation. Since sockets are a complex and advanced topic that could fill an entire book, we provide only an introductory overview here.

The primary purpose of sockets is network communication. Different domains can be distinguished, each defining how communication is performed. Common socket domains include:

Internet domain
: Used for IPv4 network communication over the Internet

Internet domain IPv6
: Used for IPv6 communication

Unix domain
: Enables communication between processes on the same machine

Packet domain
: Provides low-level access to network packets

Netlink domain
: Used for communication between user space and the kernel

Bluetooth domain
: Supports Bluetooth communication

Sockets provide an interface for communication that relies on underlying protocols (such as TCP or UDP) to define how data is transmitted. They use structured data types to represent and manage the data involved in communication.

VI.5.1 Overview

A *socket* is essentially a communication endpoint identified by a *descriptor*. It is associated with a specific *protocol family* (*domain*) and can operate as either a *stream socket* or a *datagram socket*[17], respectively *connection-oriented* and *connectionless*.

Connection-oriented sockets
: Typically using TCP, establish a dedicated connection between the sender and receiver before any data is exchanged. This involves a handshake process to set up the connection, ensuring both sides are ready. Once established, data is delivered reliably, in order, and without duplication. This makes it suitable for applications like web browsing, email, and file transfer where accuracy matters more than speed

Connectionless sockets
: Typically using UDP, do not establish a formal connection before sending data. Each message (datagram) is sent independently, without guaranteeing delivery, order, or error correction. This makes them faster and more lightweight, but less reliable. They are commonly used in applications like video streaming, online gaming, and DNS queries, where speed is more important than perfect reliability

Once a socket is created, it is represented by a file descriptor, and we can interact with it in different ways:

- We can use blocking or direct I/O functions such as `read` and `write`, or socket-specific calls like `send` and `recv`, to transmit and receive data

- Alternatively, we can use I/O multiplexing mechanisms such as `select`, `poll`, or the more scalable `epoll`, which monitor one or more socket file descriptors and notify the application when they are ready for reading or writing

The *Linux socket interface* is composed by the following functions:

`socket`
: Creates a socket

`connect`
: Connects a socket to a *remote socket address*

`bind`
: Binds a socket to a *local socket address*

`listen`
: Put the socket in the state of accepting new connections

`accept`
: Gets the socket with a new incoming connection

`socketpair`
: Creates two connected *anonymous socket* (for local communication)

17 See the `socket(7)` manual page

`send`, `sendto`, `sendmsg`
: Send data over the socket

`recv`, `recvfrom`, `recvmsg`
: Receive data from a socket

`poll`, `select`
: Wait for arriving data or a readiness to send data. In addition, `write`, `writev`, `sendfile`, `read`, `readv`, can be used to read and write data

`getsockname`
: Returns the local socket address

`getpeername`
: Returns the remote socket address

`getsockopt`, `setsockopt`
: Get and set socket layer and protocol options. `ioctl` can be used to get/set other options

`close`
: Closes a socket

`shutdown`
: Closes part of a *full-duplex socket connection*

The `O_NONBLOCK` flag, if set on a socket, allows *non-blocking I/O*. In this case, all the subsequent blocking operations will return `EAGAIN`, and connect will return `EINPROGRESS`.

The following table lists the I/O events related to `poll` and `select` operations on sockets.

Event	Flag	I/O event
Read	`POLLIN`	New data arrived
Read	`POLLIN`	Connection setup has completed
Read	`POLLHUP`	A disconnection request has been initiated by the other end
Read	`POLLHUP`	Connection is broken
Write	`POLLOUT`	Buffer space is enough for writing new data
Read/Write	`POLLIN | POLLOUT`	`connect` finished
Read/Write	`POLLERR`	Asynchronous error
Read/Write	`POLLHUP`	The other end has shut down one direction
Exception	`POLLPRI`	Urgent data, `SIGURG` is sent

Tab 4. `poll` events on sockets

Besides `poll` and `select`, another approach is to let the kernel notify the application of events by sending a `SIGIO` signal. To use this mechanism, the `O_ASYNC` flag must be enabled on the socket's file descriptor via `fcntl`, and an appropriate `SIGIO` signal handler must be set up using `sigaction`.

Socket type and socket domain

A socket is defined by both its *type* and its *domain*. The *socket type* defines how the socket will communicate, while the *socket domain* (address family) defines the communication protocol family used for addressing. In addition to the *socket domain* and *socket type*, several options can be configured on a socket. The `socket(7)` manual page contains the complete list.

The following table lists some of the most commonly used *socket domains*, along with a brief description and the corresponding manual page. The complete list can be found in the `socket(7)` manual page.[18]

`AF_UNIX`	Local communication	`unix(7)`
`AF_LOCAL`	Synonym for `AF_UNIX`	
`AF_INET`	IPv4 Internet protocols	`ip(7)`
`AF_INET6`	IPv6 Internet protocols	`ipv6(7)`
`AF_KEY`	Key management protocol (IPsec)	
`AF_NETLINK`	Kernel-user interface device	`netlink(7)`
`AF_PACKET`	Low-level packet interface	`packet(7)`

Tab 5. Socket domains

The following table lists the defined types.

`SOCK_STREAM`	Provides sequenced, reliable, two-way, connection-based byte streams (TCP)
`SOCK_DGRAM`	Supports datagrams. Connectionless, unreliable messages of a fixed maximum length (UDP)
`SOCK_SEQPACKET`	Provides a sequenced, reliable, two-way connection-based data transmission path for datagrams of fixed maximum length
`SOCK_RAW`	Provides raw network protocol access
`SOCK_RDM`	Provides a reliable datagram layer that does not guarantee ordering

Tab 6. Socket types

18 The `address_families(7)` manual page contains another detailed list

In a *connection-oriented* communication (SOCK_STREAM), data is treated as a continuous sequence of bytes. Once a connection is established, the client may perform multiple write() operations, while the server retrieves data using read on the associated socket.

However, a fundamental property of stream sockets is that read operations do not correspond to individual write operations. The kernel does not preserve message boundaries; instead, it presents the data as a byte stream.

For example:

Client

```
write("Hello")
write("World")
```

Server

```
read() → "HelloWorld"
```

Or even:

```
read() → "Hel"
read() → "loWo"
read() → "rld"
```

There is no guarantee about how data written by the client is grouped or delivered to the server. The communication is therefore seen purely as a sequence of bytes without inherent structure.

In contrast, *connectionless communication* (SOCK_DGRAM) is *message-oriented*. Each message (datagram) is delivered as a self-contained unit, and message boundaries are preserved by the kernel. Unlike stream sockets, there is no continuous flow of data, but rather a sequence of independent messages, each received exactly as it was sent.

Signals

Concerning signals, there are two specific situation with sockets.

SIGPIPE when writing to a closed socket.
When a process attempts to write to a *connection-oriented socket* that has been closed, either locally or by the remote peer, a SIGPIPE signal is generated, and the call returns an EPIPE error. This signal is suppressed if the MSG_NOSIGNAL flag is specified in the write operation

SIGIO / asynchronous I/O notification.
If the socket is configured using fcntl with FIOSETOWN or ioctl with SIOCSPGRP, the system sends a SIGIO signal whenever an I/O event occurs. Within the signal handler, functions such as poll or select can be used to determine which socket triggered the event. Alternatively, Linux allows the use of real-time signals via fcntl with F_SETSIG; in this case, the signal handler receives the file descriptor through the si_fd field of the siginfo_t structure

`ioctl` operations on sockets

`ioctl` can perform special control operations on sockets, especially for querying or configuring low-level behavior. However, in modern applications it is preferred to use `getsockopt` and `fcntl`. The supported operations are the following:

`SIOCGSTAMP`
: Retrieves a struct `timeval` containing the receive timestamp of the most recent packet delivered to the user. This operation should only be used if the `SO_TIMESTAMP` and `SO_TIMESTAMPNS` options are not enabled. Otherwise, it returns the timestamp of the last packet received before those options were set, or fails (returning `-1` with errno set to `ENOENT`) if no such packet exists

`SIOCSPGRP`
: Sets the process or process group that will receive `SIGIO` or `SIGURG` signals when I/O becomes available or urgent data arrives. The argument is a pointer to a `pid_t`. This operation is equivalent to `FIOGETOWN` when used with `fcntl`

`FIOASYNC`
: Enables or disables *asynchronous I/O* mode by modifying the `O_ASYNC` flag on the socket. In this mode, a `SIGIO` signal (or a signal specified via `F_SETSIG`) is generated whenever an I/O event occurs. The argument is a boolean integer. This operation is equivalent to setting the `O_ASYNC` flag using `fcntl`

`SIOCGPGRP`
: Retrieves the process or process group currently designated to receive `SIGIO` or `SIGURG` signals. Returns 0 if none is set. This operation is equivalent to `FIOGETOWN` when used with `fcntl`

VI.5.2 Creating sockets: `socket` and `shutdown`

To create a socket we can use the `socket`[19] function, which is defined as follows

```
#include <sys/socket.h>

int socket(int domain, int type, int protocol);

            Returns: on success a file descriptor, on error -1 and errno is set
```

The function creates an enpoint for communication and returns a file descriptor referring to the endpoint.

The `domain` argument specifies the *socket domain*. The related protocol family will be used by the socket for the communication.

The `type` argument specifies the *socket type*. Not all the socket types are implemented by all protocol families.

In addition, the following flags can be ORed with `type`:

`SOCK_NONBLOCK`
: Set the `O_NONBLOCK` file status flag on the open file description referred to by the new file descriptor

`SOCK_CLOEXEC`
: Set the close-on-exec (`FD_CLOEXEC`) flag on the new file descriptor

The `protocol` parameter specifies which protocol should be used with the socket. In most cases, only one protocol supports a given socket type within a particular protocol family, so this parameter can be set to `0`. However, if multiple protocols are available, a specific protocol must be explicitly selected.
The *protocol number* depends on the communication *domain* in which the socket operates.[20] To map protocol names to their corresponding numbers, the `getprotoent` function can be used.

`SOCK_STREAM` sockets provide full-duplex, byte-stream communication and do not preserve message boundaries. A *stream socket* must be connected before data can be transmitted or received. The usual patter is as follows:

- Connection is established using `connect`
- Data are transfered via `read` and `write`, or through `send` and `recv`
- Once communication is complete, the connection is closed using `close`

Out-of-band data can also be sent and received using the appropriate `send` and `recv` mechanisms.

Protocols implementing `SOCK_STREAM` ensure reliable communication: data is neither lost nor duplicated. If data cannot be delivered within a reasonable time (for example, due to lack of buffer space on the receiving side), the connection is considered broken. When the `SO_KEEPALIVE` option is enabled, the protocol periodically checks whether the peer is still reachable. If a process

19 See the `socket(2)` manual page
20 The `/etc/protocols` file defines the protocols numbers. See the `protocols(5)` manual page

attempts to send or receive data on a broken connection, a `SIGPIPE` signal is generated, which may terminate the process if not handled.

`SOCK_SEQPACKET` sockets use the same system calls as `SOCK_STREAM`, but preserve message boundaries. Each `read` call returns only the requested amount of data, and any excess data from the current packet is discarded.

`SOCK_DGRAM` and `SOCK_RAW` sockets support connectionless communication. Data is sent as discrete datagrams using `sendto` and received with `recvfrom`, which also provides the sender's address.

When the network reports an error to the protocol module (for example, via an ICMP message in IP), the socket is marked with a pending error. The next operation performed on that socket will return the corresponding error code. For some protocols, it is possible to enable a per-socket error queue to obtain more detailed error information.[21]

In full-duplex communication, `shutdown` allows selectively disabling reading, writing, or both, while `close` completely closes the socket and releases its resources.

The `shutdown`[22] function is defined as follows

```
#include <sys/socket.h>

int shutdown(int sockfd, int how);

                              Returns: on success 0, on error -1 and errno is set
```

The `shutdown` function causes all or part of a full-duplex connection on the socket associated with `sockfd` to be shut down.

If the `how` argument is `SHUT_RD` (`0`), further receptions will be disallowed.
If `how` is `SHUT_WR` (`1`), further transmissions will be disallowed.
If `how` is `SHUT_RDWR` (`2`), further receptions and transmissions will be disallowed.

`shutdown` is useful because it affects the underlying socket immediately, regardless of how many file descriptors refer to it. In contrast, `close` only releases the socket when the last reference is closed. If multiple file descriptors (e.g., created with `dup`) refer to the same socket, `close` on one descriptor does not terminate communication, whereas `shutdown` can explicitly disable sending, receiving, or both.

21 See the section `IP_RECVERR` in the `ip(7)` manual page for further details
22 See the `shutdown(2)` manual page

VI.5.3 Byte order conversion functions

A processor uses a specific byte ordering scheme to store multi-byte data in memory. The two main types of ordering are *big-endian* and *little-endian*.

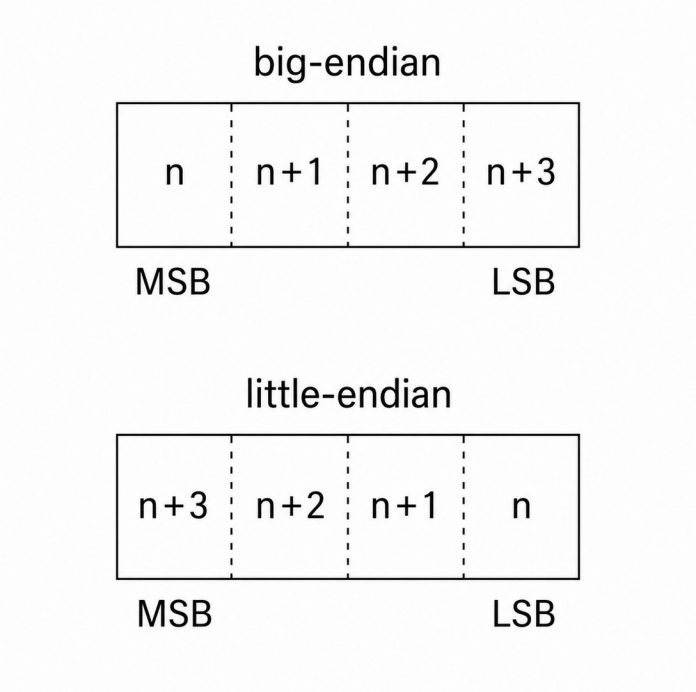

Fig 1. Byte order in a 32-bit integer

In big-endian systems, the *most significant byte* (MSB) is stored at the *lowest memory address*. In little-endian systems, the *least significant byte* (LSB) is stored at the *lowest memory address*. Regardless of the byte ordering, the *most significant byte* is conventionally written on the left and the least significant byte on the right.

A 32-bit value as `0x12345678` wil be stored as follows (the first colon is the address):

- *Big endian* (MSB first)

  0x00 `0x12` (highest byte at lowest address)
  0x01 `0x34`
  0x02 `0x56`
  0x03 `0x78`

- *Little endian* (LSB first)

  0x00 `0x78` (lowest byte at lowest address)
  0x01 `0x56`
  0x02 `0x34`
  0x03 `0x12`

Network protocols define a standard *byte ordering* to allow different computer systems to exchange data correctly. TCP/IP uses *big-endian* format, also known as *network byte order.* Therefore, multi-byte values such as *addresses* and *port numbers* must be represented in this format when transmitted over the network. When working with these values in a program, it may be necessary to convert between the *host byte order* and the *network byte order.*

This conversion can be performed using the following functions[23].

```
#include <arpa/inet.h>

uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);

uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

The `htonl` function converts the unsigned integer `hostlong` from host byte order to network byte order.
The `htons` function converts the unsigned short integer `hostshort` from host byte order to network byte order.
The `ntohl` function converts the unsigned integer `netlong` from network byte order to host byte order.
The `ntohs` function converts the unsigned short integer `netshort` from network byte order to host byte order.

These functions use the following conventions: `h` stands for *host* byte order, `n` stands for *network* byte order, `l` stands for *long* (i.e., 4-byte) integer, and `s` stands for *short* (i.e., 2-byte) integer.

In addition to these functions, which were designed for use with TCP/IP, more general byte-order conversion functions are provided that allow explicit conversion between host, big-endian, and little-endian formats. The following functions[24] are useful when working with data formats that require a specific byte order, independent of network protocols.

```
#include <endian.h>

uint16_t htobe16(uint16_t host_16bits);
uint16_t htole16(uint16_t host_16bits);
uint16_t be16toh(uint16_t big_endian_16bits);
uint16_t le16toh(uint16_t little_endian_16bits);

uint32_t htobe32(uint32_t host_32bits);
uint32_t htole32(uint32_t host_32bits);
uint32_t be32toh(uint32_t big_endian_32bits);
uint32_t le32toh(uint32_t little_endian_32bits);

uint64_t htobe64(uint64_t host_64bits);
uint64_t htole64(uint64_t host_64bits);
uint64_t be64toh(uint64_t big_endian_64bits);
uint64_t le64toh(uint64_t little_endian_64bits);
```

These functions convert the byte encoding of integer values from the byte order that the current CPU (host) uses, to and from little-endian and big-endian byte order.

23 See the `BYTEORDER(3)` manual page
24 See the `endian(3)` manual page

The suffix `nn` in each function name indicates the size of the integer handled by the function, typically 16, 32, or 64 bits.
Functions with names of the form `htobenn` convert values from host byte order to big-endian format.
Functions with names of the form `htolenn` convert values from host byte order to little-endian format.
Functions with names of the form `benntoh` convert values from big-endian format to host byte order.
Functions with names of the form `lenntoh` convert values from little-endian format to host byte order.

The `endian(3)` manual page includes and example of the use of `htole32` and `hotobe32`.

VI.5.4 Addressing: `getaddrinfo` and `getnameinfo`

In socket-based communication, a *peer* is identified by its *network address* (host) and a *service* (port number), which together define the *communication endpoint*. The `getaddrinfo`[25] function translates a hostname and service into one or more *socket address structures* using *system name resolution* mechanisms such as DNS.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *restrict node, const char *restrict service,
                const struct addrinfo *restrict hints, struct addrinfo
                **restrict res);

void freeaddrinfo(struct addrinfo *res);

const char *gai_strerror(int errcode);

                                  Returns: 0 on success, on error an error code
```

The `node` argument specifies an Internet host. The `service` argument specifies a service. `getaddrinfo` returns one or more `addrinfo` structures, each of which contains an Internet address that can be used in subsequent bind and connect calls.

The `addrinfo` structure is defined as follows in the `netdb.h` header file

```
struct addrinfo {
   int ai_flags;
   int ai_family;
   int ai_socktype;
   int ai_protocol;
   socklen_t ai_addrlen;
   struct sockaddr *ai_addr;
   char *ai_canonname;
  struct addrinfo *ai_next;
};
```

The `hints` argument is a pointer to an `addrinfo` structure that defines criteria for selecting the socket addresses returned in the list referenced by `res`. If `hints` is not NULL, it specifies constraints through its `ai_family`, `ai_socktype`, and `ai_protocol` fields, which limit the set of addresses returned by `getaddrinfo`:

`ai_family`
: Specifies the desired *address family* (`AF_INET` for IPv4, `AF_INET6` for IPv6, etc..). The value `AF_UNSPEC` indicates that addresses from any supported family may be returned

25 See the `getaddrinfo(2)` manual page

`ai_socktype`
: Specifies the preferred *socket type* (`SOCK_STREAM`, `SOCK_DGRAM`). A value of 0 allows addresses of any socket type to be returned

`ai_protocol`
: Specifies the protocol associated with the returned addresses. A value of 0 allows any protocol to be used

`ai_flags`
: Specifies additional options. Multiple flags can be combined using a bitwise OR operation

All other fields in the `addrinfo` structure pointed to by `hints` must be set to `0` or `NULL`, as appropriate.

Specifying `hints` as `NULL` is equivalent to setting `ai_socktype` and `ai_protocol` to 0; `ai_family` to `AF_UNSPEC`; and `ai_flags` to (`AI_V4MAPPED | AI_ADDRCONFIG`).

`node` specifies either a numerical *network address:*

- IPv4 → numbers-and-dots notation as supported by inet_aton
- IPv6 → hexadecimal string format as supported by inet_pton

or a network hostname, whose network addresses are looked up and resolved.

If `AI_NUMERICHOST` is specified, `node` must be a numeric address, and hostname resolution is skipped.

When `AI_PASSIVE` is set with a NULL `node`, `getaddrinfo` returns wildcard addresses for use with `bind`; otherwise, it returns addresses suitable for `connect` and related calls, using the given `node` or the loopback address (`INADDR_LOOPBACK` for IPv4, `IN6ADDR_LOOPBACK_INIT` for IPv6) if none is provided.
Setting the `AI_PASSIVE` flag determines the type of addresses returned by `getaddrinfo`, and thus whether the socket is intended to be used as a *listening* (passive) socket or a *connecting* (active) socket.

The `service` argument specifies the port number in each returned address structure. It can be given either as a service name[26], which is translated into the corresponding port number, or as a numeric string, which is directly converted. If `service` is NULL, the port field in the returned addresses is left uninitialized.
If the `AI_NUMERICSERV` flag is set and `service` is not NULL, then service must be a numeric port string; this prevents any name resolution.
At least one of `node` or `service` must be provided, but both cannot be NULL.

The `getaddrinfo` function allocates and initializes a linked list of `addrinfo` structures, one for each network address that matches the specified `node` and `service`, subject to the constraints defined in `hints`. A pointer to the first element of this list is returned in `res`, and the entries are linked together via the `ai_next` field.

The returned linked list may have more than one `addrinfo` structure for the following reasons:

26 See the `services(5)` manual page

- The network host is multihomed, accessible over multiple protocols (e.g., both `AF_INET` and `AF_INET6`)

- The same service is available from multiple socket types, e.g., one `SOCK_STREAM` address and another `SOCK_DGRAM` address

The application should normally try using the addresses in the order in which they are returned, as specified in RFC 3484[27]. However, this order can be adjusted by editing the `/etc/gai.conf` file.

If `hints.ai_flags` includes the `AI_CANONNAME` flag, then the `ai_canonname` field of the first of the `addrinfo` structures in the returned list is set to point to the official name of the host.

The remaining fields of each returned `addrinfo` structure are initialized as follows:

- `ai_family`, `ai_socktype`, and `ai_protocol` return the socket creation parameters (same meaning as the corresponding arguments of `socket`)

- A pointer to the socket address is placed in the `ai_addr` field, and the size of the socket address, in bytes, is placed in the `ai_addrlen` field

If `hints.ai_flags` includes the `AI_ADDRCONFIG` flag, then IPv4 addresses are returned in the list pointed to by `res` only if the local system has at least one IPv4 address configured, and IPv6 addresses are returned only if the local system has at least one IPv6 address configured. The loopback address is not considered for this case as valid as a configured address.

If `AI_V4MAPPED` is set and `ai_family` is `AF_INET6`, then IPv4-mapped IPv6 addresses are returned when no matching IPv6 addresses are found. If both `AI_V4MAPPED` and `AI_ALL` are specified, then both native IPv6 and IPv4-mapped IPv6 addresses are returned. The `AI_ALL` flag has no effect unless `AI_V4MAPPED` is also set.

The `freeaddrinfo` function frees the memory that was allocated for the linked list `res`.

The `gai_strerror` function converts the error code returned by `getaddrinfo` into a descriptive, human-readable string that can be printed to report address resolution errors.

While `socket`, `bind`, and `connect` are sufficient to establish a connection, `getaddrinfo` facilitates this process by resolving hostnames to IP addresses and supplying initialized `sockaddr` structures, effectively indicating where and how to connect.

Other functions are also available to retrieve information:

- Host: `gethostbyname`, `gethostbyaddr`, `gethostent`
- Network: `getnetbyname`, `getnetbyaddr`, `getnetent`
- Protocol: `getprotobyname`, `getprotobynumber`, `getprotoent`

Each of these function has a dedicated manual page in the section 3.

27 For details about RFC 3484 see the https://www.rfc-editor.org/rfc/rfc3484

The `getnameinfo`[28] function performs the inverse operation of `getaddrinfo`.

```
#include <sys/socket.h>
#include <netdb.h>

int getnameinfo(const struct sockaddr *restrict addr, socklen_t addrlen,
                char host[_Nullable restrict hostlen],
                socklen_t hostlen,char serv[_Nullable restrict servlen],
                socklen_t servlen, int flags);

                                Returns: on success 0, on error a nonzero error code
```

`getnameinfo` converts a socket address to a corresponding host and service, in a protocol-independent manner.

The `addr` argument is a pointer to a generic socket address structure, such as `sockaddr_in` or `sockaddr_in6`, of length `addrlen`, containing the input IP address and port number.
The `host` and `serv` arguments are pointers to caller-allocated buffers (with sizes `hostlen` and `servlen`) where `getnameinfo` stores the resulting null-terminated hostname and service name strings.
The caller may indicate that either the hostname or service name is not needed by passing `NULL` for `host` or `serv`, or by setting `hostlen` or `servlen` to zero. However, at least one of these. hostname or service name, must be requested.

The `flags` argument modifies the behavior of the call:

`NI_NAMEREQD`
: An error is returned if the hostname cannot be determined

`NI_DGRAM`
: The service is datagram (UDP) based rather than stream (TCP) based

`NI_NOFQDN`
: Return only the hostname part of the fully qualified domain name for local hosts

`NI_NUMERICHOST`
: The numeric form of the hostname is returned

`NI_IDN`
: The name found in the lookup process is converted from IDN[29] format to the locale's encoding if necessary

`NI_IDN_ALLOW_UNASSIGNED`
`NI_IDN_USE_STD3_ASCII_RULES`
: Enable the `IDNA_ALLOW_UNASSIGNED` (allow unassigned Unicode code points) and `IDNA_USE_STD3_ASCII_RULES` (check output to make sure it is a STD3 conforming hostname) flags respectively to be used in the IDNA handling

28 See the `getnameinfo(3)` manual page
29 Internationalized Domain Name

The following two constantes are defined in the netdb.h header file, and represent the recommended buffer size:

```
NI_MAXHOST      1025
NI_MAXSERV      32
```

The following table lists some of the most commonly used address families.

`AF_LOCAL`	Local (same-machine) communication
`AF_UNIX`	Synonym for `AF_LOCAL` (widely used, portable)[30]
`AF_FILE`	Synonym for `AF_LOCAL` (compatibility)
`AF_INET`	IPv4 Internet protocols[31]
`AF_INET6`	IPv6 Internet protocols[32]
`AF_NETLINK`	Kernel user interface device[33]
`AF_UNSPEC`	No specific address family (unspecified)

Tab 7. Address families

The following table lists some of the most commonly used protocols.[34]

`IPPROTO_TCP`	Transmission Control Protocol (reliable, connection-oriented)
`IPPROTO_UDP`	User Datagram Protocol (connectionless, unreliable)
`IPPROTO_IP`	Default IP protocol (used with IPv4)
`IPPROTO_IPV6`	IPv6 protocol
`IPPROTO_ICMP`	Internet Control Message Protocol (IPv4)
`IPPROTO_ICMPV6`	ICMP for IPv6
`IPPROTO_RAW`	Raw IP packets

Tab 8. Protocols

The `getaddrinfo.c` program is a modified version of the one in APUE[35]. It uses `getaddrinfo` to obtain address information for the given host and service, and `getnameinfo` to convert that information into readable host and service names.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
```

30 See the `unix(7)` manual page
31 See the `ip(7)` manual page
32 See the `ipv6(7)` manual page
33 See the `netlink(7)` manual page
34 The `address_families(7)` manual page contains the complete list
35 APUE, Section 16.3, fig. 16.9, pag. 601

```
4 #include <arpa/inet.h>
5 #include <netdb.h>
6 #include <sys/socket.h>
7 #include <netinet/in.h>

8 void print_family(struct addrinfo *aip)
9 {
10    printf(" family ");
11    switch (aip->ai_family) {
12    case AF_INET:
13          printf("inet");
14          break;
15    case AF_INET6:
16          printf("inet6");
17          break;
18 #ifdef AF_UNIX
19    case AF_UNIX:
20          printf("unix");
21          break;
22 #endif
23    case AF_UNSPEC:
24          printf("unspecified");
25          break;
26    default:
27          printf("unknown");
28    }
29 }

30 void print_type(struct addrinfo *aip)
31 {
32    printf(" type ");
33    switch (aip->ai_socktype) {
34    case SOCK_STREAM:
35          printf("stream");
36          break;
37    case SOCK_DGRAM:
38          printf("datagram");
39          break;
40    case SOCK_SEQPACKET:
41          printf("seqpacket");
42          break;
43    case SOCK_RAW:
44          printf("raw");
45          break;
46    default:
47          printf("unknown (%d)", aip->ai_socktype);
48    }
49 }

50 void print_protocol(struct addrinfo *aip)
51 {
```

```
52    printf(" protocol ");
53    switch (aip->ai_protocol) {
54    case 0:
55          printf("default");
56          break;
57    case IPPROTO_TCP:
58          printf("TCP");
59          break;
60    case IPPROTO_UDP:
61          printf("UDP");
62          break;
63    case IPPROTO_RAW:
64          printf("raw");
65          break;
66    default:
67          printf("unknown (%d)", aip->ai_protocol);
68    }
69 }

70 void print_flags(struct addrinfo *aip)
71 {
72    printf(" flags");
73    if (aip->ai_flags == 0) {
74          printf(" 0");
75    } else {
76          if (aip->ai_flags & AI_PASSIVE)
77                printf(" passive");
78          if (aip->ai_flags & AI_CANONNAME)
79                printf(" canon");
80          if (aip->ai_flags & AI_NUMERICHOST)
81                printf(" numhost");
82          if (aip->ai_flags & AI_NUMERICSERV)
83                printf(" numserv");
84          if (aip->ai_flags & AI_V4MAPPED)
85                printf(" v4mapped");
86          if (aip->ai_flags & AI_ALL)
87                printf(" all");
88    }
89 }

90 int main(int argc, char *argv[])
91 {
92    struct addrinfo *ailist, *aip;
93    struct addrinfo hint;
94    int err;

95    char host[NI_MAXHOST];
96    char serv[NI_MAXSERV];

97    if (argc != 3) {
98          fprintf(stderr, "usage: %s nodename service\n", argv[0]);
```

```
99          exit(1);
100     }

101     memset(&hint, 0, sizeof(hint));
102     hint.ai_flags = AI_CANONNAME;

103     if ((err = getaddrinfo(argv[1], argv[2], &hint, &ailist)) != 0) {
104          fprintf(stderr, "getaddrinfo error: %s\n", gai_strerror(err));
105          exit(1);
106     }

107     for (aip = ailist; aip != NULL; aip = aip->ai_next) {
108          print_flags(aip);
109          print_family(aip);
110          print_type(aip);
111          print_protocol(aip);
112          printf("\n\thost %s", aip->ai_canonname ? aip->ai_canonname :
                    "-");

113          if ((err = getnameinfo(aip->ai_addr, aip→ai_addrlen, host,
                 sizeof(host), serv, sizeof(serv), NI_NUMERICHOST |
                 NI_NUMERICSERV)) != 0) {
114               fprintf(stderr, "\n\tgetnameinfo error: %s\n",
                             gai_strerror(err));
115          } else {
116               printf(" address %s", host);
117               printf(" port %s", serv);
118          }
119          printf("\n");
120     }
121     freeaddrinfo(ailist);
122     return 0;
123 }
```

---

getaddrinfo.c

The `print_family`, `print_type`, `print_protocol`, and `print_flags` functions each take an `addrinfo` structure returned by `getaddrinfo` and examine the corresponding fields to print their values in a human-readable form.

The `main` function declares two pointers to `addrinfo` structures, `ailist` and `aip`, used respectively to store the list returned by `getaddrinfo` and to iterate over it. It also declares a `hint` structure, which is initialized to zero and used to specify the criteria for address resolution. In this case, only the `AI_CANONNAME` flag is set, requesting the canonical name of the host.

After validating the command-line arguments, the program calls `getaddrinfo` with the provided node name and service. On success, it returns a linked list of `addrinfo` structures.
The program then iterates through this list, printing the flags, address family, socket type, and protocol for each entry, along with the canonical name if available.

To display the address and port in a readable format, the program uses `getnameinfo`, which converts the binary socket address into numeric host and service strings.
Finally, once all entries have been processed, the allocated list is released using `freeaddrinfo`.

Let's execute the program

```
[linux@ipc] ./getaddrinfo localhost nfs
 flags canon family inet6 type stream protocol TCP
	host localhost address ::1 port 2049
 flags canon family inet6 type datagram protocol UDP
	host - address ::1 port 2049
 flags canon family inet type stream protocol TCP
	host - address 127.0.0.1 port 2049
 flags canon family inet type datagram protocol UDP
	host - address 127.0.0.1 port 2049
```

From the output, we can observe that the host `localhost` resolves to both `::1` (IPv6) and `127.0.0.1` (IPv4), and that the `nfs` service supports both TCP and UDP. The service `nfs` is mapped to port `2049`, as defined in the system services database (e.g., `/etc/services`).
Only the first entry contains the canonical name, as requested with the `AI_CANONNAME` flag. In subsequent entries, the canonical name is not set (displayed as `–`), since `ai_canonname` is typically populated in only one structure, depending on the implementation.

To retrieve address information, `getaddrinfo` consults the Name Service Switch (NSS), which performs the lookup according to the system configuration defined in `/etc/nsswitch.conf`.

We can execute the program with `strace` to analyze the process `getaddrinfo` performs.

```
[linux@ipc] strace ./getaddrinfo localhost nfs

openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
st_size=585, ...}, 0) = 0
openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
st_size=585, ...}, 0) = 0
openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
st_size=585, ...}, 0) = 0
openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
st_size=585, ...}, 0) = 0
openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
```

```
st_size=585, ...}, 0) = 0
openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
st_size=585, ...}, 0) = 0
openat(AT_FDCWD, "/etc/services", O_RDONLY|O_CLOEXEC) = 3
...output...
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644,
st_size=585, ...}, 0) = 0
newfstatat(AT_FDCWD, "/etc/resolv.conf", {st_mode=S_IFREG|0644,
st_size=96, ...}, 0) = 0
openat(AT_FDCWD, "/etc/host.conf", O_RDONLY|O_CLOEXEC) = 3
...output...
futex(0x7f0407a4872c, FUTEX_WAKE_PRIVATE, 2147483647) = 0
openat(AT_FDCWD, "/etc/resolv.conf", O_RDONLY|O_CLOEXEC) = 3
...output...
openat(AT_FDCWD, "/etc/hosts", O_RDONLY|O_CLOEXEC) = 3
...output...
read(3, "127.0.0.1\tlocalhost\n127.0.1.1\tka"..., 4096) = 184
...output...
openat(AT_FDCWD, "/etc/gai.conf", O_RDONLY|O_CLOEXEC) = 3
...output...
futex(0x7f0407a4df84, FUTEX_WAKE_PRIVATE, 2147483647) = 0
socket(AF_NETLINK, SOCK_RAW|SOCK_CLOEXEC, NETLINK_ROUTE) = 3
bind(3, {sa_family=AF_NETLINK, nl_pid=0, nl_groups=00000000}, 12) = 0
getsockname(3, {sa_family=AF_NETLINK, nl_pid=1875866, nl_groups=00000000},
[12]) = 0
sendto(3, [{nlmsg_len=20, nlmsg_type=RTM_GETADDR, nlmsg_flags=NLM_F_REQUEST|
NLM_F_DUMP, nlmsg_seq=1775397021, nlmsg_pid=0}, {ifa_family=AF_UNSPEC, ...}],
20, 0, {sa_family=AF_NETLINK, nl_pid=0, nl_groups=00000000}, 12) = 20
recvmsg(3, {msg_name={sa_family=AF_NETLINK, nl_pid=0, nl_groups=00000000},
msg_namelen=12, msg_iov=[{iov_base=[[{nlmsg_len=76, nlmsg_type=RTM_NEWADDR,
nlmsg_flags=NLM_F_MULTI, nlmsg_seq=1775397021, nlmsg_pid=1875866},
{ifa_family=AF_INET, ifa_prefixlen=8, ifa_flags=IFA_F_PERMANENT,
ifa_scope=RT_SCOPE_HOST, ifa_index=if_nametoindex("lo")}, [[{nla_len=8,
nla_type=IFA_ADDRESS}, inet_addr("127.0.0.1")], [{nla_len=8,
nla_type=IFA_LOCAL}, inet_addr("127.0.0.1")], [{nla_len=7, nla_type=IFA_LABEL},
"lo"], [{nla_len=8, nla_type=IFA_FLAGS}, IFA_F_PERMANENT], [{nla_len=20,
nla_type=IFA_CACHEINFO}, {ifa_prefered=4294967295, ifa_valid=4294967295,
cstamp=840, tstamp=840}]]], [{nlmsg_len=88, nlmsg_type=RTM_NEWADDR,
nlmsg_flags=NLM_F_MULTI, nlmsg_seq=1775397021, nlmsg_pid=1875866},
{ifa_family=AF_INET, ifa_prefixlen=24, ifa_flags=0,
ifa_scope=RT_SCOPE_UNIVERSE, ifa_index=if_nametoindex("wlan0")}, [[{nla_len=8,
nla_type=IFA_ADDRESS}, inet_addr("192.168.1.23")], [{nla_len=8,
nla_type=IFA_LOCAL}, inet_addr("192.168.1.23")], [{nla_len=8,
nla_type=IFA_BROADCAST}, inet_addr("192.168.1.255")], [{nla_len=10,
nla_type=IFA_LABEL}, "wlan0"], [{nla_len=8, nla_type=IFA_FLAGS},
IFA_F_NOPREFIXROUTE], [{nla_len=20, nla_type=IFA_CACHEINFO},
{ifa_prefered=78313, ifa_valid=78313, cstamp=25408202, tstamp=25408202}]]],
[{nlmsg_len=72, nlmsg_type=RTM_NEWADDR, nlmsg_flags=NLM_F_MULTI,
```

```
nlmsg_seq=1775397021, nlmsg_pid=1875866}, {ifa_family=AF_INET6,
ifa_prefixlen=128, ifa_flags=IFA_F_PERMANENT, ifa_scope=RT_SCOPE_HOST,
ifa_index=if_nametoindex("lo")}, [[{nla_len=20, nla_type=IFA_ADDRESS},
inet_pton(AF_INET6, "::1")], [{nla_len=20, nla_type=IFA_CACHEINFO},
{ifa_prefered=4294967295, ifa_valid=4294967295, cstamp=840, tstamp=840}],
[{nla_len=8, nla_type=IFA_FLAGS}, IFA_F_PERMANENT|IFA_F_NOPREFIXROUTE]]],
[{nlmsg_len=72, nlmsg_type=RTM_NEWADDR, nlmsg_flags=NLM_F_MULTI,
nlmsg_seq=1775397021, nlmsg_pid=1875866}, {ifa_family=AF_INET6,
ifa_prefixlen=64, ifa_flags=IFA_F_PERMANENT, ifa_scope=RT_SCOPE_LINK,
ifa_index=if_nametoindex("wlan0")}, [[{nla_len=20, nla_type=IFA_ADDRESS},
inet_pton(AF_INET6, "fe80::8655:3896:56e9:8f73")], [{nla_len=20,
nla_type=IFA_CACHEINFO}, {ifa_prefered=4294967295, ifa_valid=4294967295,
cstamp=25408176, tstamp=25408176}], [{nla_len=8, nla_type=IFA_FLAGS},
IFA_F_PERMANENT|IFA_F_NOPREFIXROUTE]]]], iov_len=4096}], msg_iovlen=1,
msg_controllen=0, msg_flags=0}, 0) = 308
recvmsg(3, {msg_name={sa_family=AF_NETLINK, nl_pid=0, nl_groups=00000000},
msg_namelen=12, msg_iov=[{iov_base=[{nlmsg_len=20, nlmsg_type=NLMSG_DONE,
nlmsg_flags=NLM_F_MULTI, nlmsg_seq=1775397021, nlmsg_pid=1875866}, 0],
iov_len=4096}], msg_iovlen=1, msg_controllen=0, msg_flags=0}, 0) = 20
close(3)                                = 0
socket(AF_INET, SOCK_DGRAM|SOCK_CLOEXEC, IPPROTO_IP) = 3
connect(3, {sa_family=AF_INET, sin_port=htons(2049),
sin_addr=inet_addr("127.0.0.1")}, 16) = 0
getsockname(3, {sa_family=AF_INET, sin_port=htons(36191),
sin_addr=inet_addr("127.0.0.1")}, [28 => 16]) = 0
close(3)                                = 0
socket(AF_INET6, SOCK_DGRAM|SOCK_CLOEXEC, IPPROTO_IP) = 3
connect(3, {sa_family=AF_INET6, sin6_port=htons(2049), sin6_flowinfo=htonl(0),
inet_pton(AF_INET6, "::1", &sin6_addr), sin6_scope_id=0}, 28) = 0
getsockname(3, {sa_family=AF_INET6, sin6_port=htons(48990),
sin6_flowinfo=htonl(0), inet_pton(AF_INET6, "::1", &sin6_addr),
sin6_scope_id=0}, [28]) = 0
close(3)                                = 0
```

From this `trace` we can see that `getaddrinfo` performs three main tasks:

1. Service resolution → `/etc/services`
2. Host resolution (NSS) → `/etc/nsswitch.conf`, `/etc/hosts`, DNS
3. Local interface discovery → `netlink` (kernel)

The `/etc/services` file is opened multiple times and scanned to resolve the `nfs` service. This lookup maps the service name to port `2049` for both TCP and UDP, which explains why entries for both `SOCK_STREAM` and `SOCK_DGRAM` are later returned.
The `/etc/nsswitch.conf` file is consulted to determine the order in which name resolution sources are queried.
The `/etc/resolv.conf` file is read to load the DNS resolver configuration, such as the list of nameservers. Although DNS is configured, it is not used in this case because the hostname is resolved locally.

The /etc/hosts file is then checked to resolve the hostname localhost

```
read(3, "127.0.0.1\tlocalhost\n...", 4096) = 184
```

From this file, localhost is resolved locally to both 127.0.0.1 (IPv4) and ::1 (IPv6), avoiding any DNS query.

The /etc/gai.conf file is read to obtain address selection and sorting rules, which influence the order in which the resulting addresses are returned.

The kernel is then queried for available network interfaces and assigned addresses using a netlink socket (AF_NETLINK). Through this mechanism, getaddrinfo retrieves information about the system's network configuration, including

- 127.0.0.1 (loopback interface, lo)
- 192.168.1.23 (network interface, wlan0)
- ::1 (IPv6 loopback)
- fe80::... (IPv6 link-local address)

This step allows the system to determine which address families (IPv4 and/or IPv6) are actually configured and usable.

Finally, it performs a form of connectivity probing by creating UDP sockets and connecting them to the resolved addresses. This does not send any packets, but allows the kernel to select the appropriate source address and routing information, which helps refine the ordering and suitability of the returned results.

The strace output shows that getaddrinfo performs a complex resolution process: it queries local databases such as /etc/services and /etc/hosts, follows the lookup rules defined by NSS in /etc/nsswitch.conf, optionally consults DNS, retrieves network configuration from the kernel via netlink, and applies address selection policies before returning a list of suitable socket addresses.

If the program is executed with a remote host, additional system calls will be observed. In this case, a socket is created to perform a DNS query, sending the request to the configured nameserver defined in /etc/resolv.conf, which is often the local router or a designated DNS server.

VI.5.5 Address formats: `inet_ntop` and `inet_pton`

The *protocol families*, the *address families* and the `sockaddr`[36] structure, are defined in the `bits/socket.h` header file[37]. The `addrinfo` structure is defined in the `netdb.h` header file.

```
struct sockaddr {
   sa_family_t sa_family; /* Address family */
   char sa_data[];        /* Socket address */
};
```

The `sa_family_t` describes a socket's protocol family.

`sa_data` holds raw, protocol-specific address information (such as port and address), but it should not be accessed directly, instead we must use appropriate specialized structure.

For the IPv4 and IPv6 addresses, the `sockaddr_in` and `sockaddr_in6` structures[38] are defined in the `netinet/in.h` header file.

```
#include <netinet/in.h>

struct sockaddr_in {
   sa_family_t sin_family;   /* AF_INET */
   in_port_t sin_port;       /* Port number */
   struct in_addr sin_addr;  /* IPv4 address */
};

struct in_addr {
   in_addr_t s_addr;
};

typedef uint32_t in_addr_t;
typedef uint16_t in_port_t;
```

```
#include <netinet/in.h>

struct sockaddr_in6 {
   sa_family_t sin6_family;   /* AF_INET6 */
   in_port_t sin6_port;       /* Port number */
   uint32_t sin6_flowinfo;    /* IPv6 flow info */
   struct in6_addr sin6_addr; /* IPv6 address */
   uint32_t sin6_scope_id;    /* Set of interfaces for a scope */
};

struct in6_addr {
   uint8_t s6_addr[16];
};
```

36 See the `sockaddr(3type)` manual page
37 On Debian-based systems `x86_64-linux-gnu/bits/socket.h`
38 See the `sockaddr_in(3type)` and `sockaddr_in6(3type)` manual pages

Both the structures use specific data types to hold the address:

- struct `in_addr` with an `in_addr_t` field (`uint32_t`) → IPv4
- struct `in6_addr` with an `uint8_t` field → IPv6

When we need to print these addresses in an understandable format we can use the `inet_ntop`[39] functions.

```
#include <arpa/inet.h>

const char *inet_ntop(int af, const void *restrict src, char
                      dst[restrict size], socklen_t size);

        Returns: on success a non-null pointer, on error NULL and errno is set
```

The `inet_ntop` function converts the network address specified in `src` in the `af` address family into a character string.
The result is stored in the buffer pointed to by `dst`, which must be non-null, and the size of this buffer is specified by the `size` argument.

The address families supported are:

`AF_INET`
: `src` points to a struct `in_addr` (in network byte order) which is converted to an IPv4 network address in the dotted-decimal format, *ddd.ddd.ddd.ddd*. The buffer `dst` must be at least `INET_ADDRSTRLEN` bytes long

`AF_INET6`
: `src` points to a struct `in6_addr` (in network byte order) which is converted to a representation of this address in the most appropriate IPv6 network address format for this address. The buffer `dst` must be at least `INET6_ADDRSTRLEN` bytes long

The reverse operation is also possible. We can use the `inet_pton`[40] function to convert addresses from text to binary form.

```
#include <arpa/inet.h>

int inet_pton(int af, const char *restrict src, void *restrict dst);

                                                    Returns: on success 1
        0 if src does not contain a character string with a valid net address
                                  -1 if af does not contain a valid address family
```

This `inet_pton` function converts the character string `src` into a network address structure in the `af` address family, then copies the network address structure to `dst`. The `af` argument must be either `AF_INET` or `AF_INET6`. `dst` is written in network byte order.

39 See the `inet_ntop(3)` manual page
40 See the `inet_pton(3)` manual page

If `af` is `AF_INET`, `src` points to a character string containing an IPv4 network address in dotted-decimal format, *ddd.ddd.ddd.ddd*, The address is converted to a struct `in_addr` and copied to `dst`, which must be `sizeof(struct in_addr)`.

If `af` is `AF_INET6`, `src` points to a character string containing an IPv6 network address. The address is converted to a struct `in6_addr` and copied to `dst`, which must be `sizeof(struct in6_addr)`.

The format for IPv6 address[41] follows these rules:

- The preferred format for an IPv6 address is `x:x:x:x:x:x:x:x`. This form consists of eight hexadecimal numbers, each of which expresses a 16-bit value (i.e., each `x` can be up to 4 hex digits)

- A series of contiguous zero values in the preferred format can be abbreviated to `::`. Only one instance of `::` can occur in an address.
  The loopback address `0:0:0:0:0:0:0:1` can be abbreviated as `::1`
  The wildcard address, consisting of all zeros, can be written as `::`

- For expressing IPv4-mapped IPv6 addresses is used the form `x:x:x:x:x:x:d.d.d.d`, where the six leading `xs` are hexadecimal values that define the six most-significant 16-bit pieces of the address (i.e., 96 bits), and the `ds` express a value in dotted-decimal notation that defines the least significant 32 bits of the address.
  For example: `::FFFF:204.152.189.116`

Other functions are also available to perform these conversions, such as `inet_aton`, `inet_addr`, and `inet_ntoa`, but as they are older and largely superseded by modern alternatives, they are not covered here.

41 For further details about IPv6 address representation see the RFC 4291, https://www.rfc-editor.org/rfc/rfc4291.html

VI.5.6 Setting the socket address: `bind`

Once a socket is created, `bind`[42] is used to associate it with a specific local address (IP and port). This operation is called “assigning a name to a socket”. After binding, any incoming requests sent to that address are delivered to the socket.

```
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

                                Returns: on success 0, on error -1 and errno is set
```

The `bind` function assigns the address specified in `addr` to the socket referred to by `sockfd`.
The `addrlen` argument specifies the size of the address.

For a `SOCK_STREAM` (TCP) socket to accept incoming connections, it must be bound to a local address using `bind`. The exact format and rules for that address depend on the address family (e.g., `AF_INET`, `AF_INET6`, `AF_UNIX`).
The `bind` manual page lists the relevant address structures and refers to other manual pages for each family.
The `addr` argument in `bind` is a pointer to a generic structure (`struct sockaddr *`). It is used so the function can accept different types of address structures (`sockaddr_in`, `sockaddr_in6`, `socakddr_un`) depending on the address family.

42 See the `bind(2)` manual page

VI.5.7 Receiving connections: `listen` and `accept`

Once an address has been assigned to a socket using `bind`, it can wait for incoming connections. To put the socket into a listening (passive) state, we use the `listen`[43] function.

```
#include <sys/socket.h>

int listen(int sockfd, int backlog);

                                Returns: on success 0, on error -1 and errno is set
```

`listen` marks the socket referred to by `sockfd` as a *passive* (listening) *socket*. Such a socket is used to accept incoming connection requests via `accept`.
The `sockfd` argument is a file descriptor that refers to a socket of type `SOCK_STREAM` or `SOCK_SEQPACKET`.

The `backlog` argument defines the maximum length of the queue of pending connections for `sockfd`. If a connection request arrives when the queue is full, the client may receive an error (such as `ECONNREFUSED`), or, if the protocol supports retransmission, the request may be ignored so that a later connection attempt can succeed.

The *pending connections queue* refers to connections that have been completed by the protocol but have not yet been accepted by the server (i.e., they are waiting for `accept`).
For TCP (and similarly for `SOCK_STREAM` *UNIX domain sockets*), some systems internally maintain two queues:

- a half-open queue for connections in progress (e.g., during the handshake)
- a fully established queue for connections waiting to be accepted

The `backlog` parameter mainly limits the number of connections that can be queued in the fully established state waiting for `accept`. The maximum length of the queue for *incomplete sockets* can be set using `/proc/sys/net/ipv4/tcp_max_syn_backlog`. When *syncookies* are enabled there is no logical maximum length and this setting is ignored[44].
If the `backlog` argument is greater than the value in `/proc/sys/net/core/somaxconn`, then it is silently capped to that value. The default in this file is 4096.

The pattern to accept connections is as follows:

1. A socket is created with `socket`
2. The socket is bound to a local address using `bind`, so that other sockets may `connect` to it
3. A willingness to accept incoming connections and a queue limit for incoming connections are specified with `listen`
4. Connections are accepted with `accept`

Once the socket has been set to *passive mode*, connections can be accepted using the `accept`[45] function, which is defined as follows

43 See the `listen(2)` manual page
44 For details see the `tcp(7)` manual page
45 See the `accept(2)` manual page

```c
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *_Nullable restrict addr,
           socklen_t *_Nullable restrict addrlen);

#define _GNU_SOURCE
#include <sys/socket.h>

int accept4(int sockfd, struct sockaddr *_Nullable restrict addr,
            socklen_t *_Nullable restrict addrlen, int flags);

                        Returns: a file descriptor, on error -1 and errno is set
```

The `accept` system call extracts the first connection request on the queue of *pending connections* for the listening socket `sockfd`, creates a new connected socket, and returns a new file descriptor referring to that socket. The newly created socket is not in the listening state and the original socket `sockfd` is unaffected by the call.

The `sockfd` argument is a socket that has been created with `socket`, bound to a local address with `bind`, and is listening for connections after a `listen`.

The `addr` argument is a pointer to a `sockaddr` structure. This structure is filled in with the address of the *peer socket*. The exact format of `addr` is determined by the address family.
If `addr` is `NULL`, nothing is filled in and `addrlen` is not used, and should also be `NULL`.

The `addrlen` argument must be initialized to the size (in bytes) of the structure pointed to by `addr`. On return, it contains the actual size of the *peer address*. If the buffer provided is too small, the returned address is truncated, and `addrlen` will contain a value greater than the one originally supplied.

If there are no *pending connections* in the queue and the socket is not marked as *nonblocking*, `accept` blocks until a connection becomes available.
If the socket is marked as *nonblocking* and no *pending connections* are present, `accept` fails with the error `EAGAIN` or `EWOULDBLOCK`.

To be notified of *incoming connections* on a socket, we can use `select`, `poll`, or `epoll`. A socket becomes readable when a new connection is available to be accepted, at which point we can call `accept` to obtain a socket for that connection. Alternatively, we can configure the socket to deliver a `SIGIO` signal when activity occurs.

The `accept4` function works likea `accept` when the `flags` argumet is `0`. The following values can be bitwise ORed in flags to obtain different behavior:

`SOCK_NONBLOCK`
: Set the `O_NONBLOCK` file status flag on the new file descriptor

`SOCK_CLOEXEC`
: Set the close-on-exec (`FD_CLOEXEC`) flag on the new file descriptor

The new socket returned by `accept` does not inherit *file status flags* such as `O_NONBLOCK` and `O_ASYNC` from the *listening socket*. Therefore, applications should not rely on the inheritance (or noninheritance) of file status flags and should always explicitly set the required flags on the socket returned by `accept`.

Both `listen` and `accept` are used with connection-based socket types (`SOCK_STREAM`, `SOCK_SEQPACKET`). `SOCK_STREAM` is the socket type used by TCP, while `SOCK_DGRAM` is used by UDP. `SOCK_SEQPACKET` combines features of both types: it is connection-oriented, preserves message boundaries, delivers messages in order, and is reliable. It is most commonly used for *local IPC* with *UNIX domain sockets*. When using `SOCK_STREAM`, applications must define their own protocol to delimit messages, for example:

```
[length][data][length][data]
```

In contrast, with `SOCK_SEQPACKET`, each `send` corresponds to one complete message, and message boundaries are preserved by the system.

There may not always be a connection waiting after a `SIGIO` signal is delivered or after `select`, `poll`, or `epoll` report a readability event. This can occur because the connection may have been removed due to an *asynchronous network error* or consumed by another thread before `accept` is called. If this happens, a blocking `accept` call will wait for the next connection to arrive. To ensure that `accept` never blocks, the listening socket (`sockfd`) should be set to *nonblocking mode* using the `O_NONBLOCK` flag.

The `socklen_t` data type describes the length of a socket address. It is an integer type of at least 32 bits.

The `bindsock.c` program is a modified version of the one in the `bind(2)` manual page. The program creates a *UNIX domain stream socket*, binds it to a filesystem path, and marks it as a listening socket using `listen`. It then accepts a client connection with `accept`, which returns a new connected socket descriptor. Communication with the client is performed using `read` and `write` on this new descriptor. Finally, the listening socket is closed and the socket file is removed from the filesystem..

```c
#include <err.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
#include <stdio.h>

#define MY_SOCK_PATH "/somepath"
#define LISTEN_BACKLOG 50

int main(void)
{
    int sfd, cfd;
    socklen_t peer_addr_size;
    struct sockaddr_un my_addr, peer_addr;
```

```
15    sfd = socket(AF_UNIX, SOCK_STREAM, 0);
16    if (sfd == -1)
17          err(EXIT_FAILURE, "socket");

18    memset(&my_addr, 0, sizeof(my_addr));
19    my_addr.sun_family = AF_UNIX;
20    strncpy(my_addr.sun_path, MY_SOCK_PATH, sizeof(my_addr.sun_path) -
            1);

21    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(my_addr)) == -1)
22          err(EXIT_FAILURE, "bind");

23    if (listen(sfd, LISTEN_BACKLOG) == -1)
24          err(EXIT_FAILURE, "listen");

25    peer_addr_size = sizeof(peer_addr);
26    cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
27    if (cfd == -1)
28          err(EXIT_FAILURE, "accept");

29    const char *msg = "Hello from server";
30    if (write(cfd, msg, strlen(msg)) == -1)
31          err(EXIT_FAILURE, "write");

32    char buf[128];
33    ssize_t n = read(cfd, buf, sizeof(buf) - 1);
34    if (n == -1)
35          err(EXIT_FAILURE, "read");
36    buf[n] = '\0';
37    printf("\nFrom client: %s", buf);

38    if (close(sfd) == -1)
39          err(EXIT_FAILURE, "close");
40    if (unlink(MY_SOCK_PATH) == -1)
41          err(EXIT_FAILURE, "unlink");
42 }
```

---

bindsock.c

At lines 8–9, the path for the *UNIX domain socket* and the *backlog* value are defined.
At line 15, the program creates a `SOCK_STREAM` socket in the `AF_UNIX` address family, storing the returned file descriptor in `sfd`.

Next, the `my_addr` structure is initialized to zero using `memset` (line 18), and its address family is set to `AF_UNIX` (line 19).

At line 20, the socket path is copied into `my_addr.sun_path` using `strncpy`.
At line 21, the socket is associated with a local address by calling `bind`, passing the initialized `my_addr` structure.

Once the socket is bound to the local address, the call to `listen` (line 23) marks it as a passive (listening) socket, allowing it to accept incoming connection requests. The `backlog` parameter specifies the maximum number of pending connections.

At line 26, the server accepts an incoming connection using `accept`. This call:

- returns a new file descriptor (`cfd`) for the established connection
- fills `peer_addr` with the client's address
- updates `peer_addr_size` with the actual size of that address

At lines 29–35, the server communicates with the client. `write` sends a message to the client using the connected socket (`cfd`) and `read` receives data from the client into `buf`, which is then printed.

In the next section, we examine the `connect` function and provide a client program that interacts with this server.

VI.5.8 Connecting to sockets: `connect`

The `connect`[46] function initiates a connection on a socket to a specified peer address, allowing the socket to communicate with that peer.

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

                                Returns: on success 0, on error -1 and errno is set
```

The `connect` system call establishes a connection between the socket identified by the file descriptor `sockfd` and the address specified by `addr`.
The `addrlen` argument indicates the size of the address structure and the format of `addr` depends on the address family of the socket.

Since `SOCK_DGRAM` sockets are not connection-oriented, the `addr` argument specifies the default destination for outgoing datagrams and the only source from which datagrams are received.
In contrast, for sockets of type `SOCK_STREAM` and `SOCK_SEQPACKET`, connect attempts to establish a connection with the socket bound to the specified address.

Some protocol families (such as *UNIX domain stream sockets*) allow `connect` to be called only once per socket. Other sockets (such as *datagram sockets* in UNIX or Internet domains) allow multiple calls to `connect` in order to change the associated peer address.
In some cases (e.g., *TCP sockets* and certain *datagram sockets*), the association can be removed by calling `connect` with an address whose `sa_family` is set to `AF_UNSPEC`. After this, the socket may be connected to a different address.

For sockets in the `AF_UNIX` address family, `addr.sun_path` must be null-terminated. If `connect` fails, the state of the socket is unspecified and should not be relied upon.

The `consock.c` program is a client for `bindsock.c`. The program creates a *UNIX domain stream socket*, initializes a `sockaddr_un` structure with the server's address, and connects to the server using `connect`. It then sends a message using `write` and receives a response using `read`. The received data is printed to *standard output*, and the socket is finally closed.

```
1 #include <err.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <sys/un.h>
6 #include <unistd.h>

7 #define MY_SOCK_PATH "/somepath"

8 int main(void)
9 {
10    int sfd;
11    struct sockaddr_un addr;
```

46 See the `connect(2)` manual page

```
12    sfd = socket(AF_UNIX, SOCK_STREAM, 0);
13    if (sfd == -1)
14          err(EXIT_FAILURE, "socket");

15    memset(&addr, 0, sizeof(addr));
16    addr.sun_family = AF_UNIX;
17    strncpy(addr.sun_path, MY_SOCK_PATH, sizeof(addr.sun_path) - 1);

18    if (connect(sfd, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
19          close(sfd);
20          err(EXIT_FAILURE, "connect");
21    }

22    const char *msg = "Hello from client";
23    if (write(sfd, msg, strlen(msg)) == -1)
24          err(EXIT_FAILURE, "write");

25    char buf[100];
26    ssize_t n = read(sfd, buf, sizeof(buf) - 1);
27    if (n == -1)
28          err(EXIT_FAILURE, "read");
29    buf[n] = '\0';
30    write(STDOUT_FILENO, buf, n);
31    close(sfd);
32 }
```

---

consock.c

At line 11, the program declares a `struct sockaddr_un` structure (`addr`), which will be used to specify the server's address.

At line 12, the program creates a `SOCK_STREAM` socket in the `AF_UNIX` address family, storing the resulting file descriptor in `sfd`.

At lines 15–17, the `addr` structure is initialized:

- it is first zeroed using `memset`
- the address family is set to `AF_UNIX`
- the socket path is copied into `addr.sun_path` using `strncpy`

At line 18, the program calls `connect` to initiate a connection to the server. The server is identified by the address stored in `addr`, and the connection is established through the socket descriptor `sfd`.

At lines 22–29, the client communicates with the server. `write` sends a message to the server and `read` receives data from the server into `buf`. The buffer is properly null-terminated using the number of bytes returned by `read` .

At line 30, the received data is written to *standard output* using `write`. Finally, at line 31, the socket is closed using `close`.

Let’s execute the two programs

Shell 1

```
[linux@ipc] ./bindsock
```

Shell 2

```
[linux@ipc] ./consock
Hello from server
```

Shell 1

```
From client: Hello from client
```

As demonstrated by the execution, the two programs are able to communicate through the socket and exchange data.

VI.5.9 Socket information: `getsockname, getpeername, getsockopt`

We can retrieve useful information about a socket using the functions presented in this section. In particular, `getsockname` and `getpeername` allow us to obtain the local and peer addresses associated with a socket, respectively, while `getsockopt` can be used to query various options and parameters related to the socket.

To retrieve the current address to which the socket is bound we can use the `getsokname`[47] function.

```
#include <sys/socket.h>

int getsockname(int sockfd, struct sockaddr *restrict addr, socklen_t
                *restrict addrlen);

                                Returns: on success 0, on error -1 and errno is set
```

The `sockfd` argument is the socket descriptor. The *local address* is returned in the buffer pointed to by `addr`. The `addrlen` argument should be initialized with the size of `addr`.

To retrieve the address of the peer currently connected to the socket we can use the `getpeername`[48] function.

```
#include <sys/socket.h>

int getpeername(int sockfd, struct sockaddr *restrict addr, socklen_t
                *restrict addrlen);

                                Returns: on success 0, on error -1 and errno is set
```

The `sockfd` argument is the socket descriptor. The *peer address* is returned in the buffer pointed to by `addr`. The `addrlen` argument should be initialized with the size of `addr`.

For a *datagram socket*, `getpeername` returns the *peer address* that was set by a prior call to `connect`.

To get and set options on a socket we can use the `getsockopt` and `setsockopt`[49] functions.

```
#include <sys/socket.h>

int getsockopt(int sockfd, int level, int optname, void optval[_Nullable
               restrict *optlen], socklen_t *restrict optlen);

int setsockopt(int sockfd, int level, int optname, const void
               optval[optlen], socklen_t optlen);

                                Return: on success 0, on error -1 and errno is set
```

47 See the `getsockname(2)` manual page
48 See the `getpeername(2)` manual page
49 See the `getsockopt(2)` manual page

The `getsockopt` and `setsockopt` functions respectively retrieve and modify options associated with a socket identified by the file descriptor `sockfd`.
Socket options[50] can exist at different protocol levels, but they are always available at the socket level. When working with socket options, both `level` and `optname` must be specified.

To operate at the socket API level, the level is set to `SOL_SOCKET`. For options at other levels, the appropriate protocol number must be provided. For example, to access TCP-specific options, the level should be set to the protocol number corresponding to TCP[51].

The `optval` and `optlen` arguments are used to access option values.
For `setsockopt`, they specify the value to be set. For `getsockopt`, they indicate a buffer where the option value will be stored. In this case, `optlen` initially contains the size of the buffer and is updated on return to reflect the actual size of the retrieved value. If no value is to be provided or returned, `optval` may be set to `NULL`.
The `optname` and its associated option are passed without interpretation to the relevant protocol layer, which handles them accordingly. Definitions for socket-level options are listed in the `sys/socket.h` header file, while options for other protocols vary and are documented in their respective manual pages.
Most socket-level options use an integer value for `optval`. For `setsockopt`, a nonzero value typically enables an option, while zero disables it.

The following table lists some of the most commonly used options.[52]

`SO_ACCEPTCONN`	Returns whether or not the socket is enabled for listening
`SO_BROADCAST`	Broadcast datagram
`SO_DEBUG`	Enable socket debugging
`SO_ERROR`	Get and clear the pending socket error
`SO_DONTROUTE`	Send packet only to directly connected hosts
`SO_KEEPALIVE`	Send keep-alive messages
`SO_LINGER`	`close` and `shutdown` will not return until all queued messages have been sent or the timeout has been reached
`SO_OOBINLINE`	Out-of-band data is placed into the receive data stream
`SO_RCVBUF`	Size in bytes of the receive buffer
`SO_RCVLOWAT, SO_SNDLOWAT`	Minimum number of bytes to send/receive
`SO_RCVTIMEO, SO_SNDTIMEO`	Receiving/sending timeouts
`SO_REUSEADDR`	Reuse address
`SO_REUSEPORT`	Reuse port
`SO_SNDBUF`	Size in bytes of the send bufer
`SO_TYPE`	Get the socket type

Tab 9. Socket options

50 See the `socket(7)` manual page and the manual pages corresponding to the several protocols
51 See the `getprotoent(3)` manual page
52 See the `socket(7)` manual page

Configuration files

The `/proc/sys/net/core` directory contains several files that provide information about socket parameters as listed in the following table.

`rmem_default`	Dfault setting in bytes of the socket receive buffer
`rmem_max`	Maximum socket receive buffer size in bytes set by `SO_RCVBUF`
`wmem_default`	Default setting in bytes of the socket send buffer
`wmem_max`	Maximum socket send buffer size in bytes set by `SO_SNDBUF`
`message_cost` `message_burst`	Configure the token bucket filter used to load limit warning messages caused by external network events
`netdev_max_backlog`	Maximum number of packets in the global input queue
`optmem_max`	Maximum size of ancillary data and user control data

Tab 10. Socket parameters configuration files

VI.5.10 Data transmission: `send` and `recv`

We have seen that data can be sent through a socket using `write`. Another method is provided by the `send` function, along with its related functions `sendto` and `sendmsg`[53], which are defined as follows.

```
#include <sys/socket.h>

ssize_t send(int sockfd, const void buf[size], size_t size, int flags);

ssize_t sendto(int sockfd, const void buf[size], size_t size, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);

ssize_t sendmsg(int sockfd, const struct msghdr *msg, int flags);

                    Returns: number of bytes sent, on error -1 and errno is set
```

These system calls are used to transmit a message to another socket.
If the `flags` argument of `send` is 0 the call is equivalent to `write`. `send` is used only when the socket is in the *connected state*.

The call

```
send(sockfd, buf, size, flags);
```

is equivalent to

```
sendto(sockfd, buf, size, flags, NULL, 0);
```

The `sockfd` argument is the file descriptor of the *sending socket*.

If `sendto` is used with a connection-oriented socket, such as `SOCK_STREAM` or `SOCK_SEQPACKET`, the `dest_addr` and `addrlen` arguments are ignored. An `EISCONN` error may be returned if these arguments are not `NULL` and `0`. Conversely, if the socket has not been connected, an `ENOTCONN` error is returned.

For connectionless communication, the destination address is specified using `dest_addr`, with `addrlen` indicating its size. In the case of `sendmsg`, the destination address is provided through `msg.msg_name`, and its size is specified by `msg.msg_namelen`.

When using `send` and `sendto`, the message to send is specified in `buf` and its size in `size`.
When using `sendmsg`, the message is pointed to by the array `msg.msg_iov`. In addition, `sendmsg` allows to send control information.
If the message is too long to pass atomically through the underlying protocol, a `EMSGSIZE` error is returned.
When a message exceeds the available space in the socket's `send` buffer, `send` will block until space becomes available, unless the socket is set to *nonblocking mode*. In *nonblocking mode*, the call fails with an `EAGAIN` or `EWOULDBLOCK` error in this situation. The `select` system call can be used to determine when the socket is ready to send more data.

53 See the `send(2)` manual page

The `flags` argument is a bitwise OR of zero or more of the following:[54]

`MSG_CONFIRM`
: Informs the *link layer* that forward progress has occurred, meaning a successful reply has been received from the peer. If this confirmation is not received, the link layer will periodically probe the neighbor again. This flag is used only with `SOCK_DGRAM` and `SOCK_RAW` sockets and is currently implemented only for IPv4 and IPv6.

`MSG_DONTROUTE`
: Instructs the system not to use a *gateway* when sending the packet, limiting delivery to hosts on directly connected networks. This flag is defined only for protocol families that support routing and is not applicable to *packet sockets*

`MSG_DONTWAIT`
: Enables *nonblocking* operation. If the operation would block, `EAGAIN` or `EWOULDBLOCK` is returned. This is a *per-call option*, whereas setting `O_NONBLOCK` with `fcntl` is on the *open file description*, which will affect all threads in the calling process as well as other processes that hold file descriptors referring to the same open file description

`MSG_EOR`
: Marks the end of a record, for protocols that support record boundaries, such as `SOCK_SEQPACKET` sockets

`MSG_MORE`
: Indicates that the caller has additional data to send. When used with *TCP sockets*, it provides the same effect as the `TCP_CORK` socket option[55], but can be applied on a per-call basis. For *UDP sockets*, this flag informs the kernel to accumulate all data sent with the flag set into a single *datagram*, which is transmitted only when a `send` call is made without this flag

`MSG_NOSIGNAL`
: Prevents the generation of a `SIGPIPE` signal if the peer on a *stream-oriented socket* has closed the connection. In this case, the `EPIPE` error is still returned. This behavior is similar to ignoring `SIGPIPE` using `sigaction`, but unlike that approach, `MSG_NOSIGNAL` applies only to a single call, whereas ignoring `SIGPIPE` affects the entire process and all its threads

`MSG_OOB`
: Sends *out-of-band data* on sockets that support this feature, such as `SOCK_STREAM`, provided that the underlying protocol also supports out-of-band data

`MSG_FASTOPEN`
: Enables *TCP Fast Open* by sending data along with the initial SYN packet, effectively combining `connect` and `write` into a single step. For *blocking sockets*, it waits until the data is buffered and the connection is established. For *nonblocking sockets*, it returns immediately with the number of bytes sent

54 The complete list is in the `x86_64-linux-gnu/bits/socket.h` header file on Debian-based systems
55 See the `tcp(7)` manual page

The `msghdr` structure used my `sendmsg` is defined as follows.

```
struct msghdr {
   void *msg_name;         /* Optional address */
   socklen_t msg_namelen; /* Size of address */
   struct iovec *msg_iov; /* Scatter/gather array */
   size_t msg_iovlen;     /* # elements in msg_iov */
   void *msg_control;     /* Ancillary data */
   size_t msg_controllen; /* Ancillary data buffer size */
   int msg_flags;         /* Flags (unused) */
};
```

The `msg_name` field is used with *unconnected sockets* to specify the *destination address* for a *datagram*. It points to a buffer containing the address, and `msg_namelen` must indicate the size of that address. For *connected sockets*, these fields should be set to `NULL` and `0`, respectively.

The `msg_iov` and `msg_iovlen` fields define scatter-gather buffers, similar to `writev`.

We can send *control information* (ancillary data) using the `msg_control` and `msg_controllen` fields. The maximum size of this *control buffer* is limited per socket by the system setting `/proc/sys/net/core/optmem_max`.

To receive a message from a socket we can use the `recv` function along with its related functions `recvfrom` and `recvmsg`[56], which are defined as follows.

```
#include <sys/socket.h>

ssize_t recv(int sockfd, void buf[size], size_t size, int flags);

ssize_t recvfrom(int sockfd, void buf[restrict size], size_t size, int
                 flags, struct sockaddr *_Nullable restrict src_addr,
                 socklen_t *_Nullable restrict addrlen);

ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);

                Return: number of bytes received, on error -1 and errno is set
```

These functions can be used to receive data on both *connectionless* and *connection-oriented* sockets.
`recv` behaves like `read` when `flags` is `0`. In addition, the following call

```
recv(sockfd, buf, size, flags);
```

is equivalent to

```
recvfrom(sockfd, buf, size, flags, NULL, NULL);
```

If a message is too long to fit in the supplied buffer, excess bytes may be discarded.

56 See the `recv(2)` manual page

If no messages are available at the socket, `recv` waits for a message to arrive, unless the socket is *nonblocking*, in which case `-1` is returned and `errno` is set to `EAGAIN` or `EWOULDBLOCK`.
On succces, the call returns any data available rather than waiting for receipt of the full amount requested.
We can use `select`, `poll` and `epoll` to determine when new data arrives on the socket.

The `flags` argument is a bitwise OR of zero or more of the following:

`MSG_CMSG_CLOEXEC`[57]
: Set the *close-on-exec* flag for the file descriptor received via a UNIX domain file descriptor using the `SCM_RIGHTS` operation[58]

`MSG_DONTWAIT`
: Same as described in `send`

`MSG_ERRQUEUE`
: Specifies that *queued errors* should be received from the *socket error queue*. The error is passed in an *ancillary message* with a type dependent on the protocol[59]

`MSG_OOB`
: Requests the reception of out-of-band (urgent) data that is not part of the normal data stream

`MSG_PEEK`
: Reads data without consuming it, so it remains in the queue

`MSG_TRUNC`
: Causes the actual size of a packet or datagram to be returned, even if it exceeds the size of the provided buffer. This behavior applies to several socket types, including *raw, datagram, netlink*, and certain UNIX sockets

`MSG_WAITALL`
: Blocks until all requested data is received, unless interrupted or an error occurs. It does not apply to datagram sockets

The `recvfrom` function stores the received message in the buffer `buf`, with the caller specifying its size through `size`.
If `src_addr` is not `NULL` and the protocol provides the sender's address, that address is written into the buffer pointed to by `src_addr`. In this case, `addrlen` is a value-result parameter: it should initially contain the size of the buffer, and upon return, it is updated to reflect the actual size of the address. If the buffer is too small, the address is truncated, and `addrlen` will contain a value larger than the one originally provided.
If the source address is not needed, both `src_addr` and `addrlen` should be set to `NULL`.

A call to `recv` is equivalent to

```
recvfrom(fd, buf, size, flags, NULL, NULL);
```

The `recvmsg` function uses a `msghdr` structure for the message.

---

57 `recvmsg` only
58 See the `unix(7)` manual page
59 See the `cmsg(3)` manual page

The `msg_control` field of the `msghdr` structure, with size specified by `msg_controllen`, points to a buffer used for *protocol control information* or ancillary data. Before calling `recvmsg`, `msg_controllen` should be set to the size of the available buffer. After a successful call, it is updated to indicate the actual size of the control data received.

The messages are represented by the `cmsghdr` structure, which is defined in the `bits/socket.h` header file[60]. For example, Linux uses this ancillary data mechanism to pass extended errors, IP options, or file descriptors over UNIX domain sockets.

The `msg_flags` field in the `msghdr` structure is set on return of `recvmsg`. The possible values are listed in the `recv(2)` manual page.

`recv` can return 0 in the folowing cases:

- an orderly shutdown performed by a *stream socket* (traditional *end-of-file*)
- the reception of a zero-length datagram
- zero bytes were requested from a *stream socket*.

`recv` differs from read when a zero-size datagram is pending. In this case `read` has no effect (the datagram remains pending), while `recv` consumes the pending datagram.

The following two programs are taken from the `getaddrinfo(3)` manual page. They implement a UDP-based echo server and client, making use of several functions introduced earlier.

The `echoserver.c` program acts as a *UDP echo server*: it listens on a specified port, receives datagrams from clients, prints information about the sender, and then sends the received data back to the sender.

```
1 #include <netdb.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <sys/socket.h>
6 #include <sys/types.h>
7 #include <unistd.h>

8 #define BUF_SIZE 500

9 int main(int argc, char *argv[])
10 {
11    int sfd, s;
12    char buf[BUF_SIZE];
13    ssize_t nread;
14    socklen_t peer_addrlen;
15    struct addrinfo hints;
16    struct addrinfo *result, *rp;
17    struct sockaddr_storage peer_addr;

18    if (argc != 2) {
```

60 `x86_64-linux-gnu/bits/socket.h` on Debian-based systems

```
19          fprintf(stderr, "Usage: %s port\n", argv[0]);
20          exit(EXIT_FAILURE);
21      }

22      memset(&hints, 0, sizeof(hints));
23      hints.ai_family = AF_UNSPEC;     /* Allow IPv4 or IPv6 */
24      hints.ai_socktype = SOCK_DGRAM; /* Datagram socket */
25      hints.ai_flags = AI_PASSIVE;     /* For wildcard IP address */
26      hints.ai_protocol = 0;           /* Any protocol */
27      hints.ai_canonname = NULL;
28      hints.ai_addr = NULL;
29      hints.ai_next = NULL;

30      s = getaddrinfo(NULL, argv[1], &hints, &result);
31      if (s != 0) {
32          fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(s));
33          exit(EXIT_FAILURE);
34      }

35      for (rp = result; rp != NULL; rp = rp->ai_next) {
36          sfd = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
37          if (sfd == -1)
38              continue;

40          if (bind(sfd, rp->ai_addr, rp->ai_addrlen) == 0)
41              break; /* Success */

42          close(sfd);
43      }

44      freeaddrinfo(result);
45      if (rp == NULL) {
46          fprintf(stderr, "Could not bind\n");
47          exit(EXIT_FAILURE);
48      }

49      for (;;) {
50          char host[NI_MAXHOST], service[NI_MAXSERV];

51          peer_addrlen = sizeof(peer_addr);
52          nread = recvfrom(sfd, buf, BUF_SIZE, 0, (struct sockaddr *)
                        &peer_addr, &peer_addrlen);
53          if (nread == -1)
54              continue; /* Ignore failed request */

55          s = getnameinfo((struct sockaddr *) &peer_addr, peer_addrlen,
                host, NI_MAXHOST, service, NI_MAXSERV, NI_NUMERICSERV);
56          if (s == 0)
57              printf("Received %zd bytes from %s:%s\n", nread, host,
                        service);
58          else
```

```
59              fprintf(stderr, "getnameinfo: %s\n", gai_strerror(s));

60          if (sendto(sfd, buf, nread, 0, (struct sockaddr *) &peer_addr,
                peer_addrlen) != nread)
61              fprintf(stderr, "Error sending response\n");
62      }
63  }
```

---

echoserver.c

At lines 22–29, the program initializes the `hints` structure to zero and sets it up to request

- A datagram socket (`SOCK_DGRAM`)
- Support for both IPv4 and IPv6 (`AF_UNSPEC`)
- A passive socket (`AI_PASSIVE`), meaning it will be used for binding to a local address

At line 30, the program calls `getaddrinfo` to obtain a list of suitable local addresses on which the server can bind, based on the specified port (`argv[1]`).

At lines 35–43, the program iterates through the returned address list and attempts to create a socket for each address. Next, it tries to bind it. If either `socket` or `bind` fails, it closes the socket and continues. The loop stops using `break` when a `bind` succeeds. Although multiple addresses are returned by `getaddrinfo`, the program binds to only the first address for which `bind` succeeds, ignoring the rest.

At line 44, `freeaddrinfo` is called to release the memory allocated for the address list, since it is no longer needed.

At lines 45–48, the program checks if no address succeeded in binding (`rp == NULL`). If so, it prints an error and exits.

At lines 49–61, the program enters an infinite loop to handle incoming datagrams. It receives a message from a client using `recvfrom`, prints information (retrieved using `getnameinfo`) about the sender and sends the same message back (echo) using `sendto`.

At line 52, the `recvfrom` call stores the sender's address in `peer_addr`, updates `peer_addrlen` with the actual size of the address and returns the number of bytes received (`nread`). If the call fails (`nread == -1`), the program ignores the error and continues.
At lines 55–58, `getnameinfo` is used to convert the sender's address into a human-readable form (host and service/port). If successful, it prints the number of bytes received and the sender's address, otherwise it prints an error using `gai_strerror`.

At lines 59–60, the `sendto` call sends the received data back to the client (echo) using the same address obtained from `recvfrom`. If the number of bytes sent differs from `nread`, an error is reported.

The `echoclient.c` program implements a *UDP echo client*. It takes as command-line arguments the destination host, port number, and one or more messages to send. The program resolves the server's address, creates a *datagram socket*, and uses `connect` to set the default destination. It

then sends each message to the server and waits for the echoed response, which it reads and displays on the standard output.

```c
#include <netdb.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#define BUF_SIZE 500

int main(int argc, char *argv[])
{
    int sfd, s;
    char buf[BUF_SIZE];
    size_t size;
    ssize_t nread;
    struct addrinfo hints;
    struct addrinfo *result, *rp;

    if (argc < 3) {
        fprintf(stderr, "Usage: %s host port msg...\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_DGRAM;
    hints.ai_flags = 0;
    hints.ai_protocol = 0;
    s = getaddrinfo(argv[1], argv[2], &hints, &result);
    if (s != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(s));
        exit(EXIT_FAILURE);
    }

    for (rp = result; rp != NULL; rp = rp->ai_next) {
        sfd = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
        if (sfd == -1)
            continue;

        if (connect(sfd, rp->ai_addr, rp->ai_addrlen) != -1)
            break; /* Success */

        close(sfd);
    }

    freeaddrinfo(result);
```

```
40      if (rp == NULL) {
41          fprintf(stderr, "Could not connect\n");
42          exit(EXIT_FAILURE);
43      }

44      for (size_t j = 3; j < argc; j++) {
45          size = strlen(argv[j]) + 1; /* +1 for terminating null byte */

46          if (size > BUF_SIZE) {
47              fprintf(stderr, "Ignoring long message in argument
                        %zu\n", j);
48              continue;
49          }

50          if (write(sfd, argv[j], size) != size) {
51              fprintf(stderr, "partial/failed write\n");
52              exit(EXIT_FAILURE);
53          }

54          nread = read(sfd, buf, BUF_SIZE);
55          if (nread == -1) {
56              perror("read");
57              exit(EXIT_FAILURE);
58          }
59          printf("Received %zd bytes: %s\n", nread, buf);
60      }
61      exit(EXIT_SUCCESS);
62  }
```

---

echoclient.c

At lines 21–25, the program initializes the `hints` structure.
At line 26, `getaddrinfo` is called to retrieve a list of possible addresses for the given host (`argv[1]`) and port (`argv[2]`).

At lines 31–38, the program iterates through the returned address list. It attempts to create a socket for each address and then calls `connect` on the socket. If `connect` succeeds, the loop stops, otherwise, the socket is closed and the next address is tried.
For UDP, `connect` does not establish a real connection like TCP, instead, it sets the default destination address and allows the use of `write` and `read` instead of `sendto` and `recvfrom`. In addition, it filters incoming datagrams so only those from the connected peer are received.

At line 39, `freeaddrinfo` is called to release the memory allocated for the address list.
At lines 40–43, the program checks if no address succeeded (`rp == NULL`). If so, it prints an error and exits.
At lines 44–60, the program processes each message provided on the command line (from `argv[3]` onward).
At line 45, the message size is computed:

```
strlen(argv[j]) + 1
```

includes the null terminator, so the entire string is sent.

At lines 46–49, the program checks if the message fits within the buffer size. If not, it skips that message.
At lines 50–53, the message is sent using `write`. Since the socket is *connected*, `write` sends the datagram to the predefined peer. If fewer bytes are written than expected, the program reports an error and exits.
At lines 54–58, the program reads the server's response using `read`. The received datagram is stored in `buf`. If `read` fails, the program prints the error and exits.
At line 59, the program prints the number of bytes received along with the echoed message.

Let's execute the two programs

Shell 1

```
[linux@ipc] ./echoserver 7
```

Shell 2

```
[linux@ipc] ./echoclient 127.0.0.1 7 hello
Received 6 bytes: hello
```

Shell 1

```
Received 6 bytes from localhost:46557
```

Shell 2

```
[linux@ipc] ./echoclient 0.0.0.0 7 Linux is the boss
Received 6 bytes: Linux
Received 3 bytes: is
Received 4 bytes: the
Received 5 bytes: boss
```

Shell 1

```
Received 6 bytes from localhost:46557
Received 6 bytes from localhost:53478
Received 3 bytes from localhost:53478
Received 4 bytes from localhost:53478
Received 5 bytes from localhost:53478
```

In *Shell 2*, the client sends messages to the server and prints the responses it receives. Each response corresponds to a datagram echoed back by the server.
In *Shell 1*, the server prints information about each received datagram, including the number of bytes received and the sender's address (host and port).

In the second example, each word of the sentence is sent as a separate argument, and therefore as a separate datagram. As a result, the server processes and echoes each word individually, which explains why multiple send/receive operations are observed on both the client and server sides.

As shown, the client can be invoked using different addresses such as 127.0.0.1 or 0.0.0.0, although 127.0.0.1 represents the loopback interface, while 0.0.0.0 is a wildcard address typically used for binding rather than as a destination.
However, even though it may be treated as the local host in this case, the use of the wildcard address as a destination should be avoided.

Let's execute again passing an IP address and the hostname.

Shell 2

```
[linux@ipc] ./echoclient 192.168.1.23 7 Socket messaging
Received 7 bytes: Socket
Received 10 bytes: messaging
```

Shell 1

```
...output…
Received 7 bytes from kali:41343
Received 10 bytes from kali:41343
```

By passing the IP address of the local LAN, the client sends datagrams to that specific network interface. On the server side, the address is resolved into a hostname (kali) using `getnameinfo`, which performs a reverse lookup. As a result, the server prints the hostname.

127.0.0.1 → resolves to localhost
192.168.1.23 → resolves to the machine's hostname (kali)

This demonstrates that the peer address has been correctly resolved to its corresponding host name.

Since this is a `SOCK_DGRAM` socket, its state can be inspected using tools such as `netstat`[61] and `lsof`[62].

```
[linux@ipc] sudo netstat -upa
..output…
udp        0      0 0.0.0.0:echo            0.0.0.0:*                           2313978/./echoserve
...output...
```

The output shows that the UDP socket is bound to the port `echo` (port 7) on all available interfaces (`0.0.0.0`), and is associated with the `echoserver` process (PID 2313978). In addition, it is accepting datagrams form any sources (`:*`).

```
[linux@ipc] sudo lsof | grep echoser
..output…
echoserve 2313978              linux   3u     IPv4          16310043       0t0       UDP *:echo
...output...
```

From the `lsof` output we can see the `echoserver` process (PID 2313978) has an open UDP socket (file descriptor 3) bound to port echo (port 7) on all IPv4 interfaces.

61 See the `netstat(8)` manual page
62 See the `lsof(8)` manual page

We can also use `ss`[63]

```
[linux@ipc] ss -a | grep udp
..output…
udp   UNCONN 0      0                                   0.0.0.0:echo            0.0.0.0:*
...output...
```

The `ss` output shows a UDP socket in an *unconnected state*[64], listening on port 7 across all interfaces and accepting datagrams from any source.

63 See the `ss(8)` manual page

64 UDP is a connectionless protocol

VI.5.11 Connection-oriented client-server uptime service

In this section, we implement a client–server application where the client connects to a server and requests the system's uptime information. The following programs are modified versions of those in the APUE.[65]

The `ruptime.c` program implements a TCP client that connects to a server providing uptime information.

```
1 #include <sys/socket.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <netdb.h>
6 #include <err.h>

7 #define BUFLEN 128
8 #define MAXSLEEP 128

9 int connect_retry(int domain, int type, int protocol, const struct
                    sockaddr *addr, socklen_t alen)
10 {
11    int numsec, fd;
12    for (numsec = 1; numsec <= MAXSLEEP; numsec <<= 1) {
13         if ((fd = socket(domain, type, protocol)) < 0)
14              return(-1);
15         if (connect(fd, addr, alen) == 0) {
16              return(fd);
17         }
18         close(fd);
19         if (numsec <= MAXSLEEP/2)
20              sleep(numsec);
21    }
22    return(-1);
23 }

24 void print_uptime(int sockfd)
25 {
26    int n;
27    char buf[BUFLEN];

28    while ((n = recv(sockfd, buf, BUFLEN, 0)) > 0)
29         write(STDOUT_FILENO, buf, n);
30    if (n < 0)
31         err(EXIT_FAILURE, "recv error");
32 }

33 int main(int argc, char *argv[])
34 {
```

65 Section 16.5

```
35    struct addrinfo *ailist, *aip;
36    struct addrinfo hint;
37    int sockfd, lst;

38    if (argc != 2)
39          err(EXIT_FAILURE, "usage: %s hostname", argv[0]);

40    memset(&hint, 0, sizeof(hint));
41    hint.ai_socktype = SOCK_STREAM;

42    if ((lst = getaddrinfo(argv[1], "12345", &hint, &ailist)) != 0)
43          err(EXIT_FAILURE, "getaddrinfo error: %s", gai_strerror(lst));

44    for (aip = ailist; aip != NULL; aip = aip->ai_next) {
45          if ((sockfd = connect_retry(aip->ai_family, SOCK_STREAM, 0,
                aip->ai_addr, aip->ai_addrlen)) < 0) {
46                continue;
47          } else {
48                print_uptime(sockfd);
49                close(sockfd);
50                freeaddrinfo(ailist);
51                exit(0);
52          }
53    }
54    err(EXIT_FAILURE, "can't connect to %s", argv[1]);
55 }
```

---

ruptime.c

The `connect_retry` function attempts to establish a connection using exponential backoff. For each attempt, it creates a new socket and calls `connect`. If the connection fails, the socket is closed and the process sleeps for an exponentially increasing delay before retrying, up to a maximum defined by `MAXSLEEP`.

The `print_uptime` function reads data from the connected socket using `recv` and writes it directly to standard output. It continues until the server closes the connection or an error occurs.

In `main`, the program expects a hostname as a command-line argument. It initializes an `addrinfo` structure to request TCP stream sockets and calls `getaddrinfo` to resolve the hostname into a list of possible network addresses.

The program then iterates over this list, attempting to connect to each address using `connect_retry`. Upon a successful connection, it retrieves and prints the server's response, closes the socket, frees the address list, and exits. If no connection can be established, the program terminates with an error.

The `ruptimed.c` program implements a TCP-based daemon that listens for client connections and provides system uptime information. Upon receiving a connection request, the server retrieves the system uptime and sends it back to the client before closing the connection.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <syslog.h>
#include <sys/stat.h>

#define PORT 12345
#define QLEN 10
#define BUFLEN 128

void daemonize(void) {
   pid_t pid;

   pid = fork();
   if (pid < 0)
        exit(EXIT_FAILURE);
   if (pid > 0)
        exit(EXIT_SUCCESS); // parent exits

   if (setsid() < 0)
        exit(EXIT_FAILURE);

   signal(SIGHUP, SIG_IGN);
   pid = fork();
   if (pid < 0)
        exit(EXIT_FAILURE);
   if (pid > 0)
        exit(EXIT_SUCCESS);

   umask(0);
   chdir("/");
   for (int i = 0; i < 64; i++)
        close(i);

   open("/dev/null", O_RDWR); // stdin
   dup(0); // stdout
   dup(0); // stderr
}

int initserver(void) {
```

```
39    int sockfd;
40    struct sockaddr_in addr;

41    sockfd = socket(AF_INET, SOCK_STREAM, 0);
42    if (sockfd < 0)
43          return -1;

44    int opt = 1;
45    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

46    memset(&addr, 0, sizeof(addr));
47    addr.sin_family = AF_INET;
48    addr.sin_addr.s_addr = htonl(INADDR_ANY);
49    addr.sin_port = htons(PORT);

50    if (bind(sockfd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
51          goto err;

52    if (listen(sockfd, QLEN) < 0)
53          goto err;

54    return sockfd;

55 err:
56    close(sockfd);
57    return -1;
58 }

59 void get_uptime(char *buf, size_t len) {
60    FILE *fp = fopen("/proc/uptime", "r");
61    if (!fp) {
62          snprintf(buf, len, "error: %s\n", strerror(errno));
63          return;
64    }

65    double uptime;
66    if (fscanf(fp, "%lf", &uptime) != 1) {
67          snprintf(buf, len, "error reading uptime\n");
68          fclose(fp);
69          return;
70    }
71    fclose(fp);

72    int days = uptime / 86400;
73    int hours = ((int)uptime % 86400) / 3600;
74    int mins = ((int)uptime % 3600) / 60;

75    snprintf(buf, len, "uptime: %d days, %02d:%02d\n", days, hours,
              mins);
76 }
```

```c
void serve(int sockfd) {
    int clfd;
    char buf[BUFLEN];

    for (;;) {
        clfd = accept(sockfd, NULL, NULL);
        if (clfd < 0) {
            syslog(LOG_ERR, "accept error: %s", strerror(errno));
            continue;
        }

        get_uptime(buf, sizeof(buf));
        if (send(clfd, buf, strlen(buf), 0) < 0) {
            syslog(LOG_ERR, "send error: %s", strerror(errno));
        }
        close(clfd);
    }
}

int main(void) {
    int sockfd;

    openlog("ruptimed", LOG_PID, LOG_DAEMON);
    daemonize();
    sockfd = initserver();
    if (sockfd < 0) {
        syslog(LOG_ERR, "initserver failed: %s", strerror(errno));
        exit(EXIT_FAILURE);
    }

    syslog(LOG_INFO, "server started on port %d", PORT);
    serve(sockfd);
    close(sockfd);
    closelog();
    return 0;
}
```

---

ruptimed.c

The `daemonize` function daemonizes the process. In the function, a double `fork` is used to ensure the daemon detaches from the terminal and can never acquire a controlling terminal again.
After the first `fork` the parent exits and the child continues. Now the child is guaranteed not to be a process group leader.
The `setsid` call creates a new session, makes the process the session leader, and detaches it from the controlling terminal.
After the second `fork`, the parent (which is the session leader) exits, and the child continues. The resulting process is no longer a session leader, therefore it cannot acquire a controlling terminal.

Although modern applications often use service managers such as `systemd` to handle daemonization, this pattern remains the canonical UNIX method and ensures that the program is fully independent of any terminal.
Once the process has completely detached from the terminal, we proceed with environment cleanup.
The `umask(0)` call clears the file mode creation mask inherited from the parent process. This allows the daemon to explicitly control the permissions of any files it creates.
Next, the process changes its current working directory to `/`. This is done because this directory always exists and prevents the daemon from keeping any directory in use, which could otherwise block filesystem unmounting.
The `for` loop closes all open file descriptors, including standard input, output, and error (0, 1, 2). This ensures that no descriptors inherited from the parent process remain open, avoiding unintended resource usage or security issues.
Once all descriptors are closed, `/dev/null` is opened and duplicated. Since all descriptors were previously closed, the `open` call returns descriptor `0` (stdin), and the subsequent `dup` calls return `1` and `2`. As a result, `stdin`, `stdout`, and `stderr` are all redirected to `/dev/null`.
All these steps are unnecessary if the process is managed by `systemd`, but they are required for a standalone daemon implementation.

An alternative method is as follows:

```
int fd = open("/dev/null", O_RDWR);
dup2(fd, STDIN_FILENO);
dup2(fd, STDOUT_FILENO);
dup2(fd, STDERR_FILENO);
if (fd > 2)
      close(fd);
```

In this case, it is not strictly necessary to close all descriptors beforehand, since `dup2` explicitly assigns the desired file descriptors.

At line 24, the disposition of `SIGHUP` is set to ignore. This prevents the daemon from being terminated if the controlling terminal (if any) is closed.

The `initserver` function creates and binds the server socket.
After creating the socket using `socket`, it calls `setsockopt` to set the option `SO_REUSEADDR`. This allows the socket to be bound to an address that is in the `TIME_WAIT` state, avoiding "*Address already in use*" errors when restarting the server.
Next, it initializes the `addr` structure. The source address is set to:

```
48    addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

This means the server will accept connections on all available network interfaces (e.g., localhost, Ethernet, Wi-Fi). If we use `INADDR_LOOPBACK` at the place of `INADDR_ANY`, the server will accept connections only on the loopback interface.
The call to `htonl` (host-to-network long) converts the address from host byte order to network byte order, ensuring portability across different architectures. Similarly, the port number is converted using `htons` (host-to-network short), which ensures that the port is interpreted correctly regardless of endianness.

The function then binds the socket to the specified address using `bind`, and places it in a passive listening state using `listen`. If no error occurs, the socket descriptor is returned; otherwise, the socket is closed and `-1` is returned.

The `get_uptime` function retrieves the system uptime from the `/proc/uptime` file.
It opens the file and reads the first value, which represents the total uptime in seconds. The value is then converted into days, hours, and minutes. Finally, the formatted result is stored in the buffer using `snprintf`. If an error occurs while opening or reading the file, an appropriate error message is written into the buffer.

The `serve` function handles client requests. The infinite loop repeatedly accepts incoming connections using `accept`. If an error occurs, it logs the error using `syslog` and continues to the next iteration. For each accepted connection, it calls `get_uptime` to retrieve the uptime information, then sends the result to the client using `send`. After serving the client, the connection is closed. The server handles one client at a time (no concurrency).

The `main` function initializes and starts the daemon. It first opens the system log using `openlog`, then daemonizes the process and initializes the server socket by calling `initserver`.
If initialization fails, an error is logged and the program terminates. Otherwise, a startup message is written to the system log, and the `serve` function is called to handle incoming client requests.
Finally, the socket and logging facility are closed before the program exits (although in practice, `serve` runs indefinitely).

Let’s execute

```
[linux@ipc] ./ruptimed

[linux@ipc] tail -f /var/log/syslog
...output…
2026-04-08T16:57:04.926412+07:00 kali ruptimed[2491997]: server started on port 12345

[linux@ipc] ./ruptime  127.0.0.1
uptime: 12 days, 05:25

[linux@ipc] ./ruptime 192.168.1.23
uptime: 12 days, 05:25
```

As the execution shows, the daemon initializes successfully and listens on port `12345`, while the client establishes a connection and correctly receives the uptime information from the server.

Alternative connection-oriented server

A common technique is to redirect the standard output and standard error of the server process to the socket connected to the client. By doing so, any data written to `stdout` or `stderr` is automatically sent to the client through the socket.

We can replace the `serve` function as follows

```
1 void serve(int sockfd) {
2     int clfd;

3     for (;;) {
4           clfd = accept(sockfd, NULL, NULL);
5           if (clfd < 0) {
6                 syslog(LOG_ERR, "accept error: %s", strerror(errno));
7                 continue;
8           }

9           pid_t pid = fork();
10          if (pid < 0) {
11                syslog(LOG_ERR, "fork error: %s", strerror(errno));
12                close(clfd);
13                continue;
14          }

15          if (pid == 0) { // child
16                close(sockfd);
17                dup2(clfd, STDOUT_FILENO);
18                dup2(clfd, STDERR_FILENO);
19                close(clfd);

                  /* Option 1: use printf */
                  // char buf[128];
                  // get_uptime(buf, sizeof(buf));
                  // printf("%s", buf);

                  /* Option 2 */
20                execl("/usr/bin/uptime", "uptime", (char *)NULL);
21                perror("execl");
22                exit(1);
23          } else { // parent
24                close(clfd);
25          }
26    }
27 }
```

At line 9 a new process is created using `fork`. This allows the server to handle multiple clients concurrently: the parent process continues accepting new connections, while the child process handles the current client.
If the `fork` fails, an error is logged and the client socket is closed, since it cannot be handled.

In the child process (line 15), the listening socket `sockfd` is closed at line 16, because the child does not accept new connections and therefore does not need this descriptor. This is important to avoid descriptor leaks and unintended sharing of the listening socket.

At lines 17–18, the standard output (`stdout`) and standard error (`stderr`) are redirected to the client socket using `dup2`. After these calls, any data written to `stdout` or `stderr` will be sent directly to the client through the socket.

At line 19, the original client socket descriptor is closed. This is safe because its functionality has been duplicated onto file descriptors 1 and 2.
At this point, the child process is effectively configured so that all output operations target the connected client.
Two alternative approaches can be used:

- Call `get_uptime` and use `printf` to send the formatted uptime string. Since `stdout` is redirected, the output is automatically transmitted to the client

- Execute an external program using `execl`, as shown at line 20. The `execl` call replaces the current process image with the `uptime` program. Because file descriptors are preserved across `exec`, the output of the `uptime` command is sent directly to the client

If `execl` fails, an error message is printed using `perror`, which is also redirected to the client, and the process exits.
In the parent process (line 23), the client socket is closed at line 24, since the parent does not handle the communication.

```
[linux@ipc] ./ruptimed

[linux@ipc] ./ruptime  127.0.0.1
 18:19:59 up 12 days,  5:00,  1 user,  load average: 1.64, 1.08, 0.88
```

The output of the `uptime` command execution is sent to the client that prints it.

Although the program works correctly, it is important to prevent the creation of zombie processes. This can be achieved by handling terminated child processes, for example by setting the disposition of `SIGCHLD` to `SIG_IGN`

```
signal(SIGCHLD, SIG_IGN);
```

Alternatively, the parent process could explicitly call `wait` or `waitpid` to reap terminated child processes.

By duplicating the standard output and standard error file descriptors and redirecting them to the socket, data can be transmitted to the client without explicitly calling `send`.

VI.5.12 Non-blocking sockets and asynchronous I/O

The client–server model presented so far uses *blocking* and *synchronous sockets*, meaning that system calls such as `accept`, `recv`, and `send` block until they complete. Additionally, it only serves one request at a time. To achieve *asynchronous*, *non-blocking* behavior, sockets must be set to *non-blocking mode*, and I/O multiplexing mechanisms such as `select`, `poll`, or `epoll` must be used to monitor multiple events efficiently.

Both `recv` and `send`, when used on a *non-blocking socket*, will return immediately instead of blocking. If no data is available for reading, or if the socket's output buffer cannot accept more data, these calls fail with `-1` and set `errno` to `EWOULDBLOCK` or `EAGAIN`.
By avoiding blocking operations, the server can continue processing other sockets instead of waiting on a single connection. This enables the handling of multiple client requests concurrently within a single process. Modern high-performance servers rely on this non-blocking, event-driven model to efficiently manage large numbers of simultaneous connections. In this section, a simple example of non-blocking sockets is presented to demonstrate asynchronous handling of multiple connections.

The `asyncserv.c` program implements a *TCP server* using *non-blocking sockets* and the `epoll` I/O multiplexing facility. The listening socket is configured with `O_NONBLOCK`, ensuring that system calls such as `accept`, `recv`, and `send` do not block the execution flow. An `epoll` instance is created via `epoll_create1`, and file descriptors (the *listening socket* and *accepted client sockets*) are registered with the kernel using `epoll_ctl`.
The server enters an event loop driven by `epoll_wait`, which returns a set of file descriptors that are ready for I/O operations. When the *listening socket* becomes readable, the server accepts pending connections; when a *client socket* is ready, it performs *non-blocking* read or write operations.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <sys/epoll.h>

#define PORT 8080
#define MAX_EVENTS 10
#define BUFFER_SIZE 1024

int set_nonblocking(int fd) {
    return fcntl(fd, F_SETFL, fcntl(fd, F_GETFL, 0) | O_NONBLOCK);
}

int main() {
    int server_fd, epoll_fd;
    struct sockaddr_in address;
    struct epoll_event ev, events[MAX_EVENTS];
    char buffer[BUFFER_SIZE];
```

```
19    server_fd = socket(AF_INET, SOCK_STREAM, 0);
20    address.sin_family = AF_INET;
21    address.sin_addr.s_addr = INADDR_ANY;
22    address.sin_port = htons(PORT);

23    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
24    listen(server_fd, 10);
25    set_nonblocking(server_fd);

26    epoll_fd = epoll_create1(0);
27    ev.events = EPOLLIN;
28    ev.data.fd = server_fd;
29    epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &ev);

30    printf("Epoll server listening on port %d...\n", PORT);
31    while (1) {
32          int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
33          for (int i = 0; i < nfds; i++) {
34            if (events[i].data.fd == server_fd) {
35             int client_fd;
36             while ((client_fd = accept(server_fd, NULL, NULL)) != -1) {
37                     set_nonblocking(client_fd);
38                     ev.events = EPOLLIN;
39                     ev.data.fd = client_fd;
40                     epoll_ctl(epoll_fd, EPOLL_CTL_ADD, client_fd, &ev);
41                     printf("New client: %d\n", client_fd);
42             }
43               if (errno != EAGAIN && errno != EWOULDBLOCK) {
44                perror("accept");
45             }
46            } else {
47               int client_fd = events[i].data.fd;
48               int count = recv(client_fd, buffer,BUFFER_SIZE, 0);
49               if (count <= 0) {
50                if (count <0 &&(errno==EAGAIN || errno==EWOULDBLOCK)) {
51                     continue;
52                }
53                epoll_ctl(epoll_fd, EPOLL_CTL_DEL, client_fd, NULL);
54                close(client_fd);
55                printf("Client disconnected: %d\n", client_fd);
56               } else {
57                     buffer[count] = '\0';
58                     printf("Client %d: %s\n", client_fd, buffer);
59                     send(client_fd, buffer, count, 0);
60               }
61             }
62          }
63    }
64    close(server_fd);
65    return 0;
66 }
```

asyncserv.c

At line 19 the program creates a TCP socket using `socket`, and at lines 20–22 initializes the fields of the `sockaddr_in` structure, specifying the IPv4 family, binding to all available network interfaces (`INADDR_ANY`), and setting the port number in network byte order.

At line 23, the program binds the socket to the specified address using `bind`, and at line 24 places the socket into the listening state using `listen`.

At line 25, the function `set_nonblocking` is called. It uses `fcntl` with `O_NONBLOCK` to set the socket file descriptor to *non-blocking mode*. This ensures that subsequent system calls such as `accept`, `recv`, and `send` will return immediately instead of blocking.

Next, at line 26, the program creates an *epoll instance* using `epoll_create1`.
At line 27, the `ev` structure is configured to monitor *input events* (`EPOLLIN`).
At line 28, the *listening socket file descriptor* is assigned to the `ev` structure, and at line 29 it is added to the *epoll interest list* using `epoll_ctl`.

An infinite `while` loop is entered at line 31. The program waits for I/O events using `epoll_wait`. This call blocks until one or more registered file descriptors become ready, returning up to `MAX_EVENTS` ready descriptors.

A `for` loop at line 33, iterates over the set of ready descriptors returned by `epoll_wait`.

If an event occurs on the *listening socket descriptor*, this indicates that one or more incoming connection requests are pending. The program repeatedly calls `accept` in a loop at line 36 to handle all queued connections. Since the socket is *non-blocking*, multiple connections may be available. For each accepted connection, a new client socket is created, set to non-blocking mode at line 37, configured for input events at lines 38–39, and added to the *epoll interest list* at line 40 using `epoll_ctl`.
In this way we can monitor future events on this socket. If `accept` fails for reasons other than no pending connections, an error is reported.

If an event occurs on a *client socket descriptor*, this indicates that the client socket is ready for reading. The program reads the data using `recv` at line 48. Since the socket is non-blocking, this call may return immediately even if no data is available. If `recv` returns a negative value due to `EAGAIN` or `EWOULDBLOCK`, this indicates that no data is currently available and the event can be ignored (`continue`). If `recv` returns `0` or another negative value, the connection is considered closed or erroneous, and the socket is removed from the *epoll interest list* at line 53 using the `EPOLL_CTL_DEL` operation of `epoll_ctl` and closed at line 54.

If data is successfully received, a null terminator is appended to the buffer, and the message is printed. Finally, the program sends the received data back to the client using `send`.[66]

66 This implementation assumes that the entire message is sent in a single call, although in practice partial sends may occur and should be handled

The `tcp_client.c` program implements a simple blocking TCP client that connects to a specified server, sends a message, and prints the server's response.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char ip_str[INET_ADDRSTRLEN];

    if (argc < 4) {
        printf("Usage: %s <IP> <PORT> <MESSAGE>\n", argv[0]);
        return -1;
    }
    char *ip = argv[1];
    int port = atoi(argv[2]);
    char *message = argv[3];

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket creation error");
        return -1;
    }

    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(port);

    if (inet_pton(AF_INET, ip, &serv_addr.sin_addr) <= 0) {
        perror("invalid address");
        return -1;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr,
        sizeof(serv_addr)) <0) {
        perror("connection failed");
        return -1;
    }

    printf("send data to %s\n", inet_ntop(AF_INET, &serv_addr.sin_addr,
            ip_str, sizeof(ip_str)));
    send(sock, message, strlen(message), 0);
    printf("Sent: %s\n", message);
```

```
38    int n = recv(sock, buffer, BUFFER_SIZE - 1, 0);
39    if (n > 0) {
40          buffer[n] = '\0';
41          printf("Server: %s\n", buffer);
42    }
43    close(sock);
44    return 0;
45 }
```

---

tcp_client.c

At lines 12–18, the program processes the command-line arguments. It expects three parameters: the server IP address, the port number, and the message to be sent.

At line 19, a TCP socket is created using `socket`.

At line 24, the `serv_addr` structure is initialized to zero using `memset`.
At lines 25–26, the address structure is configured by setting the protocol family to IPv4 (`AF_INET`) and converting the port number to network byte order using `htons`.

At line 27, the IP address provided on the command line is converted from text to binary form using `inet_pton`. This initializes the `sin_addr` field of the structure. If the conversion fails, the program reports an invalid address and exits.

At line 31, the client attempts to establish a connection to the server using `connect`. This call is blocking and completes when the TCP three-way handshake succeeds or fails.

At line 35, the program converts the binary network address back into a human-readable string using `inet_ntop`, storing the result in `ip_str`. This is used only for display purposes, confirming the destination address.

At line 36, the client sends the message to the server using `send`.[67]
At line 38, the program waits for a response from the server using `recv`. This is a blocking call that returns when data is available or the connection is closed. If data is successfully received, a null terminator is appended to the buffer, and the message is printed.
Finally, at line 43, the socket is closed using `close`.

Let’s execute

Shell 1

```
[linux@ipc] ./asyncserv
Epoll server listening on port 8080...
```

67 This call attempts to transmit the entire buffer, although in general it may send fewer bytes than requested and should be checked

Shell 2

```
[linux@ipc] ./tcp_client 127.0.0.1 8080 "Hello Linux sockets"
end data to 127.0.0.1
Sent: Hello Linux sockets
Server: Hello Linux sockets
```

Shell 1

```
Epoll server listening on port 8080...
New client: 5
Client 5: Hello Linux sockets
Client disconnected: 5
```

The server has received the message and processed the request.

Let’s send some data using `nc` (without stopping the server)

Shell 3

```
[linux@ipc] nc 127.0.0.1 8080
Hello from nc [enter]
Hello from nc
```

Shell 1

```
Epoll server listening on port 8080...
New client: 5
Client 5: Hello Linux sockets
Client disconnected: 5
New client: 5
Client 5: Hello from nc
```

Shell 3

```
Hello from nc [enter]
Hello from nc
New data from client [enter]
New data from client
```

Shell 1

```
Epoll server listening on port 8080...
New client: 5
Client 5: Hello Linux sockets
Client disconnected: 5
New client: 5
Client 5: Hello from nc

Client 5: New data from client
```

As we see from the output, the server processes each request.

Since the server is designed to handle multiple connections concurrently using a non-blocking, event-driven model (based on `epoll`), it is capable of serving several client requests without blocking on any single connection. Therefore, multiple client instances can be executed in parallel to test this behavior.
In practice, this can be achieved by launching several client processes simultaneously, for example by opening multiple terminal windows or running clients in the background. Each client establishes an independent TCP connection to the server, and the server monitors all active sockets through the `epoll` instance.

VI.5.13 Out-of-band data: `sockatmark`

Out-of-band (OOB) data refers to high-priority information that certain communication protocols deliver ahead of normal data. The Transmission Control Protocol (TCP) supports OOB data, while the User Datagram Protocol (UDP) does not. In TCP, this data is considered "*urgent*," and only one byte can be marked as urgent, although it may still be transmitted separately from the main data stream.
To send urgent data, the `MSG_OOB` flag is used with the `send`* functions. If multiple bytes are sent with this flag, only the final byte is treated as urgent.
When urgent data arrives, a `SIGURG` signal is generated. To receive this signal, the process ID (PID) that should handle it must be specified, which can be done using the `fcntl` function with the `F_SETOWN` operation.
The *urgent mark* indicates the position in the data stream where *urgent data* is located. A socket can receive urgent data inline with the normal data stream if the `SO_OOBINLINE` option is enabled.

To determine whether a socket is at the out-of-band mark (urgent mark) we can use the `sockatmark`[68] function, which is defined as follows.

```
#include <sys/socket.h>

int sockatmark(int sockfd);

        Returns: on success 1 if at urgent mark, 0 if not, on error -1 and errno
```

The `sockatmark`[69] function checks whether the socket associated with the file descriptor `sockfd` is currently positioned at the out-of-band (OOB) mark. It returns `1` if the socket is at the mark, and `0` if it is not. Calling this function does not remove or alter the OOB mark.

If `sockatmark` returns `1`, the out-of-band data can be read using the `MSG_OOB` flag with `recv`. Additionally, `sockatmark` is safe to call within a handler for the `SIGURG` signal.

If out-of-band (OOB) data is present in a socket's read queue, the `select` function will indicate this by marking the file descriptor as having an exceptional condition.
Urgent data can be handled in two ways: it can either be received inline with normal data (if configured), or it can be read separately using the `MSG_OOB` flag with a `recv` function, allowing it to be processed before any other queued data.

To check for the mark and read the data with `recv` a typical pattern is as follows[70]:

```
for (;;) {
      atmark = sockatmark(sockfd);
      if (atmark == -1)
            ...
      if (atmark)
            break;

      s = read(sockfd, buf, BUF_LEN);
      ...
```

68 See the `sockatmark(3)` manual page
69 This function is implemented using the `SIOCATMARK ioctl` operation
70 The `sockatmark(3)` manual page contains an example

```
if (atmark == 1) {
      if (recv(sockfd, &oobdata, 1, MSG_OOB) == -1) {
      …
      }
}
```

VI.5.14 Sending and receiving multiple messages: `sendmmsg` and `recvmmsg`

We can use a socket to send multiple messages using a single system call. The `sendmmsg`[71] function is an extension of `sendmsg` that allows this.

```
#define _GNU_SOURCE
#include <sys/socket.h>

int sendmmsg(int sockfd, struct mmsghdr msgvec[n], unsigned int n, int
             flags);

  Returns: on success the number of messages sent, on error -1 and errno is set
```

The `sockfd` argument is the file descriptor of the socket on which data is to be transmitted.
The `msgvec` argument is a pointer to an array of `mmsghdr` structures.
The `n` argument is the size of the array.

The `mmsghdr` structure is defined in `sys/socket.h` as follows.

```
struct mmsghdr {
   struct msghdr msg_hdr;  /* Message header */
   unsigned int msg_len;   /* Number of bytes transmitted */
};
```

The `msg_len` field is used to return the number of bytes sent from the message in `msg_hdr`, which is the same value returned by a single `sendmsg` call.

The `flags` argument is the same as that in `sendmsg`.

A blocking `sendmmsg` call blocks until `n` messages have been sent. A nonblocking call sends as many messages as possible, up to the limit specified by `n`, and returns immediately.

When `sendmmsg` returns, the `msg_len` fields of successive elements of `msgvec` are updated to contain the number of bytes transmitted from the corresponding `msg_hdr`.
The return value of `sendmmsg`() indicates how many elements in `msgvec` were successfully processed.
On success, the function returns the number of messages actually sent. If this number is smaller than `n`, it means not all messages were transmitted, and the caller can invoke `sendmmsg` again to send the remaining ones.

The value specified in `n` is capped to `UIO_MAXIOV` (1024). This constant represents the maximum number of I/O vectors that can be used in a single scatter/gather I/O operation.

The same concept applies to receiving: multiple messages can be received with a single system call. The `recvmmsg`[72] function provides this capability and is defined as follows.

71 See the `sendmmsg(2)` manual page
72 See the `recvmmsg(2)` manual page

```
#define _GNU_SOURCE
#include <sys/socket.h>

int recvmmsg(int sockfd, struct mmsghdr msgvec[n], unsigned int n, int
             flags, struct timespec *timeout);

     Returns: on success the number of received messages, on error -1 and errno
```

The `recvmmsg` system call extends `recvmsg` by allowing multiple messages to be received from a socket in a single call. Additionally, it provides support for specifying a `timeout` for the receive operation.

The arguments are similar to those of `sendmmsg`, but they are used for receiving messages instead of sending them.

The `flags` argument can have the same values as for `recvmsg` with the following addition:

`MSG_WAITFORONE`
: Turns on `MSG_DONTWAIT` after the first message has been received

The `timeout` argument points to a struct `timespec`, which specifies the maximum time to wait for the receive operation in seconds and nanoseconds. If `timeout` is set to `NULL`, the call will block indefinitely until data is received.

A blocking `recvmmsg` call blocks until `n` messages have been received or until the `timeout` expires. A nonblocking call reads as many messages as are available, up to the limit specified by `n`, and returns immediately.

Both `sendmmsg` and `recvmmsg` rely on the `iovec` mechanism to handle data, allowing multiple buffers to be sent or received efficiently in a single operation. The `sendmmsg(2)` and `recvmmsg(2)` manual pages include two example programs that demonstrate their usage. The two programs are included in the source code provided with the book. The files are `sendm.c` and `recvm.c`.

VI.6 UNIX Domain socket

The UNIX domain socket mechanism[73] is an advanced form of IPC that allows processes running on the same machine to communicate. They allow processes to exchange data and, in addition, support the transfer of open file descriptors and credentials. Unlike Internet domain sockets, which require protocol processing, UNIX domain sockets operate through data copying within the kernel, resulting in higher performance. The `AF_UNIX` socket family is used to communicate between proceses on the same machine. UNIX domain sockets can be *unnamed*, or bound to a filesystem pathname. In addition, Linux supports an *abstract namespace* independent of the filesystem.

UNIX domain sockets can be classified into three types:

Unnamed (socket pair)
: Provide a full-duplex communication channel, similar to a pipe, typically used between related processes

Named (pathname sockets)
: Represented as socket files in the filesystem, allowing unrelated processes to establish communication by referencing the same path

Abstract sockets
: Similar to named sockets but without a filesystem entry; they exist only within the kernel. This is a Linux-specific feature.

Three types of communication are supported:

`SOCK_STREAM`
: Connection oriented, byte stream (no message boubdaries)

`SOCK_DGRAM`
: Connectionless, message oriented (preserves boundaries)

`SOCK_SEQPACKET`
: Connection-oriented, message-oriented (preserving boundaries)

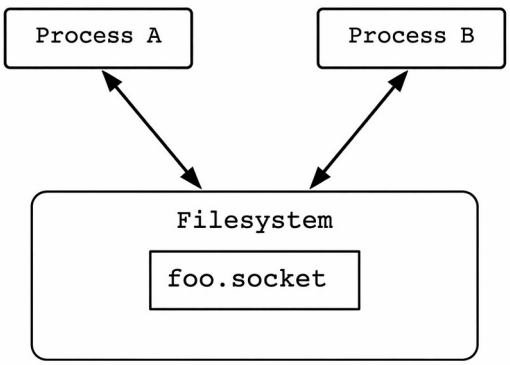

Fig 2. Named socket

73 See the `unix(7)` manual page

VI.6.1 Unnamed UNIX domain sockets: `socketpair`

A socket pair is a full-duplex communication channel composed of two file descriptors, each representing one endpoint of the connection.

The `socketpair`[74] function allows to create a pair of connected sockets and is defined as follows.

```
#include <sys/socket.h>

int socketpair(int domain, int type, int protocol, int sv[2]);

                                Returns: on success 0, on error -1 and errno is set
```

The `socketpair` function creates an *unnamed pair* of *connected sockets* in the specified `domain`, of the specified `type`, and using the optionally specified `protocol`.
The file descriptors are returned in `sv[0]` and `sv[1]` and the two sockets are indistinguishable.
The supported domain are `AF_UNIX` andd `AF_TIPC`.

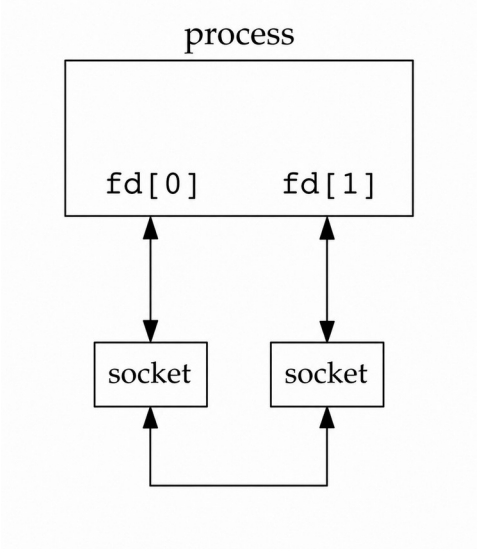

Fig 2. Socket pair

In a *socket pair*, both ends are open for reading and writing, similar to a *full-duplex pipe*.

The `sockpair.c` program demonstrates how to create a *socket pair* and how it works.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/socket.h>

5 #define MSG1 "hello"
6 #define MSG2 "Hi!"

7 int fd_pipe(int fd[2])
```

74 See the `socketpair(2)` manual page

```
8 {
9     return(socketpair(AF_UNIX, SOCK_STREAM, 0, fd));
10 }

11 int main() {
12    int sp[2];
13    char buf[6];

14    if (fd_pipe(sp) == -1) {
15         perror("socketpair");
16         exit(EXIT_FAILURE);
17    }
18    printf("sp[0] = %d\n", sp[0]);
19    printf("sp[1] = %d\n", sp[1]);

      /* Write to sp[0] and read from sp[1] */
20    write(sp[0], MSG1, sizeof(MSG1) -1);
21    read(sp[1], buf, sizeof(MSG1) -1);
22    buf[sizeof(MSG1) -1] = '\0';
23    printf("Received: %s\n", buf);

      /* Write to sp[1] and read from sp[0] */
24    write(sp[1], MSG2, sizeof(MSG2) -1);
25    read(sp[0], buf, sizeof(MSG2) -1);
26    buf[sizeof(MSG2) -1] = '\0';
27    printf("Received: %s\n", buf);

28    close(sp[0]);
29    close(sp[1]);
30    return 0;
31 }
```

---

sockpair.c

The `fd_pipe` function creates a UNIX domain socket pair by invoking `socketpair`. The two resulting file descriptors are stored in the array provided as an argument, and the function returns the result of the system call.

At line 14, the function is called to initialize the socket pair. In case of failure, an error message is printed and the program terminates. The values of the two file descriptors are printed at lines 18–19. Since file descriptors 0, 1, and 2 are typically reserved for standard input, output, and error, the new descriptors are usually assigned the next available values, commonly 3 and 4.

The subsequent code illustrates the behavior of the socket pair. First, data is written to `sp[0]` and read from `sp[1]`. The received data is then null-terminated and printed.
The direction of communication is then reversed: data is written to `sp[1]` and read from `sp[0]`, followed again by null-termination and output.
This exchange demonstrates that both file descriptors can be used for reading and writing, confirming that a socket pair provides a full-duplex communication channel.
Finally, both descriptors are closed, and the program terminates.

Let’s execute the program

```
[linux@ipc] ./sockpair
sp[0] = 3
sp[1] = 4
Received: hello
Received: Hi!
```

As the execution shows, when data is written to one descriptor, it can be read from the other, and the same happens in the opposite direction, confirming that the socket pair supports full-duplex communication.

The sockets created with `socketpair` do not have a name; they are therefore called *unnamed sockets*. Since they are not bound to a filesystem path, they cannot be addressed by unrelated processes, except by passing the associated file descriptors.

VI.6.2 Named UNIX domain sockets: `sockaddr_un`

A *named* UNIX domain socket is represented by an entry in the filesystem, with file type `socket`.

To create *named* UNIX domain sockets, we must use the `sockaddr_un` structure, which is defined as follows in the `sys/un.h` header file.

```
struct sockaddr_un {
   sa_family_t sun_family;              /* AF_UNIX */
   char sun_path[108];                  /* Pathname */
};
```

The `sun_family` field is always `AF_UNIX`, which is the address family used for UNIX domain sockets.

There are three types of address:

pathname
: The socket is bound to null-terminated filesystem pathname using `bind`. The length of the returned address is

  ```
  offsetof(struct sockaddr_un, sun_path) + strlen(sun_path) + 1
  ```
  [75]

  The `sun_path` field contains the pathname.

unnamed
: The socket is not bound to any address, so the `sun_path` field is not used. Since only the address family is specified, the length of the address is set to

  ```
  sizeof(sa_family_t)
  ```

abstract
: Abstract sockets are identified by a name stored in kernel memory rather than a filesystem path

If a socket is not explicitly bound, either because `bind` is invoked with `addrlen` equal to `sizeof(sa_family_t)` or because the `SO_PASSCRED` option is enabled, the system automatically assigns it an *abstract address*. This address begins with a null byte, followed by five bytes selected from the set `[0-9a-f]`, yielding a total of 2^20 possible addresses.

The `namedsock.c` program creates a UNIX domain stream socket and binds it to a pathname.

```
1 #include <sys/socket.h>
2 #include <sys/un.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
```

75 This call returns the same value as `sizeof(sa_family_t)`

```c
int main(void)
{
    int fd, size;
    struct sockaddr_un un;

    fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    un.sun_family = AF_UNIX;
    strncpy(un.sun_path, "foo.socket", sizeof(un.sun_path) - 1);
    un.sun_path[sizeof(un.sun_path) - 1] = '\0';
    unlink("foo.socket");
    size = offsetof(struct sockaddr_un,sun_path) + strlen(un.sun_path);

    if (bind(fd, (struct sockaddr *)&un, size) < 0) {
        perror("bind");
        exit(EXIT_FAILURE);
    }

    printf("UNIX domain socket bound to %s\n", un.sun_path);

  /*
    listen(fd, 5);
    int client_fd = accept(fd, NULL, NULL);
    char buf[100];
    read(client_fd, buf, sizeof(buf));
    printf("Received: %s\n", buf);
    close(client_fd);
  */

    close(fd);
    return 0;
}
```

---

namedsock.c

At line 10, the program creates a UNIX domain stream socket using `socket`. If the call fails, an error message is printed and the program terminates.
At line 15, the address structure is initialized by setting the `sun_family` field to `AF_UNIX`. The pathname of the socket is then copied into the `sun_path` field using `strncpy` at line 16, and explicitly null-terminated at line 17 to ensure it forms a valid string.

At line 18, `unlink` is called to remove any existing file with the same name. This is necessary because `bind` will fail if the specified pathname already exists in the filesystem.
At line 19, the size of the address structure is computed as

```c
    size = offsetof(struct sockaddr_un,sun_path) + strlen(un.sun_path);
```

The `offsetof` macro returns the number of bytes from the beginning of the structure to the `sun_path` field. In other words, it gives the size of the structure up to (but not including) the pathname field. By adding the length of the pathname, we obtain the exact number of bytes that must be passed to `bind`.
This approach is preferred over using `sizeof(struct sockaddr_un)` because the `sun_path` field is variable in length. If `sizeof(struct sockaddr_un)` is used, the full `sun_path` array (108 bytes) is taken into account, even if the actual pathname occupies only a small portion of it, leaving the remaining bytes unused.

At line 20, the socket is bound to the specified pathname using `bind`. The address structure is cast to `struct sockaddr *`, as required by the system call interface.

Finally, at line 24, a confirmation message is printed, and at line 25 the socket is closed before the program terminates.

Let's execute the program and see the file on the filesystem.

```
[linux@ipc] ./socknamed
UNIX domain socket bound to foo.socket

[linux@ipc] ls -l foo.socket
srwxrwxr-x 1 ipc ipc 0 Apr 11 15:31 foo.socket

[linux@ipc] file foo.socket
foo.socket: socket
```

The program has created the file, that is a socket file.

After uncommenting the appropriate line in `socknamed.c`, the `socknamedclient.c` program can be used to send a message to the server.

```
1 #include <sys/socket.h>
2 #include <sys/un.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <unistd.h>

7 #define SOCKET_PATH "foo.socket"
8 #define MSG "Hello from client"

9 int main(void)
10 {
11    int fd;
12    struct sockaddr_un un;
13    int size;

14    fd = socket(AF_UNIX, SOCK_STREAM, 0);
15    if (fd < 0) {
16          perror("socket");
```

```
17          exit(EXIT_FAILURE);
18      }
19      un.sun_family = AF_UNIX;
20      strncpy(un.sun_path, SOCKET_PATH, sizeof(un.sun_path) - 1);
21      un.sun_path[sizeof(un.sun_path) - 1] = '\0';
22      size = offsetof(struct sockaddr_un, sun_path) +
                        strlen(un.sun_path);

23      if (connect(fd, (struct sockaddr *)&un, size) < 0) {
24          perror("connect");
25          exit(EXIT_FAILURE);
26      }
27      if (write(fd, MSG, sizeof(MSG) - 1) < 0) {
28          perror("write");
29          exit(EXIT_FAILURE);
30      }
31      printf("Message sent: %s\n", MSG);
32      close(fd);
33      return 0;
34  }
```

---

socknamedclient.c

Let’s execute

Shell 1

```
[linux@ipc] ./socknamed
UNIX domain socket bound to foo.socket
```

Shell 2

```
[linux@ipc] ./socknamedclient
Message sent: Hello from client
```

Shell 1

```
UNIX domain socket bound to foo.socket
Received: Hello from client
```

The two programs can communicate through the sockets.

The `offsetof`[76] macro is defined as follows

```
#include <stddef.h>

size_t offsetof(type, member);

                                        Returns: offset of member within type
```

The macro returns the offset of the field `member` from the start of the structure `type`.
This macro is used because the sizes of the fields that compose a structure can vary across implementations.

A simple logging systyem

The following two programs, `log_server.c` and `log_client.c` , compose a simple logging system. The server creates a named UNIX domain socket and binds it to a pathname in the filesystem. It then listens for incoming connections.
Each client creates its own socket and connects to the same pathname. Since the socket is identified by a filesystem entry, the processes do not need to be related. When a client sends a message, the server receives it using `read` and prints it. Multiple unrelated processes can connect to the same socket and send log messages.

```
1  #include <sys/socket.h>
2  #include <sys/un.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <unistd.h>

7  #define SOCKET_PATH "log.sock"

8  int main(void)
9  {
10     int sock, client;
11     struct sockaddr_un addr;
12     char buf[128];

13     sock = socket(AF_UNIX, SOCK_STREAM, 0);
14     if (sock < 0) {
15           perror("socket");
16           exit(EXIT_FAILURE);
17     }

18     memset(&addr, 0, sizeof(addr));
19     addr.sun_family = AF_UNIX;
20     strcpy(addr.sun_path, SOCKET_PATH);
21     unlink(SOCKET_PATH);
```

76 See the `offsetof(3)` manual page

```c
    if (bind(sock, (struct sockaddr *)&addr, offsetof(struct
            sockaddr_un, sun_path) + strlen(addr.sun_path)) < 0) {
        perror("bind");
        exit(EXIT_FAILURE);
    }

    if (listen(sock, 5) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }
    printf("Logger server listening...\n");

    while (1) {
        client = accept(sock, NULL, NULL);
        if (client < 0) {
            perror("accept");
            continue;
        }
        int n = read(client, buf, sizeof(buf) - 1);
        if (n > 0) {
            buf[n] = '\0';
            printf("Log: %s\n", buf);
        }
        close(client);
    }
}
```

---

log_server.c

An infinite loop calls `accept` to allow new connections from clients. The server reads the data and prints it.

```c
#include <sys/socket.h>
#include <sys/un.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define SOCKET_PATH "log.sock"

int main(int argc, char *argv[])
{
    int sock;
    struct sockaddr_un addr;
    if (argc < 2) {
        fprintf(stderr, "Usage: %s message\n", argv[0]);
        exit(EXIT_FAILURE);
```

```
15    }

16    sock = socket(AF_UNIX, SOCK_STREAM, 0);
17    if (sock < 0) {
18          perror("socket");
19          exit(EXIT_FAILURE);
20    }

21    memset(&addr, 0, sizeof(addr));
22    addr.sun_family = AF_UNIX;
23    strcpy(addr.sun_path, SOCKET_PATH);

24    if (connect(sock, (struct sockaddr *)&addr, offsetof(struct
          sockaddr_un, sun_path) + strlen(addr.sun_path)) < 0) {
25          perror("connect");
26          exit(EXIT_FAILURE);
27    }
28    write(sock, argv[1], strlen(argv[1]));
29    close(sock);
30    return 0;
31 }
```

---

log_client.c

After initializing the address structure, the client connects to the server using `connect`, and once the connection is established, it transmits data using `write`.

Let's execute

Shell 1

```
[linux@ipc] ./log_server
Logger server listening…
```

Shell 2

```
[linux@ipc] ./log_client "Msg 1"

[linux@ipc] ./log_client "Msg 2"
```

Shell 2

```
Logger server listening…
Log: Msg 1
Log: Msg 2
```

The server processes the requests and prints the data.

Handling multiple connections with `epoll`

To handle multiple connections, the server can use a multithreaded model, where each client is served in a separate thread, allowing concurrent processing of requests. Alternatively, event-driven mechanisms such as `select`, `poll`, or `epoll` can be used to manage multiple clients within a single thread.

The `log_server_epoll.c` program uses `epoll` to handle client requests and avoids blocking operations on any single client. Instead of using a one-thread-per-client model or a blocking `accept`/`read` loop, the server relies on `epoll` to multiplex all connections within a single execution flow. This allows it to efficiently monitor multiple file descriptors and react only when they become ready for I/O.

```
1  #define _GNU_SOURCE
2  #include <sys/socket.h>
3  #include <sys/un.h>
4  #include <sys/epoll.h>
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <string.h>
8  #include <unistd.h>

9  #define SOCKET_PATH "log.sock"
10 #define MAX_EVENTS 10

11 int main(void)
12 {
13    int sock, epfd;
14    struct sockaddr_un addr;

15    sock = socket(AF_UNIX, SOCK_STREAM, 0);
16    if (sock < 0) {
17          perror("socket");
18          exit(EXIT_FAILURE);
19    }
20    memset(&addr, 0, sizeof(addr));
21    addr.sun_family = AF_UNIX;
22    strcpy(addr.sun_path, SOCKET_PATH);
23    unlink(SOCKET_PATH);
24    if (bind(sock, (struct sockaddr *)&addr, offsetof (struct
          sockaddr_un, sun_path) + strlen(addr.sun_path)) < 0) {
25          perror("bind");
26          exit(EXIT_FAILURE);
27    }

28    if (listen(sock, 5) < 0) {
29          perror("listen");
30          exit(EXIT_FAILURE);
31    }
```

```
32    epfd = epoll_create1(0);
33    if (epfd < 0) {
34          perror("epoll_create1");
35          exit(EXIT_FAILURE);
36    }

37    struct epoll_event ev, events[MAX_EVENTS];
38    ev.events = EPOLLIN;
39    ev.data.fd = sock;

40    if (epoll_ctl(epfd, EPOLL_CTL_ADD, sock, &ev) < 0) {
41          perror("epoll_ctl");
42          exit(EXIT_FAILURE);
43    }
44    printf("Logger (epoll) running...\n");

45    while (1) {
46          int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
47          if (n < 0) {
48                perror("epoll_wait");
49                exit(EXIT_FAILURE);
50          }
51          for (int i = 0; i < n; i++) {
52                if (events[i].data.fd == sock) {
53                      int client = accept(sock, NULL, NULL);
54                      if (client < 0) {
55                            perror("accept");
56                            continue;
57                      }
58                      ev.events = EPOLLIN;
59                      ev.data.fd = client;
60                      epoll_ctl(epfd, EPOLL_CTL_ADD, client, &ev);
61                } else {
62                      char buf[128];
63                      int fd = events[i].data.fd;
64                      int nread = read(fd, buf, sizeof(buf) - 1);
65                      if (nread <= 0) {
66                            close(fd);
67                            epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
68                      } else {
69                            buf[nread] = '\0';
70                            printf("Log: %s\n", buf);
71                      }
72                }
73          }
74    }
75 }
```

---

log_server_epoll.c

The program implements a concurrent logger server using UNIX domain sockets combined with the `epoll` event notification mechanism.
A UNIX domain socket is created using `socket`. This socket will act as the listening endpoint for all client connections. The socket is then bound to a filesystem pathname, making it accessible to unrelated processes. The `unlink` call ensures that any previous instance of the socket file is removed before binding, avoiding conflicts.
After binding, the socket is placed into listening mode using `listen`, allowing it to accept incoming connection requests.

The server then creates an `epoll` instance using

```
32    epfd = epoll_create1(0);
```

This creates a kernel-managed event table that will be used to monitor multiple file descriptors.
A listening socket event is then registered with `epoll_ctl`. `EPOLLIN` is specified, meaning the server is interested in read events (incoming connections). The listening socket file descriptor is stored in `ev.data.fd`.
At this point, `epoll` is configured to notify the process whenever a new client attempts to connect.

The server enters an infinite loop where it waits for events using

```
46          int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
```

This call blocks until at least one file descriptor becomes ready. When `epoll_wait` returns, it provides a list of active events that must be processed.

If the event corresponds to the listening socket, this means a new client is connecting. The server accepts the connection using

```
53                      int client = accept(sock, NULL, NULL);
```

The returned client socket represents a new communication channel with that specific process. This new client descriptor is then added to the `epoll` instance, again using `EPOLLIN`, so that the server will be notified when the client sends data.

If the event does not correspond to the listening socket, it must be a client socket with incoming data. The server reads from the socket

```
64                      int nread = read(fd, buf, sizeof(buf) - 1);
```

If `read` returns `0` or a negative value, the client has disconnected or an error occurred. The file descriptor is closed and it is removed from the `epoll` set using `EPOLL_CTL_DEL`. Otherwise, the received data is null-terminated and printed as a log message

To connect we can use the `log_client.c` program. Let's execute

Shell 1

```
[linux@ipc] ./log_server_epoll
Logger (epoll) running…
```

Shell 2

[linux@ipc] ./log_client “Msg from client 1”

Shell 1

Logger (epoll) running…
Log: Msg from client 1

Shell 3

[linux@ipc] ./log_client “Hello from new client ”

Shell 1

Logger (epoll) running…
Log: Msg from client 1
Log: Hello from new client

As the execution shows, the server correctly receives and prints messages from multiple clients connecting at different times. Each client establishes its own connection to the same UNIX domain socket, and the server handles each request through the `epoll` event loop without blocking on any single connection.
Of course, we can run multiple clients simultaneously, since the `epoll`-based design allows the server to monitor and serve all active connections concurrently within a single execution flow.

We can use the `connect.sh` script in a loop to perform concurrent requests.

```
#!/usr/bin/zsh

pid=`echo $$`
./log_client "Msg from $pid"
sleep 2
./log_client "Msg from $pid"
```

---

connect.sh

Shell 1

[linux@ipc] ./log_server_epoll
Logger (epoll) running…

Shell 2

[linux@ipc] for i in `seq 1 10`
do
xterm -e ./connect.sh &
done

```
[2] 3474916
[3] 3474917
[4] 3474918
[5] 3474919
...output...
```

Shell 1

```
Logger (epoll) running…
Log: Msg from 3475024
Log: Msg from 3475036
Log: Msg from 3475026
Log: Msg from 3475032
Log: Msg from 3475030
Log: Msg from 3475039
Log: Msg from 3475050
Log: Msg from 3475049
Log: Msg from 3475056
Log: Msg from 3475069
Log: Msg from 3475024
Log: Msg from 3475036
Log: Msg from 3475026
Log: Msg from 3475032
Log: Msg from 3475030
Log: Msg from 3475039
Log: Msg from 3475050
Log: Msg from 3475049
Log: Msg from 3475056
Log: Msg from 3475069
```

The `for` loop opens an `Xterm` for each value of `i` and runs the `connect.sh` script.

VI.6.3 Datagram sockets: `SOCK_DGRAM`

As we have already seen with network sockets, datagram-oriented UNIX domain sockets provide a connectionless communication model. In this case, data is exchanged as independent messages using `send*` and `recv*`, without requiring the server to perform `listen` or `accept`.
Unlike stream-oriented sockets, datagram sockets do not establish a persistent connection between endpoints. Each message is delivered as a self-contained unit, preserving message boundaries at the kernel level. There is no concept of a continuous byte stream, and ordering or reliability guarantees are not enforced beyond what the local kernel provides for each individual datagram.
`SOCK_STREAM` models a continuous *byte stream* between two endpoints, while `SOCK_DGRAM` models discrete, *independent messages* delivered to a single endpoint without connection semantics. This makes `SOCK_DGRAM` ideal for logging, notifications, and telemetry-style systems where reliability of individual messages is more important than session management.

The `log_server_dgram.c` and `log_client_dgram.c` programs implement a simple logging system based on datagram-oriented UNIX domain sockets.
The server creates a socket, binds it to a pathname, and continuously receives messages from clients using `recvfrom`. Each incoming datagram is handled independently, without establishing any connection or maintaining client state.
The client creates its own socket and sends a log message, provided as a command-line argument, using `sendto`. No connection setup is required, and each message is delivered as a self-contained unit to the server.

```c
#define _GNU_SOURCE
#include <sys/socket.h>
#include <sys/un.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define SOCKET_PATH "log_dgram.sock"

int main(void)
{
    int sock;
    struct sockaddr_un addr;

    sock = socket(AF_UNIX, SOCK_DGRAM, 0);
    if (sock < 0) {
          perror("socket");
          exit(EXIT_FAILURE);
    }

    unlink(SOCKET_PATH);
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCKET_PATH);
```

```
22    if (bind(sock,(struct sockaddr *)&addr,offsetof(struct sockaddr_un,
          sun_path) + strlen(addr.sun_path)) < 0) {
23          perror("bind");
24          exit(EXIT_FAILURE);
25    }
26    printf("DGRAM logger running (stateless)\n");

27    while (1) {
28          char buf[128];
29          struct sockaddr_un client_addr;
30          socklen_t len = sizeof(client_addr);

31          int n = recvfrom(sock, buf, sizeof(buf) - 1, 0,
                             (struct sockaddr *)&client_addr, &len);
32          if (n <= 0)
33                continue;
34          buf[n] = '\0';
35          printf("Log: %s\n", buf);
36    }
37 }
```

---

log_server_dgram.c

The client creates a socket and sends the message provided on the command line using `sendto`, without establishing any prior connection with the server.

```
1 #define _GNU_SOURCE
2 #include <sys/socket.h>
3 #include <sys/un.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <unistd.h>

8 #define SOCKET_PATH "log_dgram.sock"

9 int main(int argc, char *argv[])
10 {
11    int sock;
12    struct sockaddr_un addr;

13    if (argc < 2) {
14          fprintf(stderr, "Usage: %s <message>\n", argv[0]);
15          exit(EXIT_FAILURE);
16    }

17    sock = socket(AF_UNIX, SOCK_DGRAM, 0);
18    if (sock < 0) {
```

```
19          perror("socket");
20          exit(EXIT_FAILURE);
21      }
22      memset(&addr, 0, sizeof(addr));
23      addr.sun_family = AF_UNIX;
24      strcpy(addr.sun_path, SOCKET_PATH);

25      sendto(sock, argv[1], strlen(argv[1]),0, (struct sockaddr *)&addr,
              sizeof(addr));
26      printf("Message sent\n");
27      close(sock);
28      return 0;
29  }
```

---

log_client_dgram.c

Let’s execute

Shell 1

```
[linux@ipc] log_server_dgram
DGRAM logger running (stateless)
```

Shell 2

```
[linux@ipc] log_client_dgram “Hello”
Message sent

[linux@ipc] log_client_dgram “Another message”
Message sent
```

Shell 1

```
DGRAM logger running (stateless)
Log: Hello
Log: Another message
```

The server receives the single messages and prints the data.

VI.6.4 Stream-oriented sockets and message-oriented sockets

As discussed, *stream-oriented sockets* treat data as a continuous sequence of bytes, while *message-oriented sockets* operate on discrete units of data, preserving the boundaries of each individual message.
To better understand and observe this difference in practice, we can analyze the behavior of both `SOCK_STREAM` and `SOCK_DGRAM` using `strace`. By tracing the system calls performed by the client and server, it becomes possible to see how data is actually transmitted and received at the kernel level, highlighting the absence of message boundaries in stream sockets and their preservation in datagram sockets.

We modify the `log_client.c` program adding a second `write` as follows

```
28    write(sock, argv[1], strlen(argv[1]));
      write(sock, "data", 5);
```

We execute `log_server.c` with `strace`

Shell 1

```
[linux@ipc] strace ./log_server
...output…
write(1, "Logger server listening...\n", 27Logger server listening...) = 27
accept(3, NULL, NULL
```

The server is waiting for new connections, we send some data.

Shell 2

```
[linux@ipc] ./log_client “Hello”
```

Shell 1

```
accept(3, NULL, NULL
read(4, "Hellodata\0", 127)          = 10
write(1, "Log: Hellodata\n", 15Log: Hellodata)       = 15
close(4)                            = 0
accept(3, NULL, NULL
```

As we can see multiple `write` (in this case 2) are merged into a single `read`.
In a stream-oriented communication (`SOCK_STREAM`), data is treated as a continuous sequence of bytes without message boundaries. This can be observed when multiple write operations performed by the client are received as a single block by the server. This behavior shows that the kernel may coalesce multiple `write` operations into a single `read`. The receiving process has no information about how the data was originally segmented. This demonstrates that stream sockets do not preserve the boundaries of write operations, but instead provide a continuous flow of bytes to the receiving process.

With `SOCK_DGRAM`, communication is message-oriented. Each call to `sendto` transmits a complete datagram, and message boundaries are preserved by the kernel. As a result, if the client

sends two messages using two separate `sendto` calls, the server will receive them as two distinct units through two corresponding `recvfrom` calls.
This behavior contrasts with stream-oriented sockets, where multiple writes may be merged or split arbitrarily, and no direct correspondence exists between send and receive operations.

For example, the client sends two separate messages:

```
sendto(sock, argv[1], strlen(argv[1]), 0, (struct sockaddr *)&addr,
       sizeof(addr));
sendto(sock, "Data", 4, 0, (struct sockaddr *)&addr, sizeof(addr));
```

On the server side, two consecutive `recvfrom` calls are performed:

```
int n = recvfrom(sock, buf, sizeof(buf) - 1, 0, (struct sockaddr
                 *)&client_addr, &len);
int n2 = recvfrom(sock, buf2, sizeof(buf2) - 1, 0, (struct sockaddr
                  *)&client_addr, &len);
```

Tracing the execution with `strace` shows:

```
recvfrom(3, "Hello", 127, 0, 0x7ffe9435bdf0, [110 => 0]) = 5
recvfrom(3, "Data", 127, 0, 0x7ffe9435bdf0, [0]) = 4
```

Each `recvfrom` call returns exactly one message, matching the corresponding `sendto` operation performed by the client. This clearly demonstrates that datagram sockets preserve message boundaries: data is not merged or split across multiple receive calls, but delivered as discrete units exactly as sent.

This one-to-one correspondence between send and receive operations is a defining characteristic of message-oriented communication.

VI.6.5 Abstract sockets

Abstract UNIX domain sockets provide an alternative addressing mechanism in which socket names are stored entirely within the kernel, avoiding the need for filesystem entries.

An *abstract socket address* is distinguished from a *pathname-based socket* by the fact that the first byte of `sun_path` is a null byte (`'\0'`). The actual name of the socket is stored in the remaining bytes of `sun_path`, up to the length specified for the *address structure*.

Unlike *pathname sockets*, *abstract socket names* are not associated with the filesystem. Furthermore, null bytes within the name have no special meaning and are treated as part of the address.

When an abstract socket address is returned, the value of `addrlen` is greater than `sizeof(sa_family_t)` (typically greater than 2). The socket name is then contained in the first (`addrlen - sizeof(sa_family_t)`) bytes of `sun_path`.

Abstract sockets are not subject to filesystem permission semantics: `umask` does not apply, and modifications to ownership or permissions do not affect accessibility. They are automatically deleted when no references remain. This namespace is a Linux-specific, nonportable extension.

The pattern for assigning an address to an abstract socket is as follows:

```
addr.sun_path[0] = '\0';
strcpy(addr.sun_path + 1, "my_socket");
```

The first byte of `sun_path` is set to `'\0'` to indicate an *abstract socket*, while the actual name is stored starting from the next byte.

All the programs presented so far that use *named sockets* can be rewritten to use *abstract sockets*, requiring only changes to the address initialization.

Abstract sockets are particularly useful for temporary inter-process communication, as they avoid filesystem management operations and are automatically cleaned up by the kernel. Additionally, they are less visible during debugging since they do not appear in the filesystem, making them slightly harder to discover. However, this reduced visibility can also make troubleshooting more difficult.

VI.6.6 Socket options and ancillary messages: `cmsg`

For `AF_UNIX` sockets, the following options can be configured using the `SOL_SOCKET` socket option level:

`SO_PASSCRED`
: Allows a process to receive the credentials of the sending process with each message, delivered as an `SCM_CREDENTIALS` ancillary message. These credentials are either explicitly provided by the sender or, if not specified, automatically include the sender's process ID, real user ID, and real group ID. If this option is enabled on a socket that is not yet connected, the system automatically assigns it a unique name in the abstract namespace

`SO_PASSSEC`
: Enables receiving of the SELinux security label of the peer socket in an ancillary message of type `SCM_SECURITY`

`SO_PEEK_OFF`
: Controls the *peek offset* used by `recv` when the `MSG_PEEK` flag is specified. By default, this option is set to `-1`, which preserves the traditional behavior: `recv` with `MSG_PEEK` reads data from the beginning of the receive queue without removing it.
If the option is set to a value greater than or equal to zero, peeking starts at the specified byte offset within the queued data. After each peek operation, the offset is automatically increased by the number of bytes read, so that subsequent peeks continue from the next position in the queue.
If data is consumed from the queue using `recv` without the `MSG_PEEK` flag, the peek offset is decreased accordingly. This ensures that the offset remains aligned with the current contents of the queue, as if no data had been removed.
For datagram sockets, if the peek offset falls in the middle of a packet, the returned data is marked with the `MSG_TRUNC` flag

`SO_PEERCRED`
: Returns the credentials of the peer process connected to this socket. The returned credentials are those that were in effect at the time of the call to `connect`, `listen`, or `socketpair`. The argument to `getsockoptis` a pointer to a `ucred` structure[77]

```
struct ucred
{
   pid_t pid; /* PID of sending process. */
   uid_t uid; /* UID of sending process. */
   gid_t gid; /* GID of sending process. */
};
```

77 Defined in `sys/socket.h`, on Debian-based system `x86_64-linux-gnu/bits/socket.h`

Ancillary messages

The `sys/socket.h` header file defines the following macros to deal with ancillary data.[78]

```
#include <sys/socket.h>

struct cmsghdr *CMSG_FIRSTHDR(struct msghdr *msgh);

struct cmsghdr *CMSG_NXTHDR(struct msghdr *msgh, struct cmsghdr *cmsg);

size_t CMSG_ALIGN(size_t length);
size_t CMSG_SPACE(size_t length);
size_t CMSG_LEN(size_t length);

unsigned char *CMSG_DATA(struct cmsghdr *cmsg);
```

These macros are used to create and access control messages (also known as ancillary data), which are separate from the normal socket payload. This control information may include details such as the network interface on which a packet was received, optional header fields, extended error information, file descriptors, or UNIX credentials. For example, ancillary data can be used to transmit additional header information, such as IP options.

Ancillary data is transmitted using `sendmsg` and received using `recvmsg`. Structurally, it consists of a sequence of `cmsghdr` structures, each followed by its associated data.

```
struct cmsghdr {
   size_t cmsg_len; /* Data byte count,including header (socklen_t) */
   int cmsg_level;  /* Originating protocol */
   int cmsg_type;   /* Protocol-specific type */
   /* followed by unsigned char cmsg_data[]; */
};
```

The `cmsghdr` structure is defined in the `sys/socket.h` header file[79].

The ancillary data should be accessed only by the macros defined in the `cmsg(3)` manual page.

`CMSG_FIRSTHDR`
: Returns a pointer to the first `cmsghdr` in the ancillary data buffer associated with the passed `msghdr`. It returns `NULL` if there isn't enough space for a `cmsghdr` in the buffer

`CMSG_NXTHDR`
: Returns the next valid `cmsghdr` after the passed `cmsghdr`. It returns `NULL` when there isn't enough space left in the buffer.
When initializing a buffer that will contain a series of `cmsghdr` structures (e.g., to be sent with `sendmsg`), that buffer should first be zero-initialized to ensure the correct operation of `CMSG_NXTHDR`

78 See the `cmsg(3)` manual page
79 On Debian-based system `x86_64-linux-gnu/bits/socket.h`

`CMSG_ALIGN`
: Given a length, returns it including the required alignment

`CMSG_SPACE`
: Returns the number of bytes an ancillary element with payload of the passed data length occupies

`CMSG_DATA`
: Returns a pointer to the data portion of a `cmsghdr`. The pointer returned cannot be assumed to be suitably aligned for accessing arbitrary payload data types. Applications should use `memcpy` to copy data to or from a suitably declared object

`CMSG_LEN`
: Returns the value to store in the `cmsg_len` member of the `cmsghdr` structure, taking into account any necessary alignment. It takes the data length as an argument

To construct ancillary data, we must begin by initializing the `msg_controllen` field of the `msghdr` structure with the size of the control message buffer. The first control message can be obtained using `CMSG_FIRSTHDR`, and any subsequent messages can be accessed using `CMSG_NXTHDR`.
For each control message, we initialize the `cmsg_len` field using `CMSG_LEN`, set the appropriate header fields, and populate the data portion using `CMSG_DATA`.
Finally, we must ensure that the `msg_controllen` field reflects the total space occupied by all control messages in the buffer, calculated as the sum of `CMSG_SPACE` for each message.

The available types of control messages depend on the underlying protocol; in particular, UNIX domain sockets provide their own set of ancillary data types.
Although these message types are specific to `AF_UNIX` sockets, they are specified at the `SOL_SOCKET` level for historical reasons.
When constructing an *ancillary message*, the `cmsg_level` field of the `cmsghdr` structure should be set to `SOL_SOCKET`, while `cmsg_type` identifies the specific type of ancillary data being sent.[80]

`SCM_RIGHTS` – *Passing file descriptor*
: Send or receive a set of open file descriptors from another process. The data portion contains an integer array of the file descriptors.
  This operation is referred to as *passing a file descriptor* to another process. However, more accurately, what is being passed is a reference to an open file description, and in the receiving process it is likely that a different file descriptor number will be used.
  This operation is equivalent to duplicating (`dup`) a file descriptor into the file descriptor table of another process.
  If the buffer used to receive the ancillary data containing file descriptors is too small or absent, then the ancillary data is truncated or discarded and the excess file descriptors are automatically closed in the receiving process.
  If the number of file descriptors received in the ancillary data would cause the process to exceed its `RLIMIT_NOFILE` resource limit, the excess file descriptors are automatically closed in the receiving process.
  The kernel constant `SCM_MAX_FD` (253) defines a limit on the number of file descriptors in the array. Attempting to send an array larger than this limit causes `sendmsg` to fail with an `EINVAL`

80 See the `cmsg(3)` manual page for details

`SCM_CREDENTIALS` – *Passing credentials*

Send or receive UNIX credentials. This can be used for authentication. The credentials are passed as a `struct ucred` ancillary message.

The credentials which the sender specifies are checked by the kernel. A privileged process is allowed to specify values that do not match its own. The sender must specify its own process ID, unless it has the capability `CAP_SYS_ADMIN`, in which case the PID of any existing process may be specified, its real user ID, effective user ID, or saved set-user-ID, unless it has `CAP_SETUID`, and its real group ID, effective group ID, or saved set-group-ID unless it has `CAP_SETGID`.

To receive a `struct ucred` message, the `SO_PASSCRED` option must be enabled on the socket

`SCM_SECURITY` – *Passing SELinux context*

Receive the SELinux security context (the security label) of the peer socket. The received ancillary data is a null-terminated string containing the security context. The receiver should allocate at least `NAME_MAX` bytes in the data portion of the ancillary message for this data.

To receive the security context, the `SO_PASSSEC` option must be enabled on the socket

Only one of the above types may be included in the message sent using `sendmsg`.

When sending *ancillary data*, at least one byte of normal data should also be transmitted. On Linux, this is required when using *UNIX domain stream sockets* in order for the ancillary data to be delivered successfully. For *UNIX domain datagram sockets*, Linux does not strictly require accompanying data. However, for portability, it is recommended to always include at least one byte of regular data.

When receiving data from a *stream socket*, *ancillary data* acts as a boundary within the data stream.

Consider the following sequence of transmissions by the sender:

1. A `sendmsg` call sending four bytes of data, without ancillary data
2. A `sendmsg` call sending one byte of data, along with ancillary data
3. A `sendmsg` call sending four bytes of data, without ancillary data

If the receiver performs `recvmsg` calls with a buffer size of 20 bytes, the first call will return five bytes of data together with the ancillary data associated with the second transmission. The next call will then return the remaining four bytes.

VI.6.7 Passing file descriptors within a single process

Passing a file descriptor is a technique that allows a process, such as a server, to perform all the necessary operations to open a file and then pass a descriptor to a client process, which can use it for subsequent I/O operations.
What is actually transferred is a reference to an *open file description* in the kernel. The receiving process is then assigned a new file descriptor that refers to the same underlying open file description. Typically, after passing the descriptor, the sending process closes its own copy.

A file descriptor can be passed in the following ways:

1. Within the same process, using a UNIX domain socket pair (`socketpair`)
2. Between a parent and child process created with `fork`, using either a socket pair or a named UNIX domain socket
3. Between unrelated processes communicating via named UNIX domain sockets or abstract UNIX domain sockets

To pass a file descriptor the following pattern is used:

- Sender

1. Construct a struct msghdr containing ancillary data
2. Allocate a control buffer sized with `CMSG_SPACE`
3. Obtain a pointer to the first control message using `CMSG_FIRSTHDR`
4. Set `cmsg_level` to `SOL_SOCKET`, `cmsg_type` to `SCM_RIGHTS`, and `cmsg_len` to `CMSG_LEN(sizeof(int))`
5. Store the file descriptor in the payload using `CMSG_DATA`
6. Call sendmsg to transmit the message

- Receiver

1. Construct a struct msghdr
7. Allocate a control buffer sized with `CMSG_SPACE`
8. Call `recvmsg` to receive both data and ancillary data
9. Iterate over control messages using `CMSG_FIRSTHDR` and `CMSG_NXTHDR`
10. Locate the `SCM_RIGHTS` message and extract the file descriptor using `CMSG_DATA`
2. The kernel creates a new file descriptor in the receiving process that refers to the same open file description

The `passfd.c` program demonstrates how to pass a file descriptor through a socket pair.

```c
#define _GNU_SOURCE
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
```

```c
int main(void)
{
    int sp[2];

    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sp) == -1) {
         perror("socketpair");
         exit(EXIT_FAILURE);
    }
    int fd = open("shared.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1) {
         perror("open");
         exit(EXIT_FAILURE);
    }
    write(fd, "Hello via FD passing\n", 22);

/* -------- sender side (sp[0]) -------- */
    struct msghdr msg;
    struct iovec iov;
    char buf = 'X';
    char control[CMSG_SPACE(sizeof(int))];
    struct cmsghdr *cmsg;

    memset(&msg, 0, sizeof(msg));
    memset(control, 0, sizeof(control));
    iov.iov_base = &buf;
    iov.iov_len = sizeof(buf);
    msg.msg_iov = &iov;
    msg.msg_iovlen = 1;
    msg.msg_control = control;
    msg.msg_controllen = sizeof(control);

    cmsg = CMSG_FIRSTHDR(&msg);
    cmsg->cmsg_level = SOL_SOCKET;
    cmsg->cmsg_type = SCM_RIGHTS;
    cmsg->cmsg_len = CMSG_LEN(sizeof(int));
    *(int *)CMSG_DATA(cmsg) = fd;

    if (sendmsg(sp[0], &msg, 0) == -1) {
         perror("sendmsg");
         exit(EXIT_FAILURE);
    }
    printf("Sender: sent file descriptor %d\n", fd);

/* -------- receiver side (sp[1]) -------- */
    char recv_buf;
    char recv_control[CMSG_SPACE(sizeof(int))];
    struct msghdr rmsg;
    struct iovec riov;

    memset(&rmsg, 0, sizeof(rmsg));
    memset(recv_control, 0, sizeof(recv_control));
```

```
50    riov.iov_base = &recv_buf;
51    riov.iov_len = sizeof(recv_buf);
52    rmsg.msg_iov = &riov;
53    rmsg.msg_iovlen = 1;
54    rmsg.msg_control = recv_control;
55    rmsg.msg_controllen = sizeof(recv_control);

56    if (recvmsg(sp[1], &rmsg, 0) == -1) {
57          perror("recvmsg");
58          exit(EXIT_FAILURE);
59    }

60    struct cmsghdr *rcmsg = CMSG_FIRSTHDR(&rmsg);
61    int received_fd = *(int *)CMSG_DATA(rcmsg);
62    printf("Receiver: got file descriptor %d\n", received_fd);
63    write(received_fd, "Written via received FD\n", 25);

64    close(fd);
65    close(received_fd);
66    close(sp[0]);
67    close(sp[1]);
68    return 0;
69 }
```

---

passfd.c

At line 11, a UNIX domain socket pair is created using `socketpair`. This produces two connected endpoints stored in `sp[0]` and `sp[1]`, which will be used for communication between the sender and receiver within the same process.
At line 15, we create the file `shared.txt` using `open`. The file is opened for writing, and at line 20, we write a message into it.
At line 21, a `struct msghdr` is declared. This structure describes the entire message to be sent, including both normal data (payload) and ancillary data (control information).
At line 22, an `iovec` structure is declared. This structure is used to describe the normal data payload.
At line 23, the single character buffer `buf` is defined. This represents the regular data part of the message.

At line 24, we allocate the buffer for ancillary data using:

```
24    char control[CMSG_SPACE(sizeof(int))];
```

The `CMSG_SPACE` macro computes the total space required to store a control message, including necessary alignment and header overhead. This ensures that the buffer is large enough to hold a `cmsghdr` plus an integer file descriptor.

At line 25, a pointer to `cmsghdr` is declared. This structure will represent the *control message header* inside the *ancillary data buffer.*
At lines 26–27, both the `msghdr` structure and the control buffer are zero-initialized to ensure a clean state.

At lines 28–29, the `iovec` structure is initialized: `iov_base` points to `buf` (the normal data) and `iov_len` specifies the size of that data.

At lines 30–31, the `msghdr` structure is configured to use this `iovec`, meaning that the message will include a small payload (`buf`).

At lines 32–33, the *control buffer* is attached to the message:

- `msg_control` points to the *ancillary data buffer*
- `msg_controllen` specifies its total size

At line 34, The `CMSG_FIRSTHDR` macro returns a pointer to the first `cmsghdr` structure within the control buffer

```
34    cmsg = CMSG_FIRSTHDR(&msg);
```

At lines 35–37, the control message header is filled:

```
35    cmsg->cmsg_level = SOL_SOCKET;
```

`cmsg_level` is set to a socket-level control message

```
36    cmsg->cmsg_type = SCM_RIGHTS;
```

`SCM_RIGHTS` specifies that file descriptors are being passed

```
37    cmsg->cmsg_len = CMSG_LEN(sizeof(int));
```

`CMSG_LEN` returns the size of the actual payload (one file descriptor).

At line 38

```
38    *(int *)CMSG_DATA(cmsg) = fd;
```

The file descriptor is placed into the data portion of the *control message*, accessed via `CMSG_DATA`. This is the only place in *user space* where the descriptor value is explicitly stored. During the `sendmsg` call, the kernel interprets this integer not as raw data, but as a reference to an *open file description*, and duplicates that reference into the receiving process.

At line 39 we send the message using `sendmsg` that transmits both the normal data (`'X'` via `iovec`) and the ancillary data (`SCM_RIGHTS` carrying the file descriptor).
At line 43, the sender prints the original file descriptor value.

At lines 44–47, we prepare the variables to receive: normal data (`recv_buf`) , ancillary data (`recv_control`) , message metadata (`rmsg`) and data vector (`riov`) .

At lines 48–49, the structures are zero-initialized and at lines 50–53, the `iovec` is set up to receive the normal payload.
At lines 54–55, the control buffer is attached to `msghdr`, allowing the kernel to store any received ancillary data.

At line 56, we receive the message using `recvmsg`. `msg_iov` will contain the normal data and `msg_control` the ancillary data including the file descriptor.

At line 60

```
60    struct cmsghdr *rcmsg = CMSG_FIRSTHDR(&rmsg);
```

`CMSG_FIRSTHDR` retrieves the *first control message* in the received buffer.

At line 61

```
61    int received_fd = *(int *)CMSG_DATA(rcmsg);
```

The file descriptor is extracted from the data portion of the *control message* using the `CMSG_DATA` macro. This descriptor refers to the same open file description that was originally created at line 15.

At line 62, the program prints the received descriptor and at line 63, the received file descriptor is used to write additional data into the same file, demonstrating that both descriptors refer to the same underlying object.

At lines 64–68, all file descriptors are closed and finally, the program exits.

Let’s execute the program

```
[linux@ipc] ./passfd
Sender: sent file descriptor 5
Receiver: got file descriptor 6

[linux@ipc] cat shared.txt
Hello via FD passing
Written via received FD
```

We can see from the output that the receiving process is assigned a different file descriptor number than the one used by the sender. However, both descriptors refer to the same underlying open file description in the kernel. As a result, operations performed using the received descriptor correctly affect the same file.

VI.6.8 Passing file descriptors between parent and child processes

When a process calls `fork`, the child automatically inherits copies of all the parent's open file descriptors. These descriptors refer to the same underlying open file descriptions in the kernel.
For this reason, in a parent–child relationship, passing a file descriptor explicitly using `SCM_RIGHTS` is not strictly necessary, since the child already has access to the same descriptors.
However, explicit file descriptor passing remains useful in this context for several reasons:

- it allows the parent to decide which descriptors to share, instead of exposing all of them
- when a descriptor is created after the `fork` and must be sent to the child

In practice, file descriptor passing between parent and child processes is less common, since `fork` already provides descriptor inheritance, but it is still used when more controlled or dynamic sharing is required.

The `fd_fork.c` program uses a socket pair to pass a file descriptor created by the parent after the `fork`. To transfer the descriptor, the program performs essentially the same operations as in the previous `fdpass.c` example; however, in this case, each process closes the end of the socket pair that it does not use.

```c
#define _GNU_SOURCE
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

void print_fds(const char *msg)
{
    printf("%s\n", msg);
    for (int fd = 0; fd < 10; fd++) {
        if (fcntl(fd, F_GETFD) != -1) {
            printf(" fd %d is open\n", fd);
        }
    }
}

int main(void)
{
    int sp[2];
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sp) == -1) {
        perror("socketpair");
        exit(EXIT_FAILURE);
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
```

```
28    }

29    if (pid == 0) {
30          close(sp[0]);
31          struct msghdr msg = {0};
32          struct iovec iov;
33          char buf;
34          char control[CMSG_SPACE(sizeof(int))];
35          struct cmsghdr *cmsg;

36          iov.iov_base = &buf;
37          iov.iov_len = sizeof(buf);
38          msg.msg_iov = &iov;
39          msg.msg_iovlen = 1;
40          msg.msg_control = control;
41          msg.msg_controllen = sizeof(control);

42          print_fds("Child: before recvmsg()");
43          if (recvmsg(sp[1], &msg, 0) == -1) {
44                perror("recvmsg");
45                exit(EXIT_FAILURE);
46          }

47          cmsg = CMSG_FIRSTHDR(&msg);
48          int fd = *(int *)CMSG_DATA(cmsg);

49          printf("Child: received FD %d\n", fd);
50          print_fds("Child: after recvmsg()");
51          write(fd, "Written by child\n", 17);
52          close(fd);
53          close(sp[1]);
54          exit(EXIT_SUCCESS);
55    } else {
56          close(sp[1]);
57          int fd = open("after_fork.txt", O_CREAT|O_RDWR|O_TRUNC, 0644);
58          if (fd == -1) {
59                perror("open");
60                exit(EXIT_FAILURE);
61          }
62          write(fd, "Opened after fork\n", 18);

63          struct msghdr msg = {0};
64          struct iovec iov;
65          char buf = 'A';
66          char control[CMSG_SPACE(sizeof(int))];
67          struct cmsghdr *cmsg;

68          iov.iov_base = &buf;
69          iov.iov_len = sizeof(buf);
70          msg.msg_iov = &iov;
71          msg.msg_iovlen = 1;
```

```
72          msg.msg_control = control;
73          msg.msg_controllen = sizeof(control);

74          cmsg = CMSG_FIRSTHDR(&msg);
75          cmsg->cmsg_level = SOL_SOCKET;
76          cmsg->cmsg_type = SCM_RIGHTS;
77          cmsg->cmsg_len = CMSG_LEN(sizeof(int));
78          *(int *)CMSG_DATA(cmsg) = fd;

79          if (sendmsg(sp[0], &msg, 0) == -1) {
80               perror("sendmsg");
81               exit(EXIT_FAILURE);
82          }
83          printf("Parent: sent FD %d\n", fd);
84          close(fd);
85          close(sp[0]);
86     }
87     return 0;
88 }
```

---

fd_fork.c

At line 20 a socket pair is created and at line 24 the process is duplicated using `fork`.

In the child process, the unused end of the socket pair is closed. The child prepares a `msghdr` structure and a control buffer to receive ancillary data.
At line 42 the function `print_fds` is called to display the file descriptors currently open in the child. At this point, the descriptor opened by the parent is not present, since it was created after the `fork`.
At line 43 the child calls `recvmsg` to receive the message from the parent. The file descriptor is extracted using `CMSG_DATA` at line 48 and its value is printed.
At line 50 `print_fds` is called again. This time, a new descriptor appears, showing that it has been received through the socket rather than inherited.
The child then uses the received descriptor to write into the file.

In the parent process, the unused end of the socket pair is closed. At line 57 the file is opened after the `fork`, so the child does not inherit this descriptor.
The parent prepares a control message with `SCM_RIGHTS`, stores the descriptor in the ancillary data using `CMSG_DATA`, and sends it with `sendmsg` at line 79.
Finally, both processes close their descriptors and terminate.

Let’s execute

```
[linux@ipc] ./fd_fork
Parent: sent FD 4
Child: before recvmsg()
  fd 0 is open
  fd 1 is open
  fd 2 is open
  fd 4 is open
```

```
Child: received FD 3
Child: after recvmsg()
  fd 0 is open
  fd 1 is open
  fd 2 is open
  fd 3 is open
  fd 4 is open

[linux@ipc] cat shared.txt
Created by parent
Written by child
```

As the execution shows, before the `recvmsg` call the child does not yet have access to the file descriptor created by the parent. The output lists only the descriptors already open in the child process.
In particular, descriptor `4` corresponds to `sp[1]`, the socket endpoint used by the child for communication, not to the file opened by the parent.
After the parent sends the descriptor, the child receives it and a new file descriptor (`3` in this case) appears in the list printed after `recvmsg`.
The descriptor number differs from the one used by the parent (`4`), confirming that file descriptor values are local to each process. However, both descriptors refer to the same underlying open file description.
This output clearly demonstrates that the descriptor was not inherited through `fork`, but explicitly transferred from the parent to the child using ancillary data.

Now that we have seen that file descriptors created after a `fork` can be passed to a child process, we can use this technique to control the child's input and output by assigning specific descriptors to its standard streams (`stdin`, `stdout`, `stderr`).

The `fd_no_stdout.c` program demonstrates how a process can restore a standard file descriptor that was previously closed by receiving it from another process.
The program creates a socket pair and closes both `STDOUT_FILENO` and `STDERR_FILENO` before calling `fork`. As a result, the child process starts without valid standard output and standard error descriptors.
In the child process, an initial attempt to write to `stdout` fails, confirming that descriptor `1` is not open. The child then prepares a `msghdr` structure and calls `recvmsg` to receive a file descriptor from the parent.
In the parent process, a file is opened and associated with `STDERR_FILENO` using `dup2`. The corresponding descriptor is then sent to the child using a control message of type `SCM_RIGHTS`.
After receiving the descriptor, the child duplicates it onto `STDERR_FILENO`, thereby restoring standard error. Subsequent writes to `stderr` are redirected to the file, while `stdout` remains invalid.

```c
#define _GNU_SOURCE
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
```

```c
#include <string.h>

int main(void)
{
    int sp[2];
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, sp) == -1) {
        perror("socketpair");
        exit(EXIT_FAILURE);
    }

    close(STDOUT_FILENO);
    close(STDERR_FILENO);
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        close(sp[0]);

        write(STDOUT_FILENO, "Child: try write on stdout\n", 26);
        // fails, no data go into restored.txt

        struct msghdr msg = {0};
        struct iovec iov;
        char buf;
        char control[CMSG_SPACE(sizeof(int))];
        struct cmsghdr *cmsg;

        iov.iov_base = &buf;
        iov.iov_len = sizeof(buf);
        msg.msg_iov = &iov;
        msg.msg_iovlen = 1;
        msg.msg_control = control;
        msg.msg_controllen = sizeof(control);

        if (recvmsg(sp[1], &msg, 0) == -1) {
            perror("recvmsg");
            exit(EXIT_FAILURE);
        }

        cmsg = CMSG_FIRSTHDR(&msg);
        int fd = *(int *)CMSG_DATA(cmsg);
        if (fd != STDERR_FILENO) {
            dup2(fd, STDERR_FILENO);
            close(fd);
        }
        write(STDERR_FILENO, "Child: stderr restored via FD
             passing\n", 39);
        close(sp[1]);
```

```
48          exit(EXIT_SUCCESS);
49      } else {
50          close(sp[1]);
51          int fd = open("restored.txt", O_CREAT|O_WRONLY|O_TRUNC, 0644);
52          if (fd == -1) {
53               perror("open");
54               exit(EXIT_FAILURE);
55          }
56          dup2(fd, STDERR_FILENO);

57          struct msghdr msg = {0};
58          struct iovec iov;
59          char buf = 'X';
60          char control[CMSG_SPACE(sizeof(int))];
61          struct cmsghdr *cmsg;

62          iov.iov_base = &buf;
63          iov.iov_len = sizeof(buf);
64          msg.msg_iov = &iov;
65          msg.msg_iovlen = 1;
66          msg.msg_control = control;
67          msg.msg_controllen = sizeof(control);

68          cmsg = CMSG_FIRSTHDR(&msg);
69          cmsg->cmsg_level = SOL_SOCKET;
70          cmsg->cmsg_type = SCM_RIGHTS;
71          cmsg->cmsg_len = CMSG_LEN(sizeof(int));
72          *(int *)CMSG_DATA(cmsg) = fd;

73          if (sendmsg(sp[0], &msg, 0) == -1) {
74               perror("sendmsg");
75               exit(EXIT_FAILURE);
76          }
77          close(fd);
78          close(sp[0]);
79      }
80      return 0;
81  }
```

---

fd_no_stdout.c

At line 11 a socket pair is created, providing a bidirectional communication channel between the parent and the child.

Before calling `fork`, the program closes both `STDOUT_FILENO` and `STDERR_FILENO`. As a consequence, after the `fork`, both processes start without valid descriptors for standard output and standard error.

In the child process, the unused end of the socket pair is closed.

At line 24, the program attempts to write to `STDOUT_FILENO`. This operation fails because descriptor `1` is not open. This demonstrates that standard descriptors are not guaranteed to exist and must be explicitly managed.
The child then prepares a `msghdr` structure along with an `iovec` and a control buffer to receive ancillary data. The control buffer is sized using `CMSG_SPACE(sizeof(int))`, which ensures it is large enough to hold a file descriptor.

At line 36, the child calls `recvmsg` to receive a message from the parent. This call retrieves both the normal data and the ancillary data.

At line 40, the program accesses the first control message using `CMSG_FIRSTHDR`. The file descriptor sent by the parent is extracted from the ancillary data using `CMSG_DATA`.

The child then restores standard error

```
dup2(fd, STDERR_FILENO);
```

This duplicates the received descriptor onto descriptor `2`, effectively reassigning `stderr` to refer to the same open file description as the parent's file.

The check:

```
if (fd != STDERR_FILENO) {
```

ensures that if the received descriptor already has value `2`, the program avoids calling `dup2(2,2)` followed by `close(2)`, which would otherwise close the descriptor.
After this operation, `STDERR_FILENO` is valid again, while `STDOUT_FILENO` remains invalid.
Finally, the child writes to `STDERR_FILENO`. This write succeeds and the data is written to the file associated with the received descriptor.

In the parent process, the unused end of the socket pair is closed.
At line 51, the parent opens the file `restored.txt`. Since descriptors `1` and `2` were previously closed, the `open` call returns the lowest available descriptor, which is typically `1`.

At line 56, the program calls

```
dup2(fd, STDERR_FILENO);
```

This duplicates the file descriptor onto `STDERR_FILENO`, making descriptor `2` refer to the same file. At this point:

- descriptor `1` may already refer to the file (depending on the return value of `open`)
- descriptor `2` is explicitly set to refer to the file

The parent then prepares a control message of type `SCM_RIGHTS`. The file descriptor is stored in the ancillary data using `CMSG_DATA`.

At line 73, the parent sends the message using `sendmsg`. The kernel interprets the ancillary data and transfers a reference to the open file description to the child.
Finally, the parent closes its descriptors.

Let’s execute

```
[linux@ipc] ./fd_no_stdout

[linux@ipc] cat restored.txt
Child: stderr restored via FD passing
```

The absence of output on the terminal and the presence of data in the file confirm that `stderr` was successfully reconstructed in the child process using file descriptor passing.

This example demonstrates that a process can start without standard I/O descriptors and later reconstruct them dynamically using file descriptor passing, independently of inheritance through `fork`.

VI.6.9 Passing file descriptors and credentials between unrelated processes

Sharing a file descriptor among unrelated processes is useful when a process needs to grant another process controlled access to a resource it has already opened.
UNIX domain sockets allow both identification of the peer (via credentials) and transfer of capabilities (via file descriptors), enabling secure and flexible interprocess communication.

The `ipc_server.c` and `ipc_client.c` programs demonstrates a typical UNIX domain IPC pattern: the server authenticates the client using credentials and then shares resources by passing a file descriptor.

```c
#define _GNU_SOURCE
#include <sys/socket.h>
#include <sys/un.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define SOCKET_PATH "ipc_socket"

int main(void)
{
    int sock, client;
    struct sockaddr_un addr;
    socklen_t addrlen;

    sock = socket(AF_UNIX, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("socket");
        exit(EXIT_FAILURE);
    }

    int opt = 1;
    if (setsockopt(sock,SOL_SOCKET,SO_PASSCRED,&opt,sizeof(opt)) < 0) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strcpy(addr.sun_path, SOCKET_PATH);
    unlink(SOCKET_PATH);
    addrlen = offsetof(struct sockaddr_un, sun_path) +
                       strlen(addr.sun_path);

    if (bind(sock, (struct sockaddr *)&addr, addrlen) < 0) {
        perror("bind");
        exit(EXIT_FAILURE);
```

```
33    }
34    if (listen(sock, 1) < 0) {
35          perror("listen");
36          exit(EXIT_FAILURE);
37    }
38    printf("Server: waiting for connection...\n");
39    client = accept(sock, NULL, NULL);
40    if (client < 0) {
41          perror("accept");
42          exit(EXIT_FAILURE);
43    }

44    struct msghdr msg = {0};
45    struct iovec iov;
46    char buf[16];

47    char control[CMSG_SPACE(sizeof(struct ucred))];
48    struct cmsghdr *cmsg;
49    iov.iov_base = buf;
50    iov.iov_len = sizeof(buf);
51    msg.msg_iov = &iov;
52    msg.msg_iovlen = 1;
53    msg.msg_control = control;
54    msg.msg_controllen = sizeof(control);

55    if (recvmsg(client, &msg, 0) < 0) {
56          perror("recvmsg");
57          exit(EXIT_FAILURE);
58    }
59    printf("Server: received message: %s\n", buf);

60    cmsg = CMSG_FIRSTHDR(&msg);
61    if (cmsg && cmsg->cmsg_type == SCM_CREDENTIALS) {
62          struct ucred *cred = (struct ucred *)CMSG_DATA(cmsg);
63          printf("Client PID=%d UID=%d GID=%d\n", cred->pid, cred->uid,
                  cred->gid);
64    }

65    int fd = open("shared.txt", O_CREAT | O_RDWR | O_TRUNC, 0644);
66    write(fd, "Created by server\n", 18);
67    struct msghdr smsg = {0};
68    struct iovec siov;
69    char sbuf = 'F';
70    char scontrol[CMSG_SPACE(sizeof(int))];
71    struct cmsghdr *scmsg;

72    siov.iov_base = &sbuf;
73    siov.iov_len = sizeof(sbuf);
74    smsg.msg_iov = &siov;
75    smsg.msg_iovlen = 1;
76    smsg.msg_control = scontrol;
```

```
77    smsg.msg_controllen = sizeof(scontrol);
78    scmsg = CMSG_FIRSTHDR(&smsg);
79    scmsg->cmsg_level = SOL_SOCKET;
80    scmsg->cmsg_type = SCM_RIGHTS;
81    scmsg->cmsg_len = CMSG_LEN(sizeof(int));
82    *(int *)CMSG_DATA(scmsg) = fd;

83    if (sendmsg(client, &smsg, 0) < 0) {
84         perror("sendmsg");
85         exit(EXIT_FAILURE);
86    }

87    printf("Server: sent file descriptor\n");
88    close(fd);
89    close(client);
90    close(sock);
91    return 0;
92 }
```

---

ipc_server.c

At line 15 a UNIX domain socket is created.

At line 21

```
21    if (setsockopt(sock,SOL_SOCKET,SO_PASSCRED,&opt,sizeof(opt)) < 0) {
```

`setsockopt` is used to enable the reception of credentials by setting the `SO_PASSCRED` option. When this option is enabled, the kernel automatically attaches the credentials (PID, UID, GID) of the sending process to each received message in the form of ancillary data.

After initializing the `address` structure at lines 25–29, the program binds the socket at line 30. The pathname stored in `sun_path` identifies the socket in the filesystem.
At line 28, `unlink` is used to remove any existing socket file with the same name, preventing `bind` from failing.
At line 29, the length of the address is computed using `offsetof` plus the length of the pathname.
At line 34, `listen` sets the socket in passive mode, with a backlog of 1.
At line 39, `accept` is called to accept an incoming connection from a client. A new socket descriptor (`client`) is returned, which will be used for communication.

At lines 44–46, a `msghdr` structure, an `iovec` structure, and a buffer are declared.
The `msghdr` structure describes the message to be received, including both normal data and ancillary data.
At line 47

```
47    char control[CMSG_SPACE(sizeof(struct ucred))];
```

the `CMSG_SPACE` macro is used to allocate a control buffer large enough to store a `cmsghdr` structure and a `struct ucred`, including any required padding and alignment.

At line 48, a pointer to `cmsghdr` is declared. This structure represents a single control message within the ancillary data buffer.
At lines 49–52, the `iovec` structure is initialized to receive the normal data portion of the message.

At lines 53–54

```
53    msg.msg_control = control;
54    msg.msg_controllen = sizeof(control);
```

The control buffer is assigned to `msg_control`, allowing `recvmsg` to store ancillary data (credentials) in it.

At line 55, the server receives a message using `recvmsg`. This call fills the `iov` buffer with normal data and the `control` buffer with ancillary data (credentials). At line 59, the received message is printed.

At line 60

```
60    cmsg = CMSG_FIRSTHDR(&msg);
```

The `CMSG_FIRSTHDR` macro returns a pointer to the first control message in the ancillary data buffer.

At line 61, the program checks whether the control message contains credentials

```
61    if (cmsg && cmsg->cmsg_type == SCM_CREDENTIALS) {
```

If so, at line 62 a `struct ucred` pointer is used to access the data portion of the control message

```
62          struct ucred *cred = (struct ucred *)CMSG_DATA(cmsg);
```

At line 63, the client's credentials (PID, UID, GID) are printed. These values are provided by the kernel and identify the sending process.

At line 65, the program creates a file and at line 66 writes initial data into it. The resulting file descriptor will be passed to the client.
At lines 67–69, a new `msghdr`, an `iovec`, and a buffer are declared to prepare the outgoing message.
At line 70

```
70    char scontrol[CMSG_SPACE(sizeof(int))];
```

a control buffer is allocated to store the file descriptor. The size is computed using `CMSG_SPACE` to ensure proper alignment.

At lines 72–75, the `iovec` structure is initialized to send a small amount of normal data (required when sending ancillary data on stream sockets).
At lines 76–77, the control buffer is attached to the message.

At line 78

```
78    scmsg = CMSG_FIRSTHDR(&smsg);
```

`scmsg` is set to point to the first control message header within the buffer.

At lines 79–81, the control message is initialized with the fields set to indicate it contains a file descriptor.

At line 82

```
82    *(int *)CMSG_DATA(scmsg) = fd;
```

The file descriptor is stored in the data portion of the control message. During `sendmsg`, the kernel interprets this value and transfers a reference to the underlying open file description to the receiving process.

At line 83, the server sends the message using `sendmsg` that transmits both the normal data (via `iovec`) and the ancillary data (file descriptor via `SCM_RIGHTS`) .
At line 87, a confirmation message is printed and at lines 88–90, the file descriptor, client socket, and server socket are closed, and the program terminates.

The `ipc_client.c` program performs a connection request to the serrver.

```
1 #define _GNU_SOURCE
2 #include <sys/socket.h>
3 #include <sys/un.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <unistd.h>

8 #define SOCKET_PATH "ipc_socket"

9 int main(void)
10 {
11    int sock;
12    struct sockaddr_un addr;
13    socklen_t addrlen;

14    sock = socket(AF_UNIX, SOCK_STREAM, 0);
15    if (sock < 0) {
16         perror("socket");
17         exit(EXIT_FAILURE);
18    }

19    memset(&addr, 0, sizeof(addr));
20    addr.sun_family = AF_UNIX;
21    strcpy(addr.sun_path, SOCKET_PATH);
22    addrlen = offsetof(struct sockaddr_un, sun_path) +
                        strlen(addr.sun_path);

23    if (connect(sock, (struct sockaddr *)&addr, addrlen) < 0) {
24         perror("connect");
```

```
25          exit(EXIT_FAILURE);
26      }

27      write(sock, "Hello server", 13);
28      struct msghdr msg = {0};
29      struct iovec iov;
30      char buf;
31      char control[CMSG_SPACE(sizeof(int))];
32      struct cmsghdr *cmsg;

33      iov.iov_base = &buf;
34      iov.iov_len = sizeof(buf);
35      msg.msg_iov = &iov;
36      msg.msg_iovlen = 1;
37      msg.msg_control = control;
38      msg.msg_controllen = sizeof(control);

39      if (recvmsg(sock, &msg, 0) < 0) {
40          perror("recvmsg");
41          exit(EXIT_FAILURE);
42      }

43      cmsg = CMSG_FIRSTHDR(&msg);
44      int fd = *(int *)CMSG_DATA(cmsg);
45      printf("Client: received file descriptor %d\n", fd);
46      write(fd, "Written by client\n", 18);

47      close(fd);
48      close(sock);
49      return 0;
50  }
```

---

ipc_client.c

At line 14 a UNIX domain socket is created.
At lines 19–22 the address structure is initialized by setting the family to `AF_UNIX` and copying the pathname into `sun_path`. The length of the address is computed using `offsetof`.
At line 23 the client connects to the server using `connect`.
At line 27 the client sends a message. Since the server enabled `SO_PASSCRED`, the kernel automatically attaches the client's credentials to this message.
At lines 28–32 the program declares a `msghdr`, an `iovec`, and a control buffer to receive the file descriptor as ancillary data.
At lines 33–38 the `iovec` and `msghdr` structures are initialized, and the control buffer is assigned to `msg_control`.
At line 39 `recvmsg` is used to receive both normal data and ancillary data.
At line 43 `CMSG_FIRSTHDR` retrieves the control message, and at line 44 the file descriptor is extracted using `CMSG_DATA`.
At line 45 the descriptor is printed, and at line 46 it is used to write into the file shared by the server.
At lines 47–48 the descriptors are closed and the program terminates.

Let’s execute

Shell 1

```
[linux@ipc] ./ipc_server
Server: waiting for connection…
```

Shell 2

```
[linux@ipc] ./ipc_client
Client: received file descriptor 4
```

Shell 1

```
Server: waiting for connection…
Server: received message: Hello server
Client PID=3272732 UID=1000 GID=1000
Server: sent file descriptor
```

From the output of execution we see that the client successfully connects to the server and receives a file descriptor.
The server first waits for a connection and, once the client connects, receives the message "Hello server". Along with this message, the kernel provides the client’s credentials (PID, UID, GID), which are printed by the server.
After receiving and inspecting the credentials, the server sends a file descriptor to the client using ancillary data.
On the client side, the descriptor is received and assigned a local value (4 in this execution). This value may differ from the descriptor used by the server, since file descriptor numbers are local to each process.
The successful exchange demonstrates both the automatic transmission of credentials using `SO_PASSCRED` and the passing of a file descriptor using `SCM_RIGHTS`.

We can check the file contents and the socket file.

```
[linux@ipc] cat shared.txt
Created by server
Written by client

[linux@ipc] file socket_ipc
ipc_socket: socket
```

The combination of `SO_PASSCRED` and `SCM_RIGHTS` enables a secure IPC pattern in which a server can authenticate a client and conditionally grant access to resources.
After receiving the credentials, the server can decide whether to send the file descriptor or not, or perform different actions based on the identity of the client.

With credentials, the server can authorize access by allowing only certain users (based on UID), implement security policies by applying different behavior for each client, restrict resource sharing by deciding whether to send the file descriptor or not, and log or audit clients by tracking who is connecting.

This IPC mechanism is widely used in real-world systems such as service managers (e.g., systemd), web servers (e.g., Nginx), and secure communication frameworks (e.g., D-Bus), where processes exchange file descriptors to share access to sockets, files, or other resources.

The same example can be implemented using *abstract sockets* by modifying only the `sun_path` field. We can replace the lines 27-29 of `ipc_server.c` and lines 21-22 of `ipc_client.c` with the following:

```
addr.sun_path[0] = '\0';
strcpy(addr.sun_path + 1, "ipc_socket");
addrlen = offsetof(struct sockaddr_un, sun_path) + 1 +
                   strlen("ipc_socket");
```

Using abstract sockets, no filesystem entry is created and therefore no `unlink` call is needed. The socket address exists entirely within the kernel rather than in the filesystem. In addition, there is no need to consider filesystem permissions, since abstract sockets are not associated with files.

IV.6.10 A secure file access service

In this section, we present a complete example of a *secure file access service*, implemented through the `open_server.c` and `open_client.c` programs, using an *abstract UNIX domain socket*. The server accepts client connections, retrieves and verifies the client's credentials, and, upon successful authorization, opens the requested file and sends the corresponding file descriptor using ancillary data. The client establishes a connection with the server, transmits the filename, and receives a file descriptor via ancillary data. It then uses this descriptor to read and display the file contents.

```
1  #define _GNU_SOURCE
2  #include <sys/socket.h>
3  #include <sys/un.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <unistd.h>
8  #include <fcntl.h>

9  #define SOCKET_NAME "open_socket"

10 int main(void)
11 {
12    int sock, client;
13    struct sockaddr_un addr;

14    sock = socket(AF_UNIX, SOCK_STREAM, 0);
15    if (sock < 0) {
16         perror("socket");
17         exit(EXIT_FAILURE);
18    }

19    int opt = 1;
20    setsockopt(sock, SOL_SOCKET, SO_PASSCRED, &opt, sizeof(opt));

21    memset(&addr, 0, sizeof(addr));
22    addr.sun_family = AF_UNIX;
23    addr.sun_path[0] = '\0';
24    strcpy(addr.sun_path + 1, SOCKET_NAME);

25    socklen_t addrlen = offsetof(struct sockaddr_un, sun_path) + 1 +
                                  strlen(SOCKET_NAME);

26    if (bind(sock, (struct sockaddr *)&addr, addrlen) < 0) {
27         perror("bind");
28         exit(EXIT_FAILURE);
29    }
30    if (listen(sock, 5) < 0) {
31         perror("listen");
```

```
32          exit(EXIT_FAILURE);
33      }

34      printf("Secure open server (abstract socket) running...\n");
35      client = accept(sock, NULL, NULL);
        if (client < 0) {
            perror("accept");
            exit(EXIT_FAILURE);
        }

36      char filename[128];
37      struct iovec iov;
38      struct msghdr msg;
39      char control[CMSG_SPACE(sizeof(struct ucred))];

40      memset(&msg, 0, sizeof(msg));
41      memset(control, 0, sizeof(control));
42      iov.iov_base = filename;
43      iov.iov_len = sizeof(filename) - 1;
44      msg.msg_iov = &iov;
45      msg.msg_iovlen = 1;
46      msg.msg_control = control;
47      msg.msg_controllen = sizeof(control);

48      int n = recvmsg(client, &msg, 0);
49      if (n <= 0) {
50          perror("recvmsg");
51          exit(EXIT_FAILURE);
52      }

53      filename[n] = '\0';
54      struct cmsghdr *cmsg = CMSG_FIRSTHDR(&msg);
55      struct ucred *cred = NULL;
55      if (cmsg && cmsg->cmsg_type == SCM_CREDENTIALS) {
56          cred = (struct ucred *)CMSG_DATA(cmsg);
57          printf("Client UID=%d PID=%d\n", cred->uid, cred->pid);
58      }
59      if (!cred || cred->uid != 1000) {
60          printf("Access denied\n");
61          close(client);
62          close(sock);
63          return 0;
64      }

65      int fd = open(filename, O_RDONLY);
66      if (fd < 0) {
67          perror("open");
68          exit(EXIT_FAILURE);
69      }

70      struct msghdr smsg = {0};
```

```
71    struct iovec siov;
72    char sbuf = 'F';
73    char scontrol[CMSG_SPACE(sizeof(int))];
74    memset(scontrol, 0, sizeof(scontrol));
75    struct cmsghdr *scmsg;

76    siov.iov_base = &sbuf;
77    siov.iov_len = sizeof(sbuf);
78    smsg.msg_iov = &siov;
79    smsg.msg_iovlen = 1;
80    smsg.msg_control = scontrol;
81    smsg.msg_controllen = sizeof(scontrol);
82    scmsg = CMSG_FIRSTHDR(&smsg);
83    scmsg->cmsg_level = SOL_SOCKET;
84    scmsg->cmsg_type = SCM_RIGHTS;
85    scmsg->cmsg_len = CMSG_LEN(sizeof(int));
86    *(int *)CMSG_DATA(scmsg) = fd;

87    if (sendmsg(client, &smsg, 0) < 0) {
88         perror("sendmsg");
89         exit(EXIT_FAILURE);
90    }
91    close(fd);
92    close(client);
93    close(sock);
94    return 0;
95 }
```

---

open_server.c

The the code should be sufficiently clear to the reader; therefore, we avoid providing a detailed analysis here.

```
1 #define _GNU_SOURCE
2 #include <sys/socket.h>
3 #include <sys/un.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <unistd.h>

8 #define SOCKET_NAME "open_socket"

9 int main(int argc, char *argv[])
10 {
11    int sock;
12    struct sockaddr_un addr;
```

```
13    if (argc < 2) {
14          fprintf(stderr, "Usage: %s <file>\n", argv[0]);
15          exit(EXIT_FAILURE);
16    }

17    sock = socket(AF_UNIX, SOCK_STREAM, 0);
18    if (sock < 0) {
19          perror("socket");
20          exit(EXIT_FAILURE);
21    }

22    int opt = 1;
23    setsockopt(sock, SOL_SOCKET, SO_PASSCRED, &opt, sizeof(opt));
24    memset(&addr, 0, sizeof(addr));
25    addr.sun_family = AF_UNIX;
26    addr.sun_path[0] = '\0';
27    strcpy(addr.sun_path + 1, SOCKET_NAME);
28    socklen_t addrlen = offsetof(struct sockaddr_un, sun_path) + 1 +
                                    strlen(SOCKET_NAME);

29    if (connect(sock, (struct sockaddr *)&addr, addrlen) < 0) {
30          perror("connect");
31          exit(EXIT_FAILURE);
32    }

33    write(sock, argv[1], strlen(argv[1]));
34    struct msghdr msg;
35    struct iovec iov;
36    char buf;
37    char control[
38          CMSG_SPACE(sizeof(struct ucred)) +
39          CMSG_SPACE(sizeof(int))
40    ];

41    memset(&msg, 0, sizeof(msg));
42    memset(control, 0, sizeof(control));
43    iov.iov_base = &buf;
44    iov.iov_len = sizeof(buf);
45    msg.msg_iov = &iov;
46    msg.msg_iovlen = 1;
47    msg.msg_control = control;
48    msg.msg_controllen = sizeof(control);

49    if (recvmsg(sock, &msg, 0) < 0) {
50          perror("recvmsg");
51          exit(EXIT_FAILURE);
52    }

53    struct cmsghdr *cmsg;
54    int fd = -1;
55    for (cmsg = CMSG_FIRSTHDR(&msg); cmsg != NULL; cmsg =
```

```
            CMSG_NXTHDR(&msg, cmsg))
56      {
57        if (cmsg->cmsg_level==SOL_SOCKET && cmsg->cmsg_type==SCM_RIGHTS)
58            fd = *(int *)CMSG_DATA(cmsg);
60      }

61      if (fd == -1) {
62            fprintf(stderr, "No valid descriptor received\n");
63            exit(EXIT_FAILURE);
64      }
65      printf("Received FD=%d\n", fd);

66      char buffer[64];
67      int n;
68      while ((n = read(fd, buffer, sizeof(buffer))) > 0) {
69            write(STDOUT_FILENO, buffer, n);
70      }
71      close(fd);
72      close(sock);
73      return 0;
74  }
```

---

open_client.c

At lines 37–40, the `control` buffer is allocated with sufficient space to hold multiple ancillary messages, namely the credentials (`SCM_CREDENTIALS`) automatically attached by the kernel and the file descriptor (`SCM_RIGHTS`) sent by the server. The `CMSG_SPACE` macro ensures proper alignment and padding for each element.

At lines 55–60, the client iterates over all received control messages using `CMSG_FIRSTHDR` and `CMSG_NXTHDR`. Since multiple ancillary messages may be present, this step is necessary to locate the one of interest. When a control message of type `SCM_RIGHTS` is found, the file descriptor is extracted from the payload using `CMSG_DATA`.

The remainder of the code should be sufficiently clear to the reader; therefore, a detailed analysis is omitted.

Let’s execute

Shell 1

```
[linux@ipc] echo “hello from file” > file.txt

[linux@ipc] ./open_server
Secure open server (abstract socket) running…
```

Shell 2

```
[linux@ipc] ./open_client file.txt
Received FD=4
```

```
hello_from_file
```

Shell 1

```
Secure open server (abstract socket) running…
Client UID=1000 PID=89328
```

As the execution shows, the client successfully connects to the server and receives a valid file descriptor. The server, in turn, retrieves the client's credentials through the kernel and prints the associated UID and PID, confirming the identity of the requesting process.
Once the authorization check is satisfied, the server opens the requested file and transfers its file descriptor to the client. The client can then use this descriptor to access the file directly, demonstrating how resource access can be delegated securely between processes using UNIX domain sockets.

The open server presented here follows a simple iterative model, handling one request at a time. This design keeps the implementation straightforward and easy to understand, but it limits the server's ability to handle multiple clients efficiently.
The implementation could be extended to support concurrent request handling using different approaches, such as creating a new process with `fork` for each client, adopting a multithreaded model, or employing an event-driven architecture based on `epoll`, as demonstrated in previous examples.

IV.6.11 Sequenced-packet sockets: `SOCK_SEQPACKET`

The last type we consider in this chapter is `SOCK_SEQPACKET`, a Linux-specific socket type that combines connection-oriented communication with message boundary preservation, making it suitable for applications requiring reliable and structured data exchange.
With this type, each `write` (or `send`) corresponds to a single message, and each `recv` returns exactly one message. If the receive buffer is too small, the message is truncated: excess data is discarded rather than split across multiple reads.

The `seq_server.c` and `seq_client.c` programs are taken from the `unix`(7) manual page and demonstrate the use of sequenced-packet sockets for local interprocess communication. The server program waits for a connection from the client program, while the client sends each of its command-line arguments in separate messages. The server treats the incoming messages as integers and adds them up. Then, client sends the command string `END` and the server sends back a message containing the sum of the client's integers. Finally, the server waits for the next client to connect. To stop the server, we can pass the command-line argument `DOWN` to the client.

The `connection.h` header file defines the socket pathname and the buffer size.

```c
#ifndef CONNECTION_H
#define CONNECTION_H

#define SOCKET_NAME "/tmp/9Lq7BNBnBycd6nxy.socket"
#define BUFFER_SIZE 12

#endif
```

---

connection.h

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/un.h>
#include <unistd.h>
#include "connection.h"

int main(void)
{
    int down_flag = 0;
    int ret;
    int connection_socket;
    int data_socket;
    int result;
    ssize_t r, w;
    struct sockaddr_un name;
```

```
18    char buffer[BUFFER_SIZE];

19    connection_socket = socket(AF_UNIX, SOCK_SEQPACKET, 0);
20    if (connection_socket == -1) {
21          perror("socket");
22          exit(EXIT_FAILURE);
23    }

24    memset(&name, 0, sizeof(name));
25    name.sun_family = AF_UNIX;
26    strncpy(name.sun_path, SOCKET_NAME, sizeof(name.sun_path) - 1);
27    ret = bind(connection_socket, (const struct sockaddr *) &name,
                sizeof(name));
28    if (ret == -1) {
29          perror("bind");
30          exit(EXIT_FAILURE);
31    }

32    ret = listen(connection_socket, 20);
33    if (ret == -1) {
34          perror("listen");
35          exit(EXIT_FAILURE);
36    }

37    for (;;) {
38          data_socket = accept(connection_socket, NULL, NULL);
39          if (data_socket == -1) {
40                perror("accept");
41                exit(EXIT_FAILURE);
42          }

43          result = 0;
44          for (;;) {
45                r = read(data_socket, buffer, sizeof(buffer));
46                if (r == -1) {
47                      perror("read");
48                      exit(EXIT_FAILURE);
49                }
50                buffer[sizeof(buffer) - 1] = 0;

51                if (!strncmp(buffer, "DOWN", sizeof(buffer))) {
52                      down_flag = 1;
53                      continue;
54                }
55                if (!strncmp(buffer, "END", sizeof(buffer)))
56                      break;
57                if (down_flag)
58                      continue;
59                result += atoi(buffer);
60          }
61          sprintf(buffer, "%d", result);
```

```
62          w = write(data_socket, buffer, sizeof(buffer));
63          if (w == -1) {
64               perror("write");
65               exit(EXIT_FAILURE);
66          }
67          close(data_socket);
68          if (down_flag) {
69               break;
70          }
71     }
72     close(connection_socket);
73     unlink(SOCKET_NAME);
74     exit(EXIT_SUCCESS);
75 }
```

---

seq_server.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <sys/types.h>
6 #include <sys/un.h>
7 #include <unistd.h>
8 #include "connection.h"

9 int main(int argc, char *argv[])
10 {
11     int ret;
12     int data_socket;
13     ssize_t r, w;
14     struct sockaddr_un addr;
15     char buffer[BUFFER_SIZE];

16     data_socket = socket(AF_UNIX, SOCK_SEQPACKET, 0);
17     if (data_socket == -1) {
18          perror("socket");
19          exit(EXIT_FAILURE);
20     }

21     memset(&addr, 0, sizeof(addr));
22     addr.sun_family = AF_UNIX;
23     strncpy(addr.sun_path, SOCKET_NAME, sizeof(addr.sun_path) - 1);

24     ret = connect(data_socket, (const struct sockaddr *) &addr,
                     sizeof(addr));
25     if (ret == -1) {
26          fprintf(stderr, "The server is down.\n");
```

```
27          exit(EXIT_FAILURE);
28      }

29      for (int i = 1; i < argc; ++i) {
30          w = write(data_socket, argv[i], strlen(argv[i]) + 1);
31          if (w == -1) {
32              perror("write");
33              break;
34          }
35      }

36      strcpy(buffer, "END");
37      w = write(data_socket, buffer, strlen(buffer) + 1);
38      if (w == -1) {
39          perror("write");
40          exit(EXIT_FAILURE);
41      }

42      r = read(data_socket, buffer, sizeof(buffer));
43      if (r == -1) {
44          perror("read");
45          exit(EXIT_FAILURE);
46      }

47      buffer[sizeof(buffer) - 1] = 0;
48      printf("Result = %s\n", buffer);
49      close(data_socket);
50      exit(EXIT_SUCCESS);
51  }
```

---

seq_client.c

The code should be clear to the reader, therefore a detailed analysis is omitted.

Let’s execute

```
[linux@ipc] ./seq_server &

[linux@ipc] ./seq_client 5 2
Result = 7

[linux@ipc] ./seq_client -3 9
Result 6

[linux@ipc] ./seq_client DOWN
Result = 0
[1]  + done       ./seq_server
```

Let’s take a look at the execution with `strace`

Shell 1

```
[linux@ipc] rm /tmp/9Lq7BNBnBycd6nxy.socket

[linux@ipc] strace ./seq_server
...output…
bind(3, {sa_family=AF_UNIX, sun_path="/tmp/9Lq7BNBnBycd6nxy.socket"}, 110) = 0
listen(3, 20)                           = 0
accept(3, NULL, NULL
```

Shell 2

```
[linux@ipc] strace ./seq_client 2 3
...output…
socket(AF_UNIX, SOCK_SEQPACKET, 0)      = 3
connect(3, {sa_family=AF_UNIX, sun_path="/tmp/9Lq7BNBnBycd6nxy.socket"}, 110) = 0
write(3, "2\0", 2)                      = 2
write(3, "3\0", 2)                      = 2
write(3, "END\0", 4)                     = 4
read(3, "5\0D\0\0\1\0\0\0\0\0\0", 12)   = 12
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x1), ...}) = 0
brk(NULL)                               = 0x55787835b000
brk(0x55787837c000)                       = 0x55787837c000
write(1, "Result = 5\n", 11Result = 5 )          = 11
close(3)
```

Shell 1

```
accept(3, NULL, NULL
read(4, "2\0", 12)                      = 2
read(4, "3\0", 12)                      = 2
read(4, "END\0", 12)                     = 4
write(4, "5\0D\0\0\1\0\0\0\0\0\0", 12)  = 12
close(4)                                = 0
accept(3, NULL, NULL
```

As the execution shows, each `write` performed by the client corresponds to a distinct `read` on the server side. The messages ‘`2`’, ’`3`’, and ’`END`’ are received separately and in order, preserving their boundaries. Unlike stream-oriented sockets, the data is not merged into a continuous byte stream, but delivered as discrete messages.
This behavior confirms the semantics of `SOCK_SEQPACKET`, where communication is connection-oriented while maintaining message integrity.

Conclusion

UNIX domain sockets are widely used for efficient inter-process communication on the same machine, providing fast and secure data exchange without involving the network stack. They are commonly employed by system services, databases, and web backends for low-latency communication, as well as in graphical systems and container environments. They also support advanced IPC features such as file descriptor passing and credential exchange, enabling secure resource sharing and client authentication.

We have seen how UNIX domain sockets can be used to create full-duplex communication channels between processes, enabling not only data exchange but also the transfer of file descriptors, allowing multiple processes to access the same underlying open file description. We have also examined the distinction between *named sockets* and *abstract sockets*, and explored the three types of communication models: `SOCK_STREAM`, `SOCK_DGRAM`, and `SOCK_SEQPACKET`.

VI.7 Terminal I/O

Terminals are *character-based devices* used to host interactive login sessions. Historically, a terminal system consisted of two main components: a DTE (*Data Terminal Equipment*) and a DCE (*Data Communications Equipment*), typically a modem. These devices were connected via a serial interface such as RS-232. In this setup, the DTE generated *control signals*, while the DCE responded with *status signals*, enabling communication between a user and a remote system.

With the evolution of computing, this hardware-based model gradually transitioned into a software-based approach. Modern operating systems now provide *virtual terminal interfaces* that emulate the behavior of physical terminals. These software abstractions are commonly referred to as TTYs (*teletypewriters*). They allow multiple terminal sessions to exist simultaneously, enabling users to interact with the system through terminal emulators, remote connections (such as `SSH`), or graphical interfaces, all without requiring dedicated physical hardware.

In modern Linux systems, the hardware abstraction is implemented entirely in software through the kernel's *TTY subsystem*. The kernel exposes terminal interfaces as *character device files* under `/dev`, such as `/dev/tty`, `/dev/ttyS*` (serial ports), and *virtual consoles* like `/dev/tty1` -`/dev/ttyN`.

Linux distinguishes between several types of terminals:

Virtual Consoles (VTs)[81]
: These are kernel-managed terminals accessible via key combinations, such as `Ctrl+Alt+F1-F6`. Each VT corresponds to a device like `/dev/tty1`. They provide direct text-based interaction with the system without requiring a graphical environment.

Pseudo-terminals (PTYs)[82]
: PTYs emulate terminal behavior in user space and are essential for *terminal emulators* and *remote sessions*. A PTY consists of a *master/slave pair.*

  The *master* side is controlled by a *terminal emulator* (e.g., `GNOME Terminal`, `xterm`). The *slave* side appears as a *standard terminal device* (e.g., `/dev/pts/N`) to user processes like shells

Controlling Terminal and Sessions
: Each login session is associated with a *controlling terminal*. The kernel tracks process groups and session leaders, enabling job control.

When the system boots, one of the final steps is to present a *login prompt* to the user on a *virtual console*. This is handled by `getty`, which runs on a terminal device such as `/dev/tty1`. After the user enters their credentials, `getty` invokes the `login` program, which authenticates the user and then executes the user's shell. The shell runs attached to the same terminal, which in this case is a *real terminal* (the *virtual console*).

In a *graphical system*, users typically interact with *terminal emulators* rather than *real terminals*. Terminal emulators create *pseudoterminals*, allowing programs such as shells to run as if they were connected to a real terminal.

---

81 See the `vcs(4)` manual page

82 See the `pty(7)` manual page

The *TTY layer* in the Linux kernel handles:

- Line discipline (e.g., canonical vs non-canonical mode)
- Input processing (echoing, buffering, special character handling)
- Signal generation (interrupt, suspend, quit)
- Output processing

User-space programs interact with terminals through standard file descriptors (`stdin`, `stdout`, and `stderr`) using system calls such as `read`, `write`, and `ioctl`. Terminal behavior can be configured using APIs such as `termios`, which are covered in the following sections.

VI.7.1 Identifying terminal file descriptors: `isatty`

In the previous chapter, we discussed the `ttyname` and `ctermid` functions, which retrieve the terminal name and the controlling terminal associated with a process. In this section, we examine how to check whether a given file descriptor is associated with a terminal device.

To know if a file descriptor refers to a terminal, we can use the `isatty`[83] function, which is defined as follows.

```c
#include <unistd.h>

int isatty(int fd);

            Returns: 1 if fd refers to a terminal, otherwise 0 and errno is set
```

The `fd` argument is the descriptor to check.
If `fd` refers to a terminal the call returns `1`, otherwise it returns `0`.

The `istty.c` program tests whether some descriptors refer to terminals using `isatty`.

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    if (isatty(STDIN_FILENO))
         printf("\nFD %d is a terminal", STDIN_FILENO);
    if (isatty(STDOUT_FILENO))
         printf("\nFD %d is a terminal", STDOUT_FILENO);
    if (isatty(STDERR_FILENO))
         printf("\nFD %d is a terminal", STDERR_FILENO);

    int fd = open("./file.txt", O_RDONLY);
    if (fd == -1) {
         perror("open");
         return 1;
```

83 See the `isatty(3)` manual page

```
15    }
16    if (isatty(fd))
17          printf("\nFD: %d is a terminal", fd);
18    else
19          printf("\nFD: %d not a terminal", fd);

20    return 0;
21 }
```

---

istty.c

Let's execute

```
[linux@ipc] ./istty
FD 0 is a terminal
FD 1 is a terminal
FD 2 is a terminal
FD: 3 not a terminal
```

As we can see, the *standard descriptors* refer to terminals, while `fd` does not, since is refers to a regular file.

If we redirect `stdout` to a file

```
[linux@ipc] ./istty > log.log

[linux@ipc] cat log.log
FD 0 is a terminal
FD 2 is a terminal
FD: 3 not a terminal
```

`fd 1` does not appear in the list, since it refers to a regular file. The shell performs redirection before executing the program and `STDOUT_FILENO` (fd 1) points to `log.log`.

Let's replace the `isatty` and `printf` calls as follows[84]

```
printf("\nFD %d %s a terminal", STDOUT_FILENO, isatty(STDOUT_FILENO) ?
       "is" : "is NOT");
```

Let's execute redirecting both `stdout` and `stderr`

```
[linux@ipc] ./istty2 > log.log 2>&1

[linux@ipc] cat log.log
FD 0 is a terminal
FD 1 is NOT a terminal
FD 2 is NOT a terminal
FD 3 is NOT a terminal
```

We can see the complete list. Only `fd 0` (`stdin`) refers to a temrinal.

84 Program `istty2.c` in the source code

Let's execute `istty.c` with `strace`

```
[linux@ipc] strace ./istty

ioctl(0, TCGETS2, {c_iflag=ICRNL|IXON|IUTF8, c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|
OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHOK|
IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x1), ...}) = 0
brk(NULL)                               = 0x559d96722000
brk(0x559d96743000)                     = 0x559d96743000
write(1, "\n", 1 )                       = 1
ioctl(1, TCGETS2, {c_iflag=ICRNL|IXON|IUTF8, c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|
OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHOK|
IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
write(1, "FD 0 is a terminal\n", 19FD 0 is a terminal )    = 19
ioctl(2, TCGETS2, {c_iflag=ICRNL|IXON|IUTF8, c_oflag=NL0|CR0|TAB0|BS0|VT0|FF0|
OPOST|ONLCR, c_cflag=B38400|CS8|CREAD, c_lflag=ISIG|ICANON|ECHO|ECHOE|ECHOK|
IEXTEN|ECHOCTL|ECHOKE, ...}) = 0
write(1, "FD 1 is a terminal\n", 19FD 1 is a terminal)    = 19
openat(AT_FDCWD, "./file.txt", O_RDONLY) = 3
ioctl(3, TCGETS2, 0x7ffd4bf93ee0)       = -1 ENOTTY (Inappropriate ioctl for
device)
write(1, "FD 2 is a terminal\n", 19FD 2 is a terminal)    = 19
write(1, "FD: 3 not a terminal", 20FD: 3 not a terminal)    = 20
exit_group(0)                           = ?
```

For each descriptor the program has called `ioctl` passing the descriptor number and the `TCGETS2` operation. If the file descriptor is attached to a *TTY device* (like `/dev/tty` or `/dev/pts/N`), it succeeds, otherwise it fails with `ENOTTY`.

`TCGETS2`[85] is a Linux-specific `ioctl` operation that retrieves extended terminal attributes.

85 See the `TCSETS(2const)` manual page

VI.7.2 Terminal properties: `termios`

Each terminal device maintains a set of configurable attributes that control its behavior. These attributes can be queried and modified using dedicated system calls and library functions. The terminal configuration is represented by the `struct termios`[86], which is defined in the `termios.h` header file as follows[87].

```
struct termios
{
   tcflag_t c_iflag;          /* input mode flags */
   tcflag_t c_oflag;          /* output mode flags */
   tcflag_t c_cflag;          /* control mode flags */
   tcflag_t c_lflag;          /* local mode flags */
   cc_t c_line;               /* line discipline */
   cc_t c_cc[NCCS];           /* control characters */

   /* Input and output baud rates. */
   __extension__ union {
      speed_t __ispeed;
      speed_t c_ispeed;
   };

#define _HAVE_STRUCT_TERMIOS_C_ISPEED 1
   __extension__ union {
      speed_t __ospeed;
      speed_t c_ospeed;
   };
#define _HAVE_STRUCT_TERMIOS_C_OSPEED 1
};
```

`tcflag_t` is an unsigned integer type used to represent the various bit masks for terminal flags.

`cc_t` is an unsigned integer type used to represent characters associated with various terminal control functions.

`int NCCS` is a macro defining the number of elements in the `c_cc` array.

The `termios` structure is composed of several groups of flags, each controlling a specific aspect of terminal I/O, such as input processing, output processing, control parameters, and local modes.

`c_iflag` (*input modes*)
: This field controls how input data received from the terminal is processed before being made available to user programs. Typical functionalities include:

  - Handling of special characters (e.g., interrupt, erase)
  - Input translation (e.g., carriage return to newline)
  - Enabling/disabling flow control (e.g., `XON`/`XOFF`)

86 See the `termios(3)` manual page
87 On Debian-based systems `x86_64-linux-gnu/bits/termios-struct.h`

`c_oflag` (*output modes*)
: This field determines how output data is processed before being sent to the terminal. Examples include:

  - Automatic conversion of newline (`\n`) to carriage return + newline (`\r\n`)
  - Control of output delays and formatting behavior

`c_cflag` (*control modes*)
: This field specifies low-level hardware-related settings for the terminal interface. It typically includes:

  - Baud rate (input/output speed)
  - Character size (e.g., 7-bit, 8-bit)
  - Parity and stop bits
  - Enabling/disabling the receiver

`c_lflag` (*local modes*)
: This field controls higher-level terminal features related to user interaction. Common behaviors include:

  - Canonical mode (line-based input vs raw input)
  - Echoing of input characters
  - Signal generation (e.g., `Ctrl+C` → `SIGINT`, `Ctrl+Z` → `SIGTSTP`)

`c_cc` (control characters)
: This is an array of special control characters that define key terminal actions. Examples:

  - `VINTR` → interrupt character (usually `Ctrl+C`)
  - `VEOF` → end-of-file (usually `Ctrl+D`)
  - `VERASE` → erase character (usually Backspace)

Together, these fields define how a terminal receives input, processes and buffers it, generates signals and formats output.

The following tables list the flags that can be set for each field.

IGNBRK	Ignore BREAK condition on input.
BRKINT	If IGNBRK is set, a BREAK is ignored. If it is not set but BRKINT is set, then a BREAK causes the input and output queues to be flushed, and if the terminal is the controlling terminal of a foreground process group, it will cause a SIGINT to be sent to this foreground process group. When neither IGNBRK nor BRKINT are set, a BREAK reads as a null byte ('\0'), except when PARMRK is set, in which case it reads as the sequence \377 \0 \0
IGNPAR	Ignore framing and parity errors
PARMRK	Mark input bytes with parity/framing errors using the sequence \377 \0 <byte>. Valid \377 bytes may be duplicated as \377 \377. Effective only if INPCK is set and IGNPAR is not set
INPCK	Enable input parity checking
ISTRIP	Strip off the eighth bit
INLCR	Translate newline (NL) to carriage return (CR) on input
IGNCR	Ignore carriage return (CR) on input
ICRNL	Translate carriage return (CR) to newline (NL) on input (unless IGNCR is set)
IUCLC	Map uppercase characters to lowercase on input
IXON	Enable XON/XOFF flow control on output
IXANY	Typing any character will restart stopped output. The default is to allow just the START character to restart output
IXOFF	Enable XON/XOFF flow control on input
IMAXBEL	Ring bell when input queue is full. Linux treats this as always enabled
IUTF8	Input is UTF-8; enables correct character erase in canonical mode

Tab 11. c_iflag flags

OPOST	Enable implementation-defined output processing
OLCUC	Map lowercase characters to uppercase on output
ONLCR	Map NL to CR-NL on output
OCRNL	Map CR to NL on output
ONOCR	Do not output CR at column 0
ONLRET	NL is assumed to do the CR function; column is set to 0 after both NL and CR
OFILL	Send fill characters for a delay, rather than using a timed delay
OFDEL	Fill character is ASCII DEL (0177). If unset, fill character is ASCII NUL ('\0')
NLDLY	Newline delay mask. Values: NL0 and NL1
CRDLY	Carriage return delay mask. Values: CR0, CR1, CR2, CR3
TABDLY	Horizontal tab delay mask. Values are TAB0, TAB1, TAB2, TAB3 (or XTABS). A value of TAB3, that is, XTABS, expands tabs to spaces (with tab stops every eight columns)
BSDLY	Backspace delay mask. Values: BS0 or BS1
VTDLY	Vertical tab delay mask. Values: VT0, VT1
FFDLY	Form feed delay mask. Values: FF0, FF1

Tab 12. c_oflag flags

`OFDEL` and `BSDLY` are not implemented.

`CBAUD`	Baud speed mask (4+1 bits)
`CBAUDEX`	Extra baud speed mask (1 bit), included in `CBAUD`
`CSIZE`	Character size mask. Values: `CS5`, `CS6`, `CS7`, `CS8`
`CSTOPB`	Use two stop bits instead of one
`CREAD`	Enable receiver
`PARENB`	Enable parity generation (output) and checking (input)
`PARODD`	Use odd parity if set; otherwise even parity
`HUPCL`	Lower modem control lines after last close (hang up)
`CLOCAL`	Ignore modem control lines
`LOBLK`	Block output from a noncurrent shell layer. Not implemented
`CIBAUD`	Mask for input speeds (same values as `CBAUD`, shifted). Available via `ioctl`
`CMSPAR`	Enable “stick” parity (mark/space parity)
`CRTSCTS`	Enable `RTS`/`CTS` hardware flow control

Tab 13. `c_cflag` flags

The *control mode flags* also include a field that specifies the number of bits per character. The `CSIZE` macro can be used as a mask to extract this value from the `c_cflag` field, as in the following expression:

```
settings.c_cflag & CSIZE
```

The result can then be compared, to determine the character size, against the values:

`tcflag_t CSIZE`	Mask for the number of bits per character
`tcflag_t CS5`	Five bits per byte
`tcflag_t CS6`	Six bits per byte
`tcflag_t CS7`	Seven bits per byte
`tcflag_t CS8`	Eight bits per byte

For example:

```
switch (settings.c_cflag & CSIZE) {
      case CS5: printf("5 bits per character\n"); break;
      case CS6: printf("6 bits per character\n"); break;
      case CS7: printf("7 bits per character\n"); break;
      case CS8: printf("8 bits per character\n"); break;
}
```

ISIG	Generate signals (SIGINT, SIGQUIT, SIGTSTP, etc.) when special characters (INTR, QUIT, SUSP, DSUSP) are received
ICANON	Enable canonical (line-based) input mode
XCASE	Uppercase-only terminal mode with special input/output conversions
ECHO	Echo input characters to the terminal
ECHOE	If ICANON is set, ERASE deletes the previous character and WERASE deletes the previous word
ECHOK	If ICANON is set, KILL deletes the entire current line
ECHONL	If ICANON is set, echo newline even if ECHO is disabled
ECHOCTL	Echo control characters as ^X notation (e.g., Ctrl+C → ^C)
ECHOPRT	Print characters as they are being erased
ECHOKE	Echo line deletion by erasing each character (like multiple backspaces)
DEFECHO	Echo only when a process is reading. Not implemented
FLUSHO	Output is flushed; toggled by DISCARD character
NOFLSH	Do not flush input/output queues when generating signals (INT, QUIT, SUSP)
TOSTOP	Send SIGTTOU when a background process writes to the terminal
PENDIN	Reprint input buffer when next character is read
IEXTEN	Enable implementation-defined input processing (e.g., WERASE, REPRINT, LNEXT)

Tab 14. c_lflag flags

VDISCARD	Toggle discarding of pending output (Ctrl+O)
VDSUSP	Delayed suspend (Ctrl+Y); sends SIGTSTP when read
VEOF	End-of-file (Ctrl+D); delivers buffered input immediately, or returns EOF if buffer is empty
VEOL	Additional end-of-line character
VEOL2	Secondary end-of-line character
VERASE	Erase previous character (Backspace/DEL/Ctrl+H)
VINTR	Interrupt (Ctrl+C); sends SIGINT
VKILL	Kill entire line (Ctrl+U / Ctrl+X)
VLNEXT	Literal next (Ctrl+V); next character is taken literally
VMIN	Minimum number of characters for noncanonical read
VQUIT	Quit (Ctrl+); sends SIGQUIT
VREPRINT	Reprint unread input (Ctrl+R)
VSTART	Resume output (Ctrl+Q)
VSTATUS	Status request (Ctrl+T); display process info and send SIGINFO
VSTOP	Stop output (Ctrl+S)
VSUSP	Suspend (Ctrl+Z); sends SIGTSTP
VSWTCH	Switch shell layers (historical)
VTIME	Timeout (in deciseconds) for noncanonical read
VWERASE	Erase previous word (Ctrl+W)

Tab 15. c_cc indices

An individual terminal special character can be disabled by assigning the value `_POSIX_VDISABLE` to the corresponding element in the `c_cc` array. When set to this value, the associated control character is effectively ignored by the terminal driver and no longer triggers its usual function.

Several `ioctl` operations are available to get and set terminal attributes. They are listed in the `ioctl_tty(2)` manual page.

VI.7.3 Retrieving and changing terminal settings: `tcgetattr` and `tcsetattr`

To retrieve and set the parameter of a terminal we can use the following functions[88].

```
#include <termios.h>
#include <unistd.h>

int tcgetattr(int fd, struct termios *termios_p);

int tcsetattr(int fd, int optional_actions, const struct termios
              *termios_p);

                                          Return: on success 0, on error -1
```

The `tcgetattr` function gets the parameters associated with the object referred by `fd`, and stores them in the `termios` structure referenced by `termios_p`.
`tcgetattr` may be invoked from a *background process*. However, the terminal attributes may be subsequently changed by a *foreground process*.

The `tcsetattr` function sets the parameters associated with the terminal from the `termios` structure referred to by `termios_p`.
The `optional_actions` argument specifies when the changes take effect and can have the following values:

`TCSANOW`
: The change occurs immediately

`TCSADRAIN`
: The change occurs after all output written to `fd` has been transmitted. This option should be used when changing parameters that affect output

`TCSAFLUSH`
: The change occurs after all output written to the object referred by `fd` has been transmitted, and all input that has been received but not read will be discarded before the change is made

The `tcattr.c` program uses tcsetattr to disable the `Ctrl+C` inerrupt character.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <termios.h>

4 int main() {
5     struct termios oldt, newt;

6     if (tcgetattr(STDIN_FILENO, &oldt) == -1) {
7           perror("tcgetattr");
8           return 1;
9     }
```

88 See the `termios(3)` manual page

```
10    newt = oldt;
11    newt.c_cc[VINTR] = _POSIX_VDISABLE;

12    if (tcsetattr(STDIN_FILENO, TCSANOW, &newt) == -1) {
13          perror("tcsetattr");
14          return 1;
15    }

16    printf("Ctrl+C is now disabled. Press Enter to continue...\n");
17    getchar();
18    if (tcsetattr(STDIN_FILENO, TCSANOW, &oldt) == -1) {
19          perror("tcsetattr");
20          return 1;
21    }
22    printf("Ctrl+C restored.\n");
23    return 0;
24 }
```

---

tcattr.c

At line 5, two `termios` structures (`oldt` and `newt`) are declared. These will store the original terminal settings and the modified ones, respectively.

At line 6, `tcgetattr` retrieves the current terminal attributes associated with `STDIN_FILENO` and stores them in `oldt`. This allows the program to preserve the original configuration so it can be restored later.

At line 10, the original settings are copied from `oldt` into `newt`, so that modifications can be applied without altering the saved configuration.

At line 11, the `VINTR` control character (typically mapped to `Ctrl+C`) is disabled by setting the corresponding entry in the `c_cc` array to `_POSIX_VDISABLE`. As a result, pressing `Ctrl+C` will no longer generate a `SIGINT` signal.

At line 12, `tcsetattr` applies the modified terminal settings stored in `newt`. The `TCSANOW` option specifies that the changes take effect immediately.

The program then allows the user to observe the effect of this modification by printing a message and waiting for input at lines 16–17.
At line 18, the original terminal settings are restored using another call to `tcsetattr`, ensuring that the terminal returns to its normal behavior after the program terminates.

Let’s execute

```
[linux@ipc] ./tcattr
Ctrl+C is now disabled. Press Enter to continue...
^C^C^C
Ctrl+C restored.
```

`Ctrl+C` does not work until we press `enter`.

VI.7.4 Canonical, noncanonical and raw mode

Each terminal maintains two queues: an *input queue* and an *output queue*. These queues are managed by the kernel and are independent of the buffering mechanisms used by standard I/O streams.

The *input queue* stores characters received from the terminal that have not yet been read by a process. Its maximum size is defined by `MAX_INPUT`. If *input flow control* is enabled (by setting the `IXOFF` flag), the terminal driver automatically sends `STOP` and `START` characters to the terminal to prevent the queue from overflowing. Otherwise, if data arrives too quickly, some input characters may be lost.

The *output queue* contains characters written by processes that have not yet been transmitted to the terminal. If *output flow control* is enabled (by setting the `IXON` flag), the terminal driver responds to `STOP` and `START` characters received from the terminal, suspending and resuming output transmission accordingly.

Clearing the terminal *input queue* means discarding any characters that have been received but not yet read. Similarly, clearing the terminal *output queue* means discarding any characters that have been written but not yet transmitted.

A terminal can operate in two modes: *canonical mode* (line-oriented input) and *noncanonical mode* (character-oriented input).

Canonical mode

Input is made available line by line. An input line is available when one of the line delimiters is typed. Line delimiters are: `NL`, `EOL`, `EOL2`, and `EOF` at the start of line.
The *line delimiter* is included in the buffer returned by `read`, except for `EOF`.
Line editing is enabled, meaning operations such as `ERASE` and `KILL` can be performed.
If the `IEXTEN` flag is set, also `WERASE`, `REPRINT` and `LNEXT` are available.

A `read` returns at most one line of input. If the `read` requested fewer bytes than are available in the current line of input, then only as many bytes as requested are read, and the remaining characters will be available for a future `read`.

The maximum line length is 4096 chars, including the *terminating newline character* and lines longer than 4096 characters are truncated.
After 4095 characters, input processing, such as `ISIG` and `ECHO`* processing, continues, but any input data after 4095 characters up to (but not including) the *terminating newline* is discarded

Noncanonical mode

Input is available immediately, without the user having to type a *line-delimiter character.* No input processing is performed and *line editing* is disabled.

The `read` buffer will only accept 4095 chars to provide the necessary space for a *newline char* if the input mode is switched to *canonical*.

The settings `c_cc[VMIN]` and `c_cc[VTIME]` determine how a `read` completes. There are four cases:

- `MIN == 0, TIME == 0` (*polling read*)
  If data is available, `read` returns immediately, with the lesser of the number of bytes available, or the number of bytes requested. If no data is available, `read` returns 0

- `MIN > 0, TIME == 0` (*blocking read*)
  `read` blocks until `MIN` bytes are available, and returns up to the number of bytes requested

- `MIN == 0, TIME > 0` (*read with timeout*)
  `TIME` specifies the limit for a timer in tenths of a second. The timer is started when `read` is called. `read` returns either when at least one byte of data is available, or when the timer expires. If the timer expires without any input becoming available, `read` returns 0. If data is already available at the time of the call to `read`, the call behaves as though the data was received immediately after the call

- `MIN > 0, TIME > 0` (*read with interbyte timeout*)
  `TIME` specifies the limit for a timer in tenths of a second. Once an initial byte of input becomes available, the timer is restarted after each further byte is received.

  `read` returns when any of the following conditions is met:
  - `MIN` bytes have been received
  - The *interbyte timer* expires
  - The number of bytes requested by `read` has been received

  Since the timer starts only after the initial byte becomes available, at least one byte is read. If data is already available at the time of the `read`, the call behaves as though the data was received immediately after the call.

`VMIN` and `VTIME` control how the `read` system call behaves when input is performed in noncanonical mode.

- `VMIN` specifies the minimum number of bytes that must be available in the input queue before `read` returns

- `VTIME` specifies the maximum amount of time to wait for input, expressed in tenths of a second (0.1 s units), before `read` returns even if no sufficient data has arrived

The `tccanon.c` program illustrates the behavior of the terminal in both *canonical* and *noncanonical* modes. It begins in *canonical mode*, where input is processed line by line, and subsequently switches to *noncanonical mode*, where input is made available immediately on a per-character basis, highlighting the differences in input handling between the two configurations.

```c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <stdlib.h>
```

```c
void set_canonical(struct termios *oldt) {
    if (tcsetattr(STDIN_FILENO, TCSANOW, oldt) == -1) {
        perror("tcsetattr");
        exit(1);
    }
}

void set_noncanonical(struct termios *newt) {
    newt->c_lflag &= ~(ICANON | ECHO);
    newt->c_cc[VMIN] = 1;
    newt->c_cc[VTIME] = 0;

    if (tcsetattr(STDIN_FILENO, TCSANOW, newt) == -1) {
        perror("tcsetattr");
        exit(1);
    }
}

int main() {
    struct termios oldt, newt;
    char c;

    if (tcgetattr(STDIN_FILENO, &oldt) == -1) {
        perror("tcgetattr");
        return 1;
    }

    newt = oldt;

    printf("=== Canonical mode ===\n");
    printf("Type something (input is line-buffered, press Enter):\n");

    while ((c = getchar()) != '\n' && c != EOF) {
        printf("Read char: %d ('%c')\n", c, c);
    }

    printf("\n=== Noncanonical mode ===\n");
    printf("Type characters (no Enter needed, Ctrl+D to exit):\n");
    set_noncanonical(&newt);

    while (1) {
        if (read(STDIN_FILENO, &c, 1) == 1) {
            printf("Read char: %d\n", c);
            if (c == 4) {
                break;
            }
        }
    }

    set_canonical(&oldt);
    printf("\nTerminal restored.\n");
```

```
46    return 0;
47 }
```

---

tccanon.c

At line 5, the `set_canonical` function restores the terminal to its original configuration by applying the settings stored in the `oldt` structure using `tcsetattr`.

At line 11, the `set_noncanonical` function disables the canonical mode and input echo by clearing the `ICANON` and `ECHO` flags in the `c_lflag` field.
Next, it sets `VMIN` to 1 and `VTIME` to 0, so `read` blocks until at least one byte is available and no timeout is used. This configuration results in a *blocking read* that returns as soon as a single character is received. At line 15 the new settings are applied using `tcsetattr`.
At line 23, the program retrieves the current terminal attributes associated with `STDIN_FILENO` and stores them in the `oldt` structure. The settings are then copied into `newt` at line 27.
At lines 28–29, the program prompts the user (canonical mode).
The loop at line 30 uses `getchar` to read input, which is *line-buffered*, so characters are not made available to the program until the user presses `Enter`. The program then prints each character read from the input buffer.
At line 35, the program switches to noncanonical mode by calling `set_noncanonical`. It then enters an infinite loop at line 36, where input is read using `read`.
The input is processed one character at a time without requiring the `Enter` key. Each character is immediately printed. If the user enters `Ctrl+D` (`4`), the loop terminates.
At line 44, the original settings are restored using `set_canonical`, ensuring that the terminal returns to its normal behavior before the program exits at line 46.

Let’s run the program by entering `Hello` in both modes

```
[linux@ipc] ./tccanon
=== Canonical mode ===
Type something (input is line-buffered, press Enter):
Hello
Read char: 72 ('H')
Read char: 101 ('e')
Read char: 108 ('l')
Read char: 108 ('l')
Read char: 111 ('o')

=== Noncanonical mode ===
Type characters (no Enter needed, Ctrl+D to exit):
Read char: 72
Read char: 101
Read char: 108
Read char: 108
Read char: 111
Read char: 4

Terminal restored.
```

In *canonical mode*, the terminal buffers the entire line, and `getchar` receives the input only after the `Enter` key is pressed.
In *noncanonical mode*, each character is delivered immediately as it is typed. There is no need to press `Enter`, and input is not *line-buffered*. In addition, *echo* is disabled, so the characters typed by the user are not displayed on the screen.

If `Backspace` is pressed in *canonical mode*, the previously typed character is deleted before the input is delivered to the program. This behavior shows that *line editing* is enabled and handled by the *terminal driver*. `Backspace` is interpreted as the erase character (`VERASE`).
In *noncanonical mode*, pressing `Backspace` does not perform any *line editing*. Instead, the corresponding character, typically ASCII value `127`, is passed directly to the program, which reads and prints it as ordinary input.

`Ctrl+C` terminates the program in both canonical and noncanonical modes, since it generates a `SIGINT` signal when signal handling (`ISIG`) is enabled.
`Ctrl+D` behaves differently in the two modes.
In canonical mode, `Ctrl+D` is interpreted as the *end-of-file* (`EOF`) control character (`VEOF`). If it is entered at the beginning of a line (i.e., when the input buffer is empty), it causes `getchar` to return `EOF` immediately. If it is entered after some characters, those characters are made available to the program without requiring the `Enter` key.
Since `Ctrl+D` is not an *end-of-line* character and does not insert a *newline* into the input stream, the program may continue to wait for additional input after consuming the buffered data, as subsequent calls to `getchar` will block until new input is provided.
In noncanonical mode, `Ctrl+D` has no special meaning (unless explicitly handled by the program). It is simply read as a normal character with ASCII value 4.

Noncanonical mode is typically used by programs that perform direct terminal interaction and screen manipulation, such as the `vi` editor. In these applications, many commands consist of single keystrokes and are not terminated by a newline, making character-by-character input essential.
In addition, such programs must prevent the terminal driver from interpreting special characters, as these may conflict with application-defined commands.
For example, `Ctrl+D` is normally interpreted by the terminal as the *end-of-file* (`VEOF`) character in *canonical mode*. However, in `vi`, `Ctrl+D` is used as a command (e.g., to scroll down), so it must be delivered to the program as ordinary input rather than being handled by the terminal.

Raw mode

An additional mode in which a terminal can operate is the *raw mode*. In this mode input is available character by character, echoing is disabled, and all special processing of terminal input and output characters is disabled.

The `cfmakeraw` function sets the terminal to operate in *raw mode.*

```
#include <termios.h>
#include <unistd.h>

void cfmakeraw(struct termios *termios_p);
```

The terminal attributes are set as follows:

```
termios_p->c_iflag &= ~(IGNBRK | BRKINT | PARMRK | ISTRIP | INLCR | IGNCR
                        | ICRNL | IXON);
termios_p->c_oflag &= ~OPOST;
termios_p->c_lflag &= ~(ECHO | ECHONL | ICANON | ISIG | IEXTEN);
termios_p->c_cflag &= ~(CSIZE | PARENB);
termios_p->c_cflag |= CS8;
```

The `tcraw.c` program sets the *raw mode* using `cfmakeraw`.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <termios.h>
4 #include <stdlib.h>

5 void set_canonical(struct termios *oldt) {
6     if (tcsetattr(STDIN_FILENO, TCSANOW, oldt) == -1) {
7          perror("tcsetattr");
8          exit(1);
9     }
10 }
11 int main() {
12    struct termios oldt, newt;
13    int c;

14    if (tcgetattr(STDIN_FILENO, &oldt) == -1) {
15         perror("tcgetattr");
16         return 1;
17    }

18    newt = oldt;
19    printf("\n=== Raw mode ===\n");
20    printf("Type characters (no Enter needed, Ctrl+D to exit):\n");
21    cfmakeraw(&newt);
22    if (tcsetattr(STDIN_FILENO, TCSANOW, &newt) == -1) {
23         perror("tcsetattr");
24         exit(1);
25    }

26    while (1) {
27         if (read(STDIN_FILENO, &c, 1) == 1) {
28              printf("Read char: %d\n", c);
29              if (c == 4) {
30                   break;
31              }
32         }
33    }
34    set_canonical(&oldt);
35    printf("\nTerminal restored.\n");
36    return 0;
```

```
37 }
```

tcraw.c

At line 21, the `cfmakeraw` function is used to modify the `newt` structure so that it represents a terminal configuration in *raw mode*. This disables canonical input processing, echoing, signal generation, and all input/output transformations.

At line 22, the new terminal settings are applied using `tcsetattr`, which switches the terminal to *raw mode* with immediate effect (`TCSANOW`).

Let's execute and enter `hello`.

```
[linux@ipc] ./tcraw

=== Raw mode ===
Type characters (no Enter needed, Ctrl+D to exit):
Read char: 104
        Read char: 101
                Read char: 108
                        Read char: 108
                                Read char: 111
                                        Read char: 3
                                                Read char: 24
                                                        Read char: 26
                                                                Read char: 4

Terminal restored.
```

As the execution shows, each character is delivered to the program immediately after it is typed, without requiring the `Enter` key. The numeric values printed correspond to the ASCII codes of the characters entered (for example, `104` for `'h'`, `101` for `'e'`, etc.).
Control characters are also read as ordinary input. For instance:

- `3` corresponds to `Ctrl+C`
- `24` corresponds to `Ctrl+X`
- `26` corresponds to `Ctrl+Z`
- `4` corresponds to `Ctrl+D`

Since the terminal is in *raw mode*, special processing is disabled. As a result, *control characters* do not generate signals (e.g., `Ctrl+C` does not produce `SIGINT`) and are instead passed directly to the program as input bytes.
The irregular spacing in the output is due to the absence of output processing (`OPOST` is disabled), so newline characters are not automatically translated (e.g., from `\n` to `\r\n`), which affects how the text is displayed on the terminal.

In raw mode, the terminal does not interpret input or output, everything is passed directly between the keyboard and the program. Raw mode is used by interactive programs that require immediate, character-by-character input and full control over special characters and terminal behavior.

VI.7.5 Line control

The *line discipline* is the component of the terminal subsystem that processes input and output data between the user program and the terminal device. It implements input processing, line editing, signal generation, and character transformations between the terminal device and user programs. The line discipline exists between the terminal and the program that uses it.

The following functions are used to perform various operations on the terminal line discipline.

```
#include <termios.h>
#include <unistd.h>

int tcsendbreak(int fd, int duration);
int tcdrain(int fd);
int tcflush(int fd, int queue_selector);
int tcflow(int fd, int action);

                                Return: on success 0, on error -1 and errno is set
```

The `tcsendbreak` function transmits a continuous stream of zero-valued bits on the terminal line, provided the device is using *asynchronous serial communication*.
If the `duration` argument is zero, the function sends zero-valued bits for a period of at least 0.25 seconds and not more than 0.5 seconds. If `duration` is nonzero, the transmission lasts for an implementation-defined interval, typically proportional to the specified value.
If the terminal is not using *asynchronous serial data transmission*, `tcsendbreak` returns without taking any action.

The `tcdrain` function waits until all output written to the object referred to by `fd` has been transmitted.

The `tcflush` function discards data written to the object referred to by `fd` but not transmitted, or data received but not read, depending on the value of `queue_selector`, which can be:

`TCIFLUSH`
: Flushes data received but not read

`TCOFLUSH`
: Flushes data written but not transmitted

`TCIOFLUSH`
: Flushes both data received but not read, and data written but not transmitted

The `tcflow` function suspends transmission or reception of data on the object referred to by `fd`, depending on the value of `action`, which can be:

`TCOOFF`
: Suspends output

`TCOON`
: Restarts suspended output

`TCIOFF`
: Transmits a `STOP` character, which stops the terminal device from transmitting data to the system

`TCION`
: Transmits a `START` character, which starts the terminal device transmitting data to the system

When a terminal device is opened, input and output are enabled by default, meaning that data transmission in both directions is active and not suspended.

The `tcline.c` program uses `tcflow` to suspend and resume terminal output and `tcflush` to discard pending input data.

```c
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <stdlib.h>

int main() {
    struct termios oldt, newt;
    int c;

    if (tcgetattr(STDIN_FILENO, &oldt) == -1) {
        perror("tcgetattr");
        return 1;
    }

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    if (tcsetattr(STDIN_FILENO, TCSANOW, &newt) == -1) {
        perror("tcsetattr");
        return 1;
    }

    printf("Noncanonical mode. Type characters (q to quit).\n");
    while ((c = getchar()) != 'q') {
        printf("Read: %c (%d)\n", c, c);

        if (c == 's') {
            printf("Stopping output for 3 seconds...\n");
            tcflow(STDOUT_FILENO, TCOOFF); // suspend output
            sleep(3);
            tcflow(STDOUT_FILENO, TCOON); // resume output
            printf("Output resumed.\n");
        }
        if (c == 'f') {
            printf("Flushing input buffer...\n");
```

```
30                tcflush(STDIN_FILENO, TCIFLUSH);
31            }
32    }

33    if (tcsetattr(STDIN_FILENO, TCSANOW, &oldt) == -1) {
34            perror("tcsetattr");
35            return 1;
36    }
37    printf("Terminal restored.\n");
38    return 0;
39 }
```

---

tcline.c

After retrieving the current terminal attributes, the program modifies the `c_lflag` field by clearing the `ICANON` and `ECHO` flags. This disables *canonical mode* and *echoing*, allowing input to be processed character by character without automatically displaying typed characters.

At line 14, the modified settings are applied using `tcsetattr`, which immediately switches the terminal to *noncanonical mode*.
At line 19, the program enters a loop where it waits for user input using `getchar`. Since canonical mode is disabled, each character is read as soon as it is typed, without requiring the `Enter` key.

When the user presses `s`, the program suspends terminal output at line 23 using `tcflow` with the `TCOOFF` argument. After a delay of 3 seconds with `sleep`, output is resumed using `tcflow` with `TCOON`.
If the user presses `f`, the program flushes the terminal input queue at line 30 using `tcflush` with the `TCIFLUSH` argument, discarding any unread input.

Finally, before terminating, the program restores the original terminal settings to ensure that normal terminal behavior is reestablished.

Let’ execute the program

```
[linux@ipc] ./tcline
Noncanonical mode. Type characters (q to quit).
Read: h (104)
Read: e (101)
Read: l (108)
Read: l (108)
Read: o (111)
Read: s (115)
Stopping output for 3 seconds...
Output resumed.
Read: L (76)
Read: i (105)
Read: n (110)
Read: u (117)
Read: x (120)
Read:
```

```
 (10)
Read: t (116)
Read: e (101)
Read: r (114)
Read: m (109)
Read: i (105)
Read: n (110)
Read: a (97)
Read: l (108)
Read: f (102)
Flushing input buffer...
Read: h (104)
Read: e (101)
Read: l (108)
Read: l (108)
Read: o (111)
Read: f (102)
Flushing input buffer...
```

As shown in the execution, each character is read and processed immediately after it is typed, without requiring the `Enter` key. This confirms that the terminal is operating in noncanonical mode.
When the user types `s`, the program invokes `tcflow` with the `TCOOFF` argument, suspending terminal output. During this period, no output is displayed for 3 seconds. After the delay, output is resumed using `tcflow` with `TCOON`, and the message “Output resumed.” is printed.
When the user types `f`, the program calls `tcflush` with the `TCIFLUSH` argument, which clears the terminal input queue by discarding any characters that have been typed but not yet read.
During the 3-second suspension, input is still accepted and buffered, but output is not displayed until it is resumed.

This example shows that we can control terminal line behavior programmatically by adjusting terminal attributes, enabling or disabling canonical mode, and using functions such as `tcflow` and `tcflush` to manage output flow and input buffering.

VI.7.6 Line speed

The *baud rate* represents the number of *bits per second* transmitted to and from the terminal. Separate baud rates can be specified for input and output. The `termios` interface provides functions to get and set the input and output baud rates stored in the termios structure. However, any changes made to these values do not take effect until `tcsetattr` is successfully called to apply the updated settings.

```
#include <termios.h>
#include <unistd.h>

speed_t cfgetispeed(const struct termios *termios_p);
speed_t cfgetospeed(const struct termios *termios_p);

int cfsetispeed(struct termios *termios_p, speed_t speed);
int cfsetospeed(struct termios *termios_p, speed_t speed);
int cfsetspeed(struct termios *termios_p, speed_t speed);

                              Return: on success 0, on error -1 and errno is set,
 cfgetispeed and cfgetospeed the input/output baud rate from the struct termios
```

Setting the speed to `B0` causes the modem to hang up by lowering the communication line. The actual transmission speed associated with `B38400` may be modified using the `setserial` utility. The input and output baud rates are stored within the `termios` structure. The `speed_t` type is an unsigned integer data type used to represent line speeds.

The function `cfgetospeed` retrieves the output baud rate from the `termios` structure referenced by `termios_p`.
The function `cfsetospeed` sets the output baud rate in the `termios` structure referenced by `termios_p` to the value specified by `speed`, which must be one of the predefined baud rate constants: B0, B50, B75, B110, B134, B150, B200, B300, B600, B1200, B1800, B2400, B4800, B9600, B19200, B38400, B57600, B115200, B230400, B460800, B500000, B576000, B921600, B1000000, B1152000, B1500000, B2000000.
These constants should be checked before use to determine whether they are supported on the system.
The zero baud rate (`B0`) is used to terminate a connection. When `B0` is specified, the modem control lines are no longer asserted, which typically results in the line being disconnected.
`CBAUDEX` is a mask that represents baud rates beyond those defined by POSIX.1, generally `57600` and higher. Therefore, the expression `B57600 & CBAUDEX` evaluates to a nonzero value.
Setting a baud rate outside the standard `Bnnn` constants can be achieved using the `TCSETS2` `ioctl`[89].

The function `cfgetispeed` returns the input baud rate stored in the `termios` structure.
The function `cfsetispeed` sets the input baud rate in the `termios` structure to the value specified by `speed`, which must be one of the predefined `Bnnn` constants. If the input baud rate is set to the value `0` (the numeric zero, not the symbolic constant B0), the input speed is set equal to the output baud rate.

The function `cfsetspeed` is a 4.4BSD extension that sets both input and output baud rates simultaneously using the same argument as `cfsetispeed`.

89 See the `ioctl_tty(2)` manual page

VI.7.7 Special characters

When a terminal operates in canonical input mode, several special characters are available.

The following table lists the characters for input editing.

`VEOF`	End-of-file character. In canonical mode, causes `read` to return 0 if at the beginning of a line; otherwise flushes the current input buffer to the program
`VEOL`	Additional end-of-line character. Marks the end of input line in canonical mode, similar to `Enter`
`VEOL2`	Secondary end-of-line character. Acts as an additional line terminator in canonical mode
`VERASE`	Erases the previous character in the input buffer (like `Backspace`)
`VWERASE`	Erases the previous word in the input buffer
`VKILL`	Erases the entire current input line (from beginning or last `EOF`)
`VREPRINT`	Reprints the current input line, useful when the terminal display becomes corrupted

Tab 16. Characters for input editing

The following table lists characters that cause signals.

`VINTR`	Interrupt character. Causes the terminal driver to send a `SIGINT` signal to the foreground process group (typically `Ctrl+C`)
`VQUIT`	Quit character. Causes the terminal driver to send a `SIGQUIT` signal, usually terminating the process and producing a core dump (typically `Ctrl+`)
`VSUSP`	Suspend character. Causes the terminal driver to send a `SIGTSTP` signal, stopping (suspending) the process (typically `Ctrl+Z`)
`VDSUSP`	Delayed suspend character. Sends `SIGTSTP` when processed by the application; used for job-control-aware programs (not available on Linux)

Tab 17. Characters generating signals

The following table lists characters for flow control.

`VSTART`	Start character. Resumes output that was previously suspended by the stop character (typically `Ctrl+Q`, `XON`). Used with software flow control (`IXON`)
`VSTOP`	Stop character. Suspends output from the terminal (typically `Ctrl+S`, `XOFF`). Used with software flow control (`IXON`)

Tab 18. Character for flow control

Another special characters is `VLNEXT` (literal next character). It disables special meaning of the next typed character (escape mechanism, typically `Ctrl+V`). The next input byte is taken literally.

VI.7.8 Terminal window size

The term *terminal window size* refers to the number of rows and columns the terminal can display.

Linux provides terminal window size management through the `winsize` structure and `ioctl` calls.

```
#include <asm/termios.h>

struct winsize {
   unsigned short ws_row;    // number of rows
   unsigned short ws_col;    // number of columns
   unsigned short ws_xpixel; // unused
   unsigned short ws_ypixel; // unused
};
```

The main `ioctl` operations are:

`TIOCGWINSZ`
: Get the current terminal window size

`TIOCSWINSZ`
: Set the terminal window size

The following code retrieves and prints the current window size.

When a change occurs in the terminal window size, a `SIGWINCH` signal is generated and delivered to the foreground process group.

The tcwinsz.c program first defines a desired terminal size by filling a `winsize` structure (`ws_row = 30`, `ws_col = 100`). It then applies this configuration to the terminal using `ioctl` with the `TIOCSWINSZ` request.
After updating the terminal size, it immediately retrieves the current terminal dimensions using `TIOCGWINSZ` to confirm the applied values. Finally, it prints the resulting number of rows and columns.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/ioctl.h>

4 int main() {
5     struct winsize ws;

6     ws.ws_row = 30;
7     ws.ws_col = 100;
8     ws.ws_xpixel = 0;
9     ws.ws_ypixel = 0;
```

```
10    if (ioctl(STDOUT_FILENO, TIOCSWINSZ, &ws) == -1) {
11          perror("ioctl TIOCSWINSZ");
12          return 1;
13    }
14    if (ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws) == -1) {
15          perror("ioctl TIOCGWINSZ");
16          return 1;
17    }
18    printf("Terminal size changed to %d:%d\n", ws.ws_row, ws.ws_col);
19    return 0;
20 }
```

---

tcwinsz.c

When `ls` is executed in a terminal, its output adapts to the current terminal width, filling the available columns. After the terminal is resized, the output layout changes accordingly, resulting in a shorter file listing per line.

```
[linux@ipc] ./tcwinsz
Terminal size changed to 30:100

[linux@ipc] ls
...output with shorter file listing per line...
```

The `tcsigwinch.c` program prints a message when the terminal is resized

---

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <signal.h>
4 #include <sys/ioctl.h>

5 void handle_winch(int sig) {
6     struct winsize ws;

7     if (ioctl(STDIN_FILENO, TIOCGWINSZ, &ws) == -1) {
8           perror("ioctl");
9           return;
10    }
11    printf("\nSIGWINCH received: new size = %d rows, %d cols\n",
              ws.ws_row, ws.ws_col);
12    fflush(stdout);
13 }

14 int main() {
15    struct sigaction sa;

16    sa.sa_handler = handle_winch;
17    sigemptyset(&sa.sa_mask);
```

```
18    sa.sa_flags = SA_RESTART;

19    if (sigaction(SIGWINCH, &sa, NULL) == -1) {
20          perror("sigaction");
21          return 1;
22    }

23    printf("Resize the terminal window to trigger SIGWINCH...\n");
24    while (1) {
25          pause();
26    }
27    return 0;
28 }
```

---

tcsigwinch.c

The code should be clear to the reader.
Let’s run the program and resize the terminal.

```
[linux@ipc] ./tcsigwinch
Resize the terminal window to trigger SIGWINCH...

SIGWINCH received: new size = 52 rows, 123 cols

SIGWINCH received: new size = 52 rows, 127 cols

SIGWINCH received: new size = 52 rows, 129 cols

SIGWINCH received: new size = 52 rows, 131 cols
```

When the terminal window is resized, the `SIGWINCH` handler is invoked, and the updated dimensions are obtained and displayed.

VI.7.9 Command-line utilities

A number of command-line utilities are provided for interacting with and managing terminals. The table below lists some of the most commonly used tools. Each utility is documented in a section 1 manual page.

`tty`	Prints the file name of the terminal connected to standard input
`stty`	Gets or sets terminal line settings
`tput`	Uses terminfo database to query terminal capabilities
`reset`	Reinitializes the terminal to a sane state
`clear`	Clears the terminal screen
`script`	Records a terminal session into a file
`screen`	Terminal multiplexer allowing multiple terminal sessions in one window
`tmux`	Modern terminal multiplexer (split panes, sessions, detaching/attaching)
`resize`	Prints or sets terminal window size variables
`infocmp`	Displays terminal capability database entries
`tic`	Compiles terminfo entries into the terminal database
`getty/agetty`	Manages physical or virtual TTY login prompts

Tab 19. Terminal utilities

The following table lists the most commonly used terminal emulators.

`gnome-terminal`	Graphical terminal emulator for GNOME desktop environment
`konsole`	KDE terminal emulator with advanced features like tabs and splits
`xterm`	Classic X11 terminal emulator, highly compatible and minimal
`alacritty`	GPU-accelerated modern terminal emulator focused on performance
`kitty`	Feature-rich GPU-based terminal with graphics support

Table 20. Terminal emulators

The `tty` command prints the file name of the terminal device associated with *standard input*. When invoked on a real terminal, it returns a device such as `/dev/ttyN`, where `N` identifies the terminal number. When invoked within a *pseudoterminal*, it returns a device of the form `/dev/pts/N`, where `N` is the pseudo-terminal number.

VI.7.10 ncurses

The *new curses* library provides a terminal-independent way for programmers to handle keyboard and mouse input and to write output to character-based displays, optimizing updates to reduce unnecessary screen changes.

`ncurses`[90] allows developers to manage the contents of the terminal screen, organize it into windows and pads, handle input events from the keyboard and mouse, and apply visual attributes such as colors, bold text, and underlining.[91] It also supports soft label keys, provides access to the `terminfo` database, includes compatibility with `termcap`, and abstracts system-level terminal interfaces like `termios`.

Since `ncurses` is a complex and feature-rich library, we will only present a simple example.

The `ncurse.c` program uses `ncurses` to display a message.

```
1 #include <ncurses.h>

2 int main() {
3     initscr();
4     printw("Hello, ncurses!");
5     refresh();
6     getch();
7     endwin();
8     return 0;
9 }
```

---

ncurse.c

Line 3. The `initscr` call initializes ncurses.
Line 4. Prints a message at the current cursor position.
Line 5. Refreshes the screen to show the output
Line 6. Waits for user input
Line 7. Ends `ncurses` mode

We must compile passing the `-lncurses` option

```
[linux@ipc] gcc -o ncurse ncurse.c -lncurses
```

Let's execute the program

```
[linux@ipc] ./ncurse
```

90 See the `ncurses(3NCURSES)` manual page
91 The library follows the interface defined by X/Open Curses Issue 7

```
Hello, ncurses!
```

The screen is cleared, and the cursor appears just after the displayed `message`. When `Enter` is pressed, the program terminates and the terminal prompt reappears.

The reader is encouraged to consult the `ncurses(3NCURSES)` manual page for further details and a more comprehensive understanding of its features and usage.

VI.8 Pseudo Terminals

VI.8.1 Overview

A *pseudoterminal* (PTY) is a software-based emulation of a real terminal[92]. It provides terminal-like behavior to programs even though no physical terminal device is involved.
It is composed by a pair of *virtual character devices* that provide a *bidirectional communication channel*. The two endpoints are referred to as the *master* and the *slave*.

Master
: It is controlled by another program like a *terminal emulator*

Slave
: It behaves like a real terminal device to the program running inside it

The *slave end* provides an interface that behaves exactly like a classical terminal. A process that expects to be connected to a terminal, can open the *slave end* of a *pseudoterminal* and then be driven by the program that has opened the *master end*.

Anything that is written on the *master end* is provided to the process on the *slave end* as though it was input typed on a terminal. For example, writing the interrupt character (`Ctrl+C`) to the *master device* would cause an *interrupt signal* (`SIGINT`) to be generated for the *foreground process group* that is connected to the *slave*. Conversely, anything that is written to the *slave end* of the *pseudoterminal* can be read by the process that is connected to the *master end*.

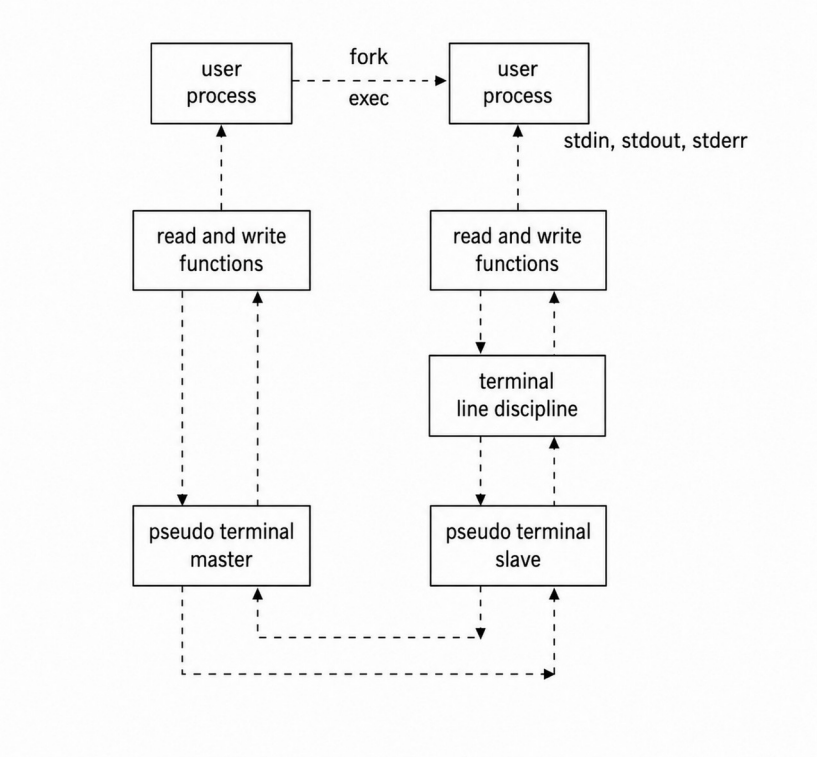

Fig 4. Processes use a pseudoterminal

The process on the left is the *terminal emulator*, such as `xterm`, which forks a child process running a program, typically a shell (`bash`/`zsh`), on the right.

92 See the `pty(7)` manual page

The emulator is connected to the *pseudo-terminal master*, while the child process (shell) is connected to the *pseudo-terminal slave*. The *slave* becomes the *controlling terminal* of the shell.
The *slave* side behaves like a real terminal device, which is why the *terminal line discipline* applies only there, enabling interactive features such as *line editing* and *signal handling*.
The kernel manages the data exchange between the *master* and *slave* ends: data written to the master appears as input to the slave, and data written by the slave appears as output on the master.

To understand the process we provide an example.

1. The user type `ls` and press Enter
2. The terminal emulator captures the keystrokes: `l`, `s`, Enter (‘`\n`’)
3. The emulator writes the 3 characters to the *master end*
4. The kernel forward this data to the *slave* (this looks like typing on a real terminal)
5. The terminal *line discipline* on the *slave* handles the buffering input until `Enter` and editing (`backspace`, `del`, etc..)
6. Once the *newline* is received (`Enter` is pressed), the full “`ls`” is delivered to the shell
7. The shell interprets the command and execute it by calling `fork` to create a child process and `exec` to execute it
8. The output of `ls` is printed on `stdout`, which is connected to the *slave*
9. The kernel passes the output to the *master*
10. The terminal emulator reads the output from the *master* and renders it as text in the window

The process follows the sequence:

Keyboard → Emulator → Master → Slave → Shell → `ls` → Slave → Master → Emulator → Screen

Shell’s `stdin/stdout` are connected to the *PTY slave*, while the emulator interacts with the *PTY master*, not directly with `bash/zsh`.

```
[linux@ipc] tty
/dev/pts/1

[linux@ipc] ls -l
crw------- 1 ipc tty 136, 1 Apr 16 14:59 /dev/pts/1
```

The `/dev/pts/1` file is the pseudoterminal to which the shell is connected. The `zsh` process runs “inside” the `xterm` emulator. Both processes are connected to the same pseudoterminal, with `xterm` attached to the *master end* and `zsh` attached to the *slave end*.
We used the term “inside” but technically both are separate processes: `xterm` creates a *pseudoterminal pair* and forks a child process running `zsh`, attaching `zsh` to the *slave end* while it retains the *master end*.
The `zsh` process thinks it’s talking to a real terminal, in reality `xterm` is handling I/O through the PTY and the communication between the *master* and the *slave* is handled asynchronously.

By using a *windowing system*, multiple windows can each run a separate shell process. Each shell executes within its own terminal window. The *terminal emulator* acts as an intermediary between the shell and the *windowing system*.

VI.8.2 UNIX 98 pseudoterminal

Early *pseudo-terminal APIs* originated from BSD and System V. On Linux, the BSD-style PTY interface is now deprecated, and the standard interface is the UNIX 98 (System V–style) pseudoterminal.

UNIX 98 pseudoterminals[93] use `/dev/ptmx` (*pseudoterminal multiplexer device*) as the *master clone device*, and the corresponding *slave devices* are dynamically created under `/dev/pts/*`. Opening `/dev/ptmx` allocates a new *PTY pair*. The *slave device* is made available in the `/dev/pts` filesystem and can be accessed after being unlocked and granted via the appropriate system calls.

The following code allocates a new *PTY pair* and returns a file descriptor (e.g., `fd = 3`) that refers to the *master*. The kernel creates a corresponding slave device `/dev/pts/n` where `n` is the next available PTY index.

```
int fd = open("/dev/ptmx", O_RDWR);
if (fd < 0)
      perror("open");

printf("FD: %d", fd);
sleep (60);
```

If we inspect `/dev/pts/` before and after the call, we observe a new device file (e.g., `/dev/pts/4`) corresponding to the *slave end* of the newly created pseudo-terminal.

```
[linux@ipc] ls /dev/pts/
total 0
crw------- 1 ipc tty  136, 0 Apr 16 15:24 0
crw------- 1 ipc tty  136, 1 Apr 16 15:39 1
crw------- 1 ipc tty  136, 2 Apr 16 15:39 2
crw------- 1 ipc tty  136, 3 Apr 16 15:39 3
c---------  1 root  root   5, 2 Apr 14 14:59 ptmx

[linux@ipc] ./ptopen &
[1] 508401

[linux@ipc] ls /dev/pts/
total 0
crw------- 1 ipc tty  136, 0 Apr 16 15:24 0
crw------- 1 ipc tty  136, 1 Apr 16 15:40 1
crw------- 1 ipc tty  136, 2 Apr 16 15:39 2
crw------- 1 ipc tty  136, 3 Apr 16 15:39 3
crw------- 1 ipc tty  136, 4 Apr 16 15:40 4
c---------  1 root  root   5, 2 Apr 14 14:59 ptmx
```

The `/dev/ptmx` file is a character file with major number `5` and minor number `2`, usually with mode `0666` and ownership `root:root`. It is used to create a *pseudoterminal master and slave pair.*

93 See the `pts(4)` manual page

When a process opens `/dev/ptmx`, it gets a file descriptor for a *pseudoterminal master* and a *pseudoterminal slave device* is automatically created in the `/dev/pts` directory.

The typical sequence to create and use a UNIX 98 pseudoterminal is:

1. Open the *master clone device*
2. Grant and unlock the slave by calling `grantpt` and `unlockpt` passing the master's file descriptor
3. Obtain the slave device name using `ptsname`
4. Open the slave device

Once both the pseudoterminal *master* and *slave* are open, the *slave end* presents processes with an interface indistinguishable from a real terminal. Data written to the *slave* appears as input on the *master* side, while data written to the *master* is delivered as input to the *slave*.

A program typically forks a child process, which calls `setsid` to create a new session. It then opens the *slave device* and attaches it to *standard input*, *output*, and *error* using `dup2`. Finally, the *slave* becomes the *controlling terminal* of the session.

The kernel imposes a limit on the number of available UNIX 98 pseudoterminals. The limit is dynamically adjustable via the `/proc/sys/kernel/pty/max` file.
In addition, the `/proc/sys/kernel/pty/nr` file indicates how many pseudoterminals are currently in use.

Pseudoterminals are used by applications such as *network login services* (e.g., `SSH`, `rlogin`, `telnet`), *terminal emulators* such as `xterm`, and utilities like `script` and `tmux`.
In *terminal emulators* such as `xterm`, data read from the pseudoterminal master is interpreted and rendered as terminal output, just as a real terminal would display data received from a physical connection.
In *remote login* programs such as `sshd`, data read from the pseudoterminal master is transmitted over the network to a client program, which then displays it on a terminal or terminal emulator.
Pseudoterminals can also be used to provide input to programs that refuse to read from *pipes*, such as `su` and `passwd`, since these programs require a terminal interface.

Linux supports pseudoterminals through the `devpts` *virtual filesystem*, which is mounted at `/dev/pts` and dynamically provides the *slave devices* of pseudoterminals as they are created.

VI.8.3 `script` and `expect`

The `script`[94] program records everything that is input to and output from a *terminal session*. It achieves this by acting as an intermediary between the terminal and the shell it launches. Instead of interacting with the shell directly, all terminal input and output is passed through `script`, which duplicates the data stream. The captured data is stored in a log file in its raw form, preserving exactly what appears on the terminal, including control characters and formatting sequences.

```
[linux@ipc] script
Script started, output log file is 'typescript'.

[linux@ipc] pwd
/home/ipc/Desktop/APLE/Code/Chapter_6

[linux@ipc] ps
   PID TTY          TIME CMD
 139850 pts/4    00:00:00 zsh
 139887 pts/4    00:00:00 ps

[linux@ipc] exit
Script done.

[linux@ipc] file typescript
typescript: Unicode text, UTF-8 text, with very long lines (305), with CRLF, CR, LF line termin
ators, with escape sequences, with overstriking

[linux@ipc] cat typescript

  Script started on 2026-04-20 14:23:20+07:00 [TERM="xterm-256color"
  TTY="/dev/pts/1" COLUMNS="95" LINES="52"]

  [linux@ipc] pwd
  /home/ipc/Desktop/APLE/Code/Chapter_6

  [linux@ipc] ps
      PID TTY          TIME CMD
   139850 pts/4    00:00:00 zsh
   139887 pts/4    00:00:00 ps

  [linux@ipc] exit

  Script done on 2026-04-20 14:23:24+07:00 [COMMAND_EXIT_CODE="0"]
```

As we can see, all terminal input and output is recorded in the log file, including the shell prompt.

The `expect`[95] program automates interactions with interactive applications by following a predefined script. It communicates with programs that normally require user input by simulating that input programmatically. This allows interactive programs to be controlled and executed in a *non-interactive* (automated) mode, making it possible to script workflows that would otherwise require manual intervention.
The reader is encouraged to consult the relevant manual pages for further understanding.

94 See the `script(1)` manual page
95 See the `expect(1)` manual page

VI.8.4 Opening pseudoterminal: `getpt` and `posix_openpt`

The `getpt`[96] function opens a new pseudoterminal device.

```
#define _GNU_SOURCE
#include <stdlib.h>

int getpt(void);

            Returns: on success a file descriptor, on error -1 and errno is set
```

The function returns a file descriptor that refers to the new device. A call to `getpt` is equivalent to open the *pseudoterminal multiplexer device* via `open` as follows

```
open("/dev/ptmx", O_RDWR);
```

The `posix_openpt`[97] function opens a *pseudoterminal master device* and returns a file descriptor.

```
#include <stdlib.h>
#include <fcntl.h>

int posix_openpt(int flags);

          Returns: on success a file descriptor, on failure -1 and errno is set
```

The `flags` argument is a bit mask that ORs zero or more of the following values

`O_RDWR`
: Open the device for both reading and writing

`O_NOCTTY`
: Do not make this device the controlling terminal for the process

`posix_openpt` creates a pathname for the corresponding *pseudoterminal slave device* that can be obtained using `ptsname`. The slave device pathname exists only as long as the *master device*.

`posix_openpt` is the standard POSIX function, while `getpt` is a GNU-specific wrapper around it. When writing portable code we chould use `posix_openpt`.

96 See the `getpt(2)` manual page
97 See the `posix_openpt(3)` manual page

VI.8.5 Unlocking pseudoterminal: `grantpt` and `unlockpt`

Once the pseudoterminal is created we must grant access to the slave device. The `grantpt`[98] function changes the mode and owner of the slave pseudoterminal device corresponding to the master device supplied as an argument.

```
#define _XOPEN_SOURCE
#include <stdlib.h>

int grantpt(int fd);

                              Returns: on success 0, on error -1 and errno is set
```

The master device is passed in the `fd` argument. The user ID of the slave is set to the real UID of the calling process and the group ID is set to an unspecified value such as `tty`. The mode of the slave device is set to `0620`, which is `crw--w---`.

`grantpt` is not *async-signal-safe*, therefore, calling it from within a signal handler, such as for `SIGCHLD`, can lead to undefined behavior.

Once permissions are set, we can unlock the pseudoterminal using the `unlockpt`[99] function, which is defined as follows.

```
#define _XOPEN_SOURCE
#include <stdlib.h>

int unlockpt(int fd);

                              Returns: on success 0, on error -1 and errno is set
```

The `unlockpt` function unlocks the slave pseudoterminal device corresponding to the master device referred to by the `fd` file descriptor. `unlockpt` should be called before opening the slave pseudoterminal device.

98 See the `grantpt(2)` manual page
99 See the `unlock(2)` manual page

VI.8.6 Getting pseudoterminal name: `ptsname`

The `ptsname` and `ptsname_r` functions[100] retrieve the name of the slave pseudoterminal corresponding to master referred to by the file descriptor passed as an argument.

```
#include <stdlib.h>

char *ptsname(int fd);

int ptsname_r(int fd, char buf[size], size_t size);

                        Return: on success a pointer to a string, on failure NULL
                                        on success 0, on failure an error number
```

`ptsname` returns the name of the pseudoterminal device.
`ptsname_r` is the reentrant equivalent and returns the name of the slave device as a null-terminated string in the buffer `buf`. The `size` argment specifies the number of bytes available in the `buf`.

The `openpt.c` program uses the functions introduced so far to create a pseudoterminal and prints the corresponding file descriptor numbers.

```
1 #define _XOPEN_SOURCE 600
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <stdlib.h>

6 int open_pty_pair (int *amaster, int *aslave) {
7     int master, slave;
8     char *name;

9     master = posix_openpt (O_RDWR | O_NOCTTY);
10    if (master < 0)
11          return -1;
12    if (grantpt (master) < 0 || unlockpt (master) < 0) {
13          close(master);
14          return -1;
15    }
16    name = ptsname (master);
17    if (name == NULL) {
18          close(master);
19          return -1;
20    }

21    slave = open (name, O_RDWR);
22    if (slave == -1) {
```

100 See the `ptsname(3)` manual page

```
23          close(master);
24          return -1;
25      }
26      *amaster = master;
27      *aslave = slave;
28      return 0;
29  }

30  int main () {
31      int master, slave;
32      if (open_pty_pair(&master, &slave) == -1) {
33          perror("open_pty_pair");
34          exit(EXIT_FAILURE);
35      }

36      printf("master fd: %d, slave fd: %d\n", master, slave);
37      return 0;
38  }
```

---

openpt.c

The code should be clear to the reader, therefore a detailed analysis is omitted.

Let's execute the program

```
[linux@ipc] ./openpt
master fd: 3, slave fd: 4
```

We can insert a `sleep` call to allow time to inspect the contents of the `/dev/pts` directory while the pseudoterminal is active.

After creating the pseudoterminal, we want the program to execute the program supplied as an argument on the command line. This is done by forking a new process and attaching it to the pseudoterminal. We will implement the following pattern:

In the child process:

1. Close `master`
2. Create a new session using `setsid`
3. Make the slave PTY the controlling terminal using `ioctl(slave, TIOCSCTTY, 0)`
4. Duplicate standard descriptors using `dup2(slave, 0/1/2)`
5. Execute the program through `execvp`

In the parent:

1. Close `slave`
2. Read data from `master`
3. Optionally write data to `master` to send input
4. Close `master`

The `newpt.c` program uses this pattern to execute the program supplied on the command-line.

```c
#define _XOPEN_SOURCE 600
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/ioctl.h>
#include <termios.h>

int open_pty_pair(int *amaster, int *aslave) {
    int master, slave;
    char *name;

    master = posix_openpt(O_RDWR | O_NOCTTY);
    if (master < 0)
        return -1;

    if (grantpt(master) < 0 || unlockpt(master) < 0) {
        close(master);
        return -1;
    }

    name = ptsname(master);
    if (name == NULL) {
        close(master);
        return -1;
    }
    slave = open(name, O_RDWR);
    if (slave == -1) {
        close(master);
        return -1;
    }
    *amaster = master;
    *aslave = slave;
    return 0;
}

int main(int argc, char *argv[]) {
    int master, slave;
    pid_t pid;

    if (argc < 2) {
        fprintf(stderr, "Usage: %s <program> [args...]\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    if (open_pty_pair(&master, &slave) == -1) {
        perror("open_pty_pair");
        exit(EXIT_FAILURE);
    }
```

```
43    pid = fork();
44    if (pid < 0) {
45          perror("fork");
46          exit(EXIT_FAILURE);
47    }

48    if (pid == 0) { // CHILD
49          close(master);
50          if (setsid() < 0) {
51                perror("setsid");
52                exit(EXIT_FAILURE);
53          }
54          if (ioctl(slave, TIOCSCTTY, 0) < 0) {
55                perror("ioctl TIOCSCTTY");
56                exit(EXIT_FAILURE);
57          }

58          dup2(slave, STDIN_FILENO);
59          dup2(slave, STDOUT_FILENO);
60          dup2(slave, STDERR_FILENO);
61          if (slave > 2)
62                close(slave);

63          execvp(argv[1], &argv[1]);
64          perror("execvp");
65          exit(EXIT_FAILURE);
66    }

67    close(slave);
68    printf("child pid: %d\n", pid);
69    printf("master fd: %d\n", master);

70    char buf[256];
71    ssize_t n;
72    while ((n = read(master, buf, sizeof(buf))) > 0) {
73          write(STDOUT_FILENO, buf, n);
74    }
75    close(master);
76    return 0;
77 }
```

---

newpt.c

At line 39, the function `open_pty_pair` is called to create a pseudoterminal pair.
At line 43, a new process is created using `fork`.

In the child, at line 49, the master PTY is closed because it is only used by the parent.
At line 50, `setsid` creates a new session and detaches the process from any controlling terminal.

At line 54 the call

```
54          if (ioctl(slave, TIOCSCTTY, 0) < 0) {
```

assigns the `slave` PTY as the controlling terminal of the child process.

At lines 58–60, `dup2` redirects standard input, output, and error to the `slave` PTY. This ensures all I/O of the program goes through the terminal device.
At line 62, the unused `slave` file descriptor is closed after duplication.
At line 63, `execvp` replaces the child process image with the program passed as argument.

In the parent process, at line 67, the `slave` PTY is closed because the parent only communicates through the master side.
At lines 72–74, the parent continuously reads from the master PTY and writes the data to standard output. This loop is essential because the `master` PTY is the interface through which the parent receives all output from the child process.

Let's execute the program

```
[linux@ipc] ./newpt ls
child pid: 676843
master fd: 3
asyncserv.c     log_client.c        passfd.c                sender.c
bindsock.c      log_client_dgram.c  pc_consumer.c           sendm.c
connect.sh      log_server.c        pc_producer.c           seq_client.c
connection.h    log_server_dgram.c  pc_shm.h                seq_server.c
consock.c       log_server_epoll.c  pshm_ucase.h            shm_send.c
cs_client.c     message.h           pshm_ucase_bounce.c     shmcount.c
cs_server.c     mq_recv.c           pshm_ucase_send.c       shmcount_send.c
cs_shm.h        mq_send.c           ptopen                  socknamed.c
echoclient.c    mqgetattr.c         ptopen.c                socknamedclient.c
echoserver.c    mqnotify.c          ptyfork.c               sockpair.c
fd_fork.c       mqrcvmsg.c          receiver.c              tcattr.c
fd_no_stdout.c  mqsig.c             recvm.c                 tcbreak.c
file.txt        mqsigwait.c         ruptime.c               tccanon.c
getaddrinfo.c   mqsndmsg.c          ruptimed.c              tcline.c
hotobele.c      msgq.c              sem_client.c            tcp_client.c
ipc_client.c    newpt               sem_server.c            tcraw.c
ipc_server.c    newpt.c             semcons.c               tcsigwinch.c
istty.c         open_client.c       semprod.c               tcwinsz.c
istty2.c        open_server.c       semutex.c
link.c          openpt              semutex_multiprocess.c
log.log         openpt.c            semutex_thread.c

[linux@ipc] ./newpt echo Hello
child pid: 677131
master fd: 3
Hello
```

The program supplied on the command-line has been executed.
The current implementation works for executing a single program, but it only supports unidirectional output handling.

To emulate a real terminal, the program must continuously manage bidirectional communication between the user and the child process. User input must be forwarded to the PTY, while child output must be read from the master PTY and rendered back to the user.
Since both streams must be handled concurrently without blocking, an *I/O multiplexing mechanism* such as `select`, `poll`, or `epoll` is required.

We move the `open_pty_pair` function in the `epollnewpt.h` header file alongside with a function to parse the command entered by the user.

```c
#define _GNU_SOURCE
#define _XOPEN_SOURCE 600
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/ioctl.h>
#include <termios.h>
#include <sys/epoll.h>
#include <string.h>
#include <sys/wait.h>

#ifndef SYS_pidfd_open
#define SYS_pidfd_open 434
#endif

#define MAX_EVENTS 15
#define BUF_SIZE 1024

int open_pty_pair(int *amaster, int *aslave) {
      ...
}

void parse_command(char *input, char **argv) {
      char *token = strtok(input, " \n");
      int i = 0;
      while (token != NULL) {
            argv[i++] = token;
            token = strtok(NULL, " \n");
      }
      argv[i] = NULL;
}
```

---

epollnewpt.h

The `epnewpt.c` program uses `epoll` to manage the I/O of the pseudo-terminal.

```
1  #include "epollnewpt.h"

2  static int pidfd_open(pid_t pid) {
3      return syscall(SYS_pidfd_open, pid, 0);
4  }

5  int main() {
6      int master, slave;

7      if (open_pty_pair(&master, &slave) == -1) {
8          perror("open_pty_pair");
9          exit(1);
10     }

11     fcntl(STDIN_FILENO, F_SETFL, O_NONBLOCK);
12     fcntl(master, F_SETFL, O_NONBLOCK);

13     int epoll_fd = epoll_create1(0);
14     if (epoll_fd == -1) {
15         perror("epoll_create1");
16         exit(1);
17     }

18     struct epoll_event ev, events[MAX_EVENTS];

19     ev.events = EPOLLIN;
20     ev.data.fd = STDIN_FILENO;
21     epoll_ctl(epoll_fd, EPOLL_CTL_ADD, STDIN_FILENO, &ev);

22     ev.events = EPOLLIN;
23     ev.data.fd = master;
24     epoll_ctl(epoll_fd, EPOLL_CTL_ADD, master, &ev);

25     printf("PTY-Shell Ready.\n> ");
26     fflush(stdout);

27     char cmd_buf[MAX_CMD];
28     int cmd_ptr = 0;
29     int current_pidfd = -1;
30     pid_t current_pid = -1;

31     while (1) {
32         int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
33         for (int i = 0; i < nfds; i++) {
34             int fd = events[i].data.fd;

35             if (fd == current_pidfd) {
36                 waitpid(current_pid, NULL, 0);
```

```
37                      epoll_ctl(epoll_fd, EPOLL_CTL_DEL, current_pidfd,
                                NULL);
38                      close(current_pidfd);
39                      current_pidfd = -1;
40                      current_pid = -1;
41                      printf("\n> ");
42                      fflush(stdout);
43                      continue;
44              }

45              if (fd == STDIN_FILENO) {
46                      char c;
47                      ssize_t n = read(STDIN_FILENO, &c, 1);
48                      if (n <= 0) continue;
49                      if (c == '\n') {
50                              cmd_buf[cmd_ptr] = '\0';
51                              if (cmd_ptr == 0) {
52                                      printf("> ");
53                                      fflush(stdout);
54                                      continue;
55                              }
56                              if (strcmp(cmd_buf, "exit") == 0) {
57                                      goto cleanup;
58                              }
59                              char *argv[MAX_ARGS];
60                              parse_command(cmd_buf, argv);
61                              pid_t pid = fork();

62                              if (pid == 0) {

63                                      dup2(slave, STDIN_FILENO);
64                                      dup2(slave, STDOUT_FILENO);
65                                      dup2(slave, STDERR_FILENO);
66                                      execvp(argv[0], argv);
67                                      perror("execvp");
68                                      exit(1);
69                              }

70                              current_pid = pid;
71                              current_pidfd = pidfd_open(pid);
72                              if (current_pidfd == -1) {
73                                      perror("pidfd_open");
74                                      exit(1);
75                              }

76                              ev.events = EPOLLIN;
77                              ev.data.fd = current_pidfd;
78                              epoll_ctl(epoll_fd, EPOLL_CTL_ADD,
                                        current_pidfd, &ev);

79                              cmd_ptr = 0;
```

```
80                      } else {
81                              if (cmd_ptr < MAX_CMD - 1) {
82                                      cmd_buf[cmd_ptr++] = c;
83                              }
84                      }
85              }

86              if (fd == master) {
87                      char buf[1024];
88                      while (1) {
89                              ssize_t n = read(master, buf, sizeof(buf));
90                              if (n > 0) {
91                                      write(STDOUT_FILENO, buf, n);
92                              } else if (n == -1 && errno == EAGAIN) {
93                                      break;
94                              } else {
95                                      break;
96                              }
97                      }
98              }
99          }
100   }
101 cleanup:
102   close(master);
103   close(slave);
104   close(epoll_fd);
105 return 0;
106 }
```

---

epnewpt.c

The function `pidfd_open` uses the Linux syscall `SYS_pidfd_open` to obtain a file descriptor that refers to a specific process. This descriptor becomes readable when the process exits.

At line 7 the program creates a pseudo-terminal pair.
At lines 11–12 both `STDIN_FILENO` and `master` are set to non-blocking mode so that `read` operations do not block the event loop.
At line 13, an `epoll` instance is created, and at line 18 event structures are declared.
At lines 19–24 `STDIN_FILENO` is registered for user input and `master` for child output.
At line 25 the program displays a prompt and flushes `stdout`.
The program declares `cmd_buf` to accumulate user input, `cmd_ptr` to represent the current write position, `current_pidfd` to monitor child termination, and `current_pid` to store the process ID used by `waitpid`.
The program enters an infinite loop where `epoll_wait` blocks until at least one monitored file descriptor is ready, and each event is processed independently.
At line 35 the program checks whether the triggered file descriptor matches `current_pidfd`, meaning the child process has exited. In this case, `waitpid` is used to reap the child process, and `epoll_ctl` removes the `pidfd` from the monitored set. At line 38 the `pidfd` is closed and the prompt is displayed again to allow the user to enter a new command.

At line 45 the program checks whether the input comes from `STDIN_FILENO`, meaning the user has entered data.
At lines 46–58 the program reads the input one character at a time, updates the buffer, and checks whether the command is `"exit"`, in which case the program terminates.
At line 59 an array is prepared to store the command arguments, and at line 60 the command is parsed using the helper function.
At line 61 a new child process is created using `fork`.
In the child process, the standard file descriptors are duplicated to the slave device using `dup2`, so that the executed program is attached to the pseudo-terminal. The command is then executed using `execvp`.
In the parent process, the PID returned by `fork` is used to obtain a pidfd via `pidfd_open`. This descriptor is then added to the `epoll` set to monitor the termination of the child process.
At lines 86–98 the program reads from the master device and writes the data to `STDOUT_FILENO`, thus displaying the output produced by the executed command.

Let's execute the program

```
[linux@ipc] ./epnewpt
PTY-Shell Ready.

> ps
    PID TTY          TIME CMD
   4723 pts/1    00:00:22 zsh
  74134 pts/1    00:00:00 epnewpt
  74218 pts/1    00:00:00 ps

> ls
asyncserv.c     istty.c             open_server.c        semutex_multiprocess.c
bindsock.c      istty2.c            openpt               semutex_thread.c
...output...

>
```

As the execution shows, when the command `ps` is entered, the program creates a new child process that is attached to the pseudo-terminal. The output displays the running processes associated with the same terminal (`pts/1`), including the shell (`zsh`), the custom program (`epnewpt`), and the executed command (`ps`). This confirms that the child process is correctly executed within the pseudo-terminal environment.
When the command `ls` is entered, the program again forks a new process, attaches it to the slave side of the pseudo-terminal, and executes the command. The output of `ls` is captured from the master side and printed to `stdout`, showing the list of files in the current directory.
After each command execution, once the child process terminates, the `pidfd` becomes readable, allowing the parent process to detect the termination event, reap the child using `waitpid`, and print a new prompt. This demonstrates that the program correctly handles both process execution and termination in an event-driven manner.

Each command entered by the user is executed in a separate child process. The child process is connected to the pseudo-terminal and the output is forwarded through the master device to the user.

VI.8.7 Pseudoterminal pair: `openpty`, `forkpty` and `loginpty`

As we have seen, a pseudoterminal consists of a pair of devices: a master and a slave. The pattern introduced so far involves several steps, including creating the master, unlocking the slave, and opening the slave device. The `openpty`[101] function combines these steps into a single call. It is defined as follows alongside `forkpty`, as follows.

```
#include <pty.h>

int openpty(int *amaster, int *aslave, char *name, const struct termios
            *termp, const struct winsize *winp);

pid_t forkpty(int *amaster, char *name, const struct termios *termp,
              const struct winsize *winp);

#include <utmp.h>

int login_tty(int fd);

  Return: on success 0, the parent of forkpty the PID of the child, on error -1
                                                              and errno is set
```

The `openpty` function allocates an available pseudoterminal pair and returns file descriptors for the master and slave through `amaster` and `aslave`. If `name` is not `NULL`, it is used to return the pathname of the slave device. If `termp` is not `NULL`, the terminal attributes of the slave are initialized using the provided settings, and if `winp` is not `NULL`, the slave's window size is set accordingly.

In `epnewpt.c` we can replace the `open_pty_pair` function with a call to `openpty`. This simplifies the code by delegating the creation of the pseudo-terminal pair to a standard library function, which internally performs the same operations (`posix_openpt`, `grantpt`, `unlockpt`, `ptsname`, and opening the slave device), while still allowing us to keep a persistent PTY across all commands.

```
7     //if (open_pty_pair(&master, &slave) == -1) {
7     if (openpty(&master, &slave, NULL, NULL, NULL) == -1) {
```

The `login_tty` function prepares a terminal for use as a controlling terminal. It creates a new session, makes the terminal referenced by the file descriptor `fd` the controlling terminal of the calling process, duplicates `fd` onto standard input, output, and error, and then closes the original file descriptor.

The `forkpty` function combines the functionality of `openpty`, `fork`, and `login_tty` to create a new process attached to a pseudoterminal. It returns a file descriptor for the master side in `amaster`. If `name` is not `NULL`, it is used to return the pathname of the slave device. The `termp` and `winp` arguments, if provided, specify the terminal attributes and window size of the slave.

101 See the `openpty(3)` manual page

We can modify the `epnewpt.c` program by using `forkpty`; this is done in the `epnewforkpt.c` program. We replace the `open_pty_pair` call with a call to `forkpty`.
This function internally wraps `posix_openpt`, `grantpt`, `unlockpt`, `ptsname`, and the opening of the slave device. In this way, the setup of the child process is performed in a single call, removing the need for explicit `dup2` calls as well as session and controlling terminal configuration.
In the `epnewpt.c` program, the pseudo-terminal is created once and then reused across all commands. In contrast, in `epnewforkpt.c`, a new pseudo-terminal is created for each command execution.
This difference implies that `epnewpt.c` provides a persistent terminal environment shared across commands, while `epnewforkpt.c` isolates each command in its own terminal instance, resetting the terminal state at every execution.

```c
#include "epollnewpt.h"
#include <pty.h>

static int pidfd_open(pid_t pid) {
    return syscall(SYS_pidfd_open, pid, 0);
}

int main() {
    int master = -1;

    fcntl(STDIN_FILENO, F_SETFL, O_NONBLOCK);
    int epoll_fd = epoll_create1(0);
    if (epoll_fd == -1) {
         perror("epoll_create1");
         exit(1);
    }

    struct epoll_event ev, events[MAX_EVENTS];
    ev.events = EPOLLIN;
    ev.data.fd = STDIN_FILENO;
    epoll_ctl(epoll_fd, EPOLL_CTL_ADD, STDIN_FILENO, &ev);
    printf("PTY-Shell (forkpty) Ready.\n> ");
    fflush(stdout);

    char cmd_buf[MAX_CMD];
    int cmd_ptr = 0;
    int current_pidfd = -1;
    pid_t current_pid = -1;

    while (1) {
         int nfds = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
         for (int i = 0; i < nfds; i++) {
              int fd = events[i].data.fd;
              if (fd == current_pidfd) {
                waitpid(current_pid, NULL, 0);
                epoll_ctl(epoll_fd, EPOLL_CTL_DEL, current_pidfd,NULL);
                close(current_pidfd);
                current_pidfd = -1;
```

```
33                  current_ pid = -1;

34                  if (master != -1) {
35                     epoll_ctl(epoll_fd, EPOLL_CTL_DEL, master, NULL);
36                     close(master);
37                     master = -1;
38                  }
39                  printf("\n> ");
40                  fflush(stdout);
41                  continue;
42               }
43               if (fd == STDIN_FILENO) {
44                     char c;
45                     ssize_t n = read(STDIN_FILENO, &c, 1);
46                     if (n <= 0) continue;
47                     if (c == '\n') {
48                       cmd_buf[cmd_ptr] = '\0';
49                       if (cmd_ptr == 0) {
50                        printf("> ");
51                        fflush(stdout);
52                        continue;
53                       }
54                       if (strcmp(cmd_buf, "exit") == 0) {
55                          goto cleanup;
56                       }
57                       char *argv[MAX_ARGS];
58                       parse_command(cmd_buf, argv);
59                       pid_t pid = forkpty(&master, NULL, NULL, NULL);
60                       if (pid == -1) {
61                          perror("forkpty");
62                          continue;
63                       }
64                       if (pid == 0) {
65                          execvp(argv[0], argv);
66                          perror("execvp");
67                          exit(1);
68                       }
69                       fcntl(master, F_SETFL, O_NONBLOCK);
70                       ev.events = EPOLLIN;
71                       ev.data.fd = master;
72                       epoll_ctl(epoll_fd, EPOLL_CTL_ADD, master, &ev);
73                       current_pid = pid;
74                       current_pidfd = pidfd_open(pid);
75                       if (current_pidfd == -1) {
76                          perror("pidfd_open");
77                          exit(1);
78                       }
79                       ev.events = EPOLLIN;
80                       ev.data.fd = current_pidfd;
81                    epoll_ctl(epoll_fd,EPOLL_CTL_ADD,current_pidfd, &ev);
82                       cmd_ptr = 0;
```

```
83                      } else {
84                          if (cmd_ptr < MAX_CMD - 1) {
85                              cmd_buf[cmd_ptr++] = c;
86                          }
87                      }
88                  }

89                  if (fd == master) {
90                      char buf[1024];
91                      while (1) {
92                          ssize_t n = read(master, buf, sizeof(buf));
93                          if (n > 0) {
94                              write(STDOUT_FILENO, buf, n);
95                          } else if (n == -1 && errno == EAGAIN) {
96                              break;
97                          } else {
98                              break;
99                          }
100                     }
101                 }
102         }
103     }

104     cleanup:
105     if (master != -1) close(master);
106     if (current_pidfd != -1) close(current_pidfd);
107     close(epoll_fd);
108     return 0;
109 }
```

---

epnewforkpt.c

The code should be clear, therefore a detailed analysis is omitted.

Let's run `sleep` using `epnewpt.c` and then `epnewforkpt.c`.

Shell 1

```
[linux@ipc] ./epnewpt
PTY-Shell Ready.
> sleep 30
```

Shell 2

```
[linux@ipc] ps -ef
...output...
ipc     99609    4723  0 11:07 pts/1    00:00:00 ./epnewpt
ipc     99625   99609  0 11:07 pts/1    00:00:00 sleep 30
```

Shell 1

```
[linux@ipc] ./epnewforkpt
PTY-Shell Ready.
> sleep 30
```

Shell 2

```
[linux@ipc] ps -ef
...output…
ipc    100727     4723  0 11:09 pts/1    00:00:00 ./epnewforkpt
ipc    100740  100727  0 11:09 pts/3    00:00:00 sleep 30
```

If we execute `epnewpt.c` and run the command `sleep 30`, the output of `ps` shows that the `sleep` process is attached to the pseudo-terminal `pts/1`, which is the same terminal used by the parent process. This happens because the pseudo-terminal is created once and reused for all commands.
If we execute `epnewforkpt` and run the same command, the `ps` output shows that the `sleep` process is attached to a different pseudo-terminal (for example `pts/3`). This occurs because a new pseudo-terminal is created for each command execution.

The `epnewpt.c` program shares a single terminal context across processes, while `epnewforkpt.c` creates an isolated terminal environment for each executed command.

This behavior comes from the fact that in `epnewpt.c` the pseudo-terminal pair is created once at the beginning of `main` and then reused for all child processes. As a result, every executed command is attached to the same terminal.
In contrast, in `epnewforkpt.c` the pseudo-terminal is created through `forkpty` at each command execution. Since `forkpty` internally creates a new PTY and sets it up for the child process, each command is attached to a different pseudo-terminal.

The standard approach used when implementing a terminal (as in `epnewpt.c`) is to create a single *pseudoterminal* and reuse it for all commands within the session. The *PTY master* is responsible for handling all input/output, while every child process attaches to the same *slave device*. When a command is executed, the shell forks a new process; the child duplicates the *slave* onto its *standard file descriptors* and then executes the program via `execvp`. This shared PTY model is how an interactive shell operates, allowing all processes to participate in the same terminal session.

The second approach, using `forkpty`, creates a new *pseudoterminal* together with a new process for each invocation. While convenient, this pattern is typically used for isolated program execution rather than for building a full shell environment. Each call to `forkpty` spawns a process attached to its own dedicated PTY, resulting in separate and independent terminal sessions per command instead of a shared interactive shell context.

VI.9 Theory and insights

VI.9.1 POSIX Message queues limits files

In the /proc directories the following files contain information about limits and attributes:

/proc/sys/fs/mqueue/msg_default
: This file defines the value used for a new queue's mq_maxmsg setting when the queue is created with a call to mq_open where attr is specified as NULL. The default values is 10. The minimum and maximum are as for /proc/sys/fs/mqueue/msg_max

/proc/sys/fs/mqueue/msg_max
: This file can be used to view and change the ceiling value for the maximum number of messages in a queue. This value acts as a ceiling on the attr->mq_maxmsg argument given to mq_open. The default value is 10. The minimum value is 1 and the upper limit is HARD_MSGMAX. The msg_max limit is ignored for privileged processes (CAP_SYS_RESOURCE), but the HARD_MSGMAX ceiling is nevertheless imposed.

/proc/sys/fs/mqueue/msgsize_default
: This file defines the value used for a new queue's mq_msgsize setting when the queue is created with a call to mq_open where attr is specified as NULL. The default value is 8192 bytes. The minimum and maximum are as for /proc/sys/fs/mqueue/msgsize_max. If msgsize_default exceeds msgsize_max, a new queue's default mq_msgsize value is capped to the msgsize_max limit.

/proc/sys/fs/mqueue/msgsize_max
: This file can be used to view and change the ceiling on the maximum message size. This value acts as a ceiling on the attr->mq_msgsize argument given to mq_open. The default value for msgsize_max is 8192 bytes. The minimum value is 128. The upper limit for msgsize_max is 16,777,216 (HARD_MSGSIZEMAX).
The msgsize_max limit is ignored for privileged process, but, HARD_MSGSIZEMAX ceiling is enforced

/proc/sys/fs/mqueue/queues_max
: This file can be used to view and change the system-wide limit on the number of message queues that can be created. The default value for queues_max is 256. No ceiling is imposed on the queues_max limit; privileged processes can exceed the limit

The RLIMIT_MSGQUEUE resource limit, places a limit on the amount of space that can be consumed by all of the message queues belonging to a process's real user ID.

VI.9.2 System V interprocess communication

Three interprocess communication mechanisms originate from System V. Although POSIX offers a more modern and improved interface, the older System V mechanisms are still used on some systems.

The interface is composed by the following functions[102]:

1. Message queues: `msgget`, `msgsnd`, `msgrcv`, `msgctl`
2. Semaphores: `semget`, `semop`, `semctl`
3. Shared memory: `shmget`, `shmat`, `shmdt`, `shmctl`

Each of these functions has a dedicated manual page in the section 2.

System V ipc is still used because legacy systems depend on it. On Linux it is supported for backward compatibility.

System V IPC mechanisms were introduced in early Unix systems to provide powerful communication tools, but they often resulted in complex and less intuitive interfaces. In contrast, POSIX IPC APIs were designed to improve portability, simplicity, and consistency across systems.
System V IPC uses a key-based identification system, where resources are accessed through integer keys, which can lead to conflicts and require careful management. POSIX IPC, on the other hand, uses a file-like interface with descriptors, making it more intuitive and consistent with standard Unix operations.
Additionally, System V APIs are generally more complex, often requiring multiple steps for tasks such as semaphore operations. POSIX APIs simplify these operations with clearer and more user-friendly function calls.
Finally, System V IPC resources may persist even after processes terminate, requiring explicit cleanup, whereas POSIX IPC provides better lifecycle management, often allowing automatic resource cleanup.

As this book concentrates on Linux programming, System V IPC is not discussed in detail. The reader is encouraged to refer to the manual pages for further information.

102 See the `sysvipc(7)` manual page

VI.9.3 Network interfaces functions

Several functions are available to work with network interfaces.

Retrieving network interface names

The `if_nameindex`[103] function retrieves the network interface names and indexes.

```
#include <net/if.h>

struct if_nameindex *if_nameindex(void);

void if_freenameindex(struct if_nameindex *ptr);

                Returns: a pointer to the array, on error NULL and errno is set
```

The `if_nameindex` function returns a dynamically allocated array of `if_nameindex` structures, each representing a network interface on the local system.
The structure is defined as follows in the net/if.h header file

```
struct if_nameindex
{
   unsigned int if_index;      /* 1, 2, ... */
   char *if_name;             /* null terminated name: "eth0", ... */
};
```

The `if_index` field is the numeric index of the interface.
The `if_name` field is a null-terminated string containing the interface name.

The array is terminated by a special entry where `if_index` is set to `0` and `if_name` is `NULL`.
Since the returned array is allocated dynamically, it must be released using `if_freenameindex` once it is no longer needed to avoid memory leaks.

Mapping names and indexes

The `if_nameotindex` and `if_indextoname` functions[104] perform the mapping between the network interface names and indexes.

```
#include <net/if.h>

unsigned int if_nametoindex(const char *ifname);

char *if_indextoname(unsigned int ifindex, char *ifname);

Returns: on success index of the network interface, on error 0 and errno is set
                              on success ifname, on error NULL and errno is set
```

103 See the `if_nameindex(3)` manual page
104 See the `if_nametoindex(3)` manual page

The `if_nametoindex` function retrieves the interface index associated with the given network interface name (`ifname`).
Conversely, the `if_indextoname` function returns the name of the network interface that matches a specified interface index (`ifindex`). The result is written into the buffer pointed to by `ifname`, which must be large enough to hold at least `IF_NAMESIZE` bytes.

Retrieving addresses

To retrieve the interface address we can use the `getifaddrs`[105] function, which is defined as follows.

```
#include <sys/types.h>
#include <ifaddrs.h>

int getifaddrs(struct ifaddrs **ifap);

void freeifaddrs(struct ifaddrs *ifa);

                              Returns: on success 0, on error -1 and errno is set
```

The `getifaddrs` function builds a linked list of structures that describe the network interfaces on the local system and stores a pointer to the first element in `ifap`. The list is made up of `ifaddrs` structures, which are defined in the `ifaddrs.h` header file as follows:

```
struct ifaddrs {
   struct ifaddrs *ifa_next;       /* Next item in list */
   char *ifa_name;                 /* Name of interface */
   unsigned int ifa_flags;         /* Flags from SIOCGIFFLAGS */
   struct sockaddr *ifa_addr;      /* Address of interface */
   struct sockaddr *ifa_netmask;   /* Netmask of interface */
   union {
      struct sockaddr *ifu_broadaddr; /* Broadcast address of nic */
      struct sockaddr *ifu_dstaddr;   /* Point-to-point dest address */
   } ifa_ifu;
#define ifa_broadaddr ifa_ifu.ifu_broadaddr
#define ifa_dstaddr ifa_ifu.ifu_dstaddr
   void *ifa_data;                 /* Address-specific data */
};
```

The `ifa_next` field points to the next structure in the list, or `NULL` if it is the final element.
The `ifa_name` field holds a null-terminated string representing the interface name.
The `ifa_flags` field contains the interface flags, as provided by the `SIOCGIFFLAGS ioctl` call.
The `ifa_addr` field points to a structure describing the interface address. Its exact format depends on the `sa_family` field within that structure. This pointer may be `NULL`.
The `ifa_netmask` field points to a structure containing the netmask associated with `ifa_addr`, when relevant for the address family. This pointer may also be `NULL`.

105 See the `getifaddrs(3)` manual page

Depending on the flags set in `ifa_flags`, either `ifa_broadaddr` or `ifa_dstaddr` is used.
If `IFF_BROADCAST` is set, `ifa_broadaddr` contains the broadcast address..
If `IFF_POINTOPOINT` is set, `ifa_dstaddr` contains the destination address for a point-to-point interface.
The `ifa_data` field points to additional, address-family-specific data. It may be `NULL` if no such data exists.
The list returned by `getifaddrs` is dynamically allocated and must be freed with `freeifaddrs` when it is no longer needed.

VI.9.4 Virtual consoles: `vcs`

Virtual consoles are used to provide multiple independent text-based terminals on a single machine. They allow to

- Run multiple sessions at once. We can switch between different command-line environments
- Access the system without a graphical interface
- Troubleshoot and recover systems. If the system freezes or the desktop breaks, you can switch to another console with `Ctrl + Alt + F2` to diagnose or fix issues
- Log in as different users simultaneously. Each console can have its own login session.
- View system output directly. Some consoles (like the first one) often display kernel messages or boot logs

In short, virtual consoles are a lightweight way to multitask and manage a system at the terminal level without needing multiple physical screens or windows.

Virtual consoles are represented in the `/dev` filesystem through a set of device files:

`/dev/vcs0` is a *character device*, with *major number* 7 and *minor number* 0, that typically has permissions `0644` and is owned by `root:tty`. It represents the memory contents of the currently active *virtual console*.

`/dev/vcs1` through `/dev/vcs63` are similar character devices for individual virtual consoles, with *minor numbers* 1–63. They also usually have permissions `0644` and ownership `root:tty`. The corresponding `/dev/vcsa0` through `/dev/vcsa63` devices provide the same data, but include additional information such as character attributes. Their data is stored as `unsigned short` values, in *host byte order*, and begins with four values describing the screen: number of lines, number of columns, and the cursor position (`x`, `y`), where (`0`,`0`) is the top-left corner.

If a 512-character font is in use, the 9th bit of character data can be retrieved using the `VT_GETHIFONTMASK` `ioctl` operation on `/dev/tty1`–`/dev/tty63`. The result is returned via the third argument as an `unsigned short`.

These `/dev/vcs*` and `/dev/vcsa*` devices replace older screen-dump `ioctl` operations, allowing administrators to manage access through standard filesystem permissions.

To create device files for the first eight virtual consoles, we can use

```
for x in 0 1 2 3 4 5 6 7 8; do
      mknod -m 644 /dev/vcs$x c 7 $x
      mknod -m 644 /dev/vcsa$x c 7 $((x+128))
done
chown root:tty /dev/vcs*
```

These devices do not support any `ioctl` operations.

The `vcs(4)` manual page includes a sample program that illustrates various features of the virtual console system.

VI.9.5 Serial terminal lines: `ttyS`

Serial terminal lines (`ttyS`) are character devices that represent serial ports on a system, allowing text-based communication over a serial connection.

They are commonly used for

- Direct hardware console access. Connecting to a machine via a physical serial port (e.g., for servers, embedded systems, or routers)
- Remote administration. Using a serial cable or terminal server to manage systems without relying on network services
- System debugging and recovery. Providing a reliable low-level interface for kernel logs, boot messages, or shell access when other interfaces fail
- Communication with external devices. Such as modems, microcontrollers, industrial equipment, or console ports on networking gear

On Linux, they are represented by `/dev/ttyS0` (COM1), `/dev/ttyS1` (COM2) and so on. Each `ttyS` device corresponds to a specific serial hardware port configured by the system.

They are typically created by

```
mknod -m 660 /dev/ttyS0 c 4 64       # base address 0x3f8
mknod -m 660 /dev/ttyS1 c 4 65       # base address 0x2f8
mknod -m 660 /dev/ttyS2 c 4 66       # base address 0x3e8
mknod -m 660 /dev/ttyS3 c 4 67       # base address 0x2e8
chown root:tty /dev/ttyS[0-3]
```

List of tables

List of figures